République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Ecole Supérieure Privée des Technologies de l'Information et de Management de Nabeul

Rapport de Projet de Fin d'Etudes soumis afin d'obtenir le

Diplôme National d'Ingénieur en Génie Logiciel

Réalisé par

Ala Nsir

Conception et Développement d'une Plateforme de Prototypage d'Interfaces Web Intégrant un Chatbot d'Assistance

Encadrant Académique : Mme Dhikra Hamdi **Encadrant Professionnel :** M. wajdi chebane

Projet de Fin d'Etudes fait à

Safran Tunisie

Année Universitaire 2024-2025

Signature des superviseurs

Superviseur professionnel
M.wajdi chebane

SAFRAN

Superviseur Académique
Mme Dhikra Hamdi

it BUSINESS SCHOOL

Remerciements

Je tiens à adresser mes plus vifs et sincères remerciements à mon encadrante académique, Madame Dhikra Hamdi. Son accompagnement rigoureux, sa disponibilité constante et ses conseils précieux ont été des piliers essentiels tout au long de la réalisation de ce projet. Sa confiance et son expertise ont grandement contribué à la réussite de ce travail.
Mes remerciements s'adressent également à Madame Mariem Kamoun pour le temps qu'elle m'a généreusement accordé, ainsi que pour la richesse des informations qu'elle m'a transmises avec attention et bienveillance.
Je tiens à remercier vivement Monsieur Wajdi Chebane et Monsieur Hakim Trabelssi, mes encadrants au sein de l'entreprise, pour leur orientation, leur disponibilité et leur implication dans mon intégration professionnelle.
Je remercie aussi l'ensemble du personnel de la société Safran, pour leur accueil chaleureux, leur aide précieuse et leur esprit de collaboration, qui ont grandement facilité mon immersion dans le milieu professionnel.
Ma gratitude va également à l'administration de l'école, pour l'encadrement pédagogique et l'opportunité offerte de mener à bien ce projet.
Je n'oublie pas d'adresser un merci sincère à tous ceux qui, de près ou de loin, ont contribué à la réussite de ce travail, sans pouvoir tous les citer ici.
Enfin, j'exprime toute ma reconnaissance à ma chère mère, Madame Mouldia Belhaj veuve Nsir, pour son soutien moral et matériel inestimable ainsi qu'à mon épouse, Madame Sirine Aounti pour sa patience, sa compréhension et son appui constant. Je remercie également toute ma famille, en particulier mes sœurs Ines et Ones, dont l'encouragement et la présence ont été essentiels tout au long de ce parcours.

Abstract

This report presents the design and development of Protoweb, a web prototyping application created for Safran. The project aims to streamline communication between technical and non-technical teams by enabling users without programming skills to visually create interface mockups. The solution features a drag-and-drop interface and was developed using Angular for the frontend and Spring Boot for the backend. Furthermore, it integrates an intelligent chatbot to provide real-time assistance and guidance to users, leveraging AI models for enhanced interaction.
Following the Agile Scrum methodology, development was conducted in iterative steps, leading to the deployment on a Safran test server and final validation by end-users. This document details all project phases, from requirements analysis to the delivery of a stable and validated product.

Keywords— Web Prototyping, Drag and Drop, UI Design, Mockups, Angular, Spring Boot, Agile Scrum, AI Chatbot, Large Language Models.

Résumé

Ce rapport présente la conception et le développement de Protoweb, une application web de prototypage réalisée pour Safran. Le projet répond à un besoin de fluidifier la communication entre les équipes techniques et les clients internes, en permettant à des utilisateurs sans compétences en programmation de créer visuellement des maquettes. La solution repose sur une interface par glisser-déposer (drag-and-drop) et a été développée avec Angular pour le frontend et Spring Boot pour le backend. De plus, elle intègre un chatbot intelligent pour offrir une assistance et des conseils en temps réel, s'appuyant sur des modèles d'IA pour une interaction améliorée.
En suivant la méthode Agile Scrum, le développement a été mené étape par étape, jusqu'au déploiement sur un serveur de test et la validation finale par les utilisateurs. Ce document retrace toutes les phases du projet, de l'analyse des besoins à la livraison d'un produit stable et validé.

Mots clés— Prototypage web, Glisser-déposer (Drag and Drop), Conception d'interfaces, Maquettage, Angular, Spring Boot, Agile Scrum, Chatbot IA, Modèles de Langage.

Table des matières

Table des figures **ix**

Liste des tableaux **xii**

Liste des acronymes **xiii**

Introduction générale **1**

1 Cadre général du projet **2**
1.1 Introduction 2
1.2 Présentation de l'organisme d'accueil 2
1.2.1 Présentation générale du groupe Safran 2
1.2.2 Présentation du Safran Tunisie 2
1.3 Présentation du projet 3
1.3.1 Contexte du projet 3
1.3.2 Étude de l'existant 4
1.3.2.1 Analyse de l'existant 4
1.3.2.2 Critiques de l'existant 4
1.3.2.3 Solution proposée 6
1.4 Choix méthodologique 6
1.4.1 Approches de gestion de projet 7
1.4.1.1 Approche classique 7
1.4.1.2 Approche Agile 7
1.4.1.3 Étude comparative entre l'approche classique et l'approche agile . . . 7
1.4.1.4 Choix d'approche de gestion du projet 8
1.4.2 Méthodologie Agile adoptée : Scrum 8
1.4.2.1 Rôles clés dans Scrum. 8
1.4.2.2 Artefacts et événements de Scrum. 8
1.5 Conclusion 9

2 État de l'art et Spécification des besoins **10**
2.1 Introduction 10
2.2 État de l'art 10
2.2.1 Intelligence Artificielle 11
2.2.2 Apprentissage Automatique 11
2.2.3 Apprentissage profond 11
2.2.4 Traitement du Langage Naturel 11
2.2.5 Chatbots et leurs différentes approches 11

2.2.5.1 Types de chatbots . . . 12
2.2.5.2 Comparaison des Approches de Chatbots . . . 12
2.2.6 Intelligence Artificielle Générative . . . 12
2.2.7 Modèles de Langage Génératifs (LLMs) . . . 12
2.2.8 Outils et Plateformes pour le Développement de Chatbots IA . . . 13
2.3 Spécification des besoins . . . 13
2.3.1 Identification des acteurs . . . 13
2.3.2 Besoins Fonctionnels . . . 13
2.3.3 Besoins non fonctionnels . . . 14
2.3.4 Vue architecturale des cas d'utilisations . . . 15
2.3.5 Planification . . . 16
2.3.5.1 Equipe et rôle . . . 16
2.3.5.2 Product Backlog . . . 17
2.3.5.3 Processus de livraison . . . 17
2.3.5.4 Chronogramme de Planification . . . 18
2.4 Environnement de travail . . . 19
2.4.1 Environnement matériel . . . 19
2.4.2 Environnement logiciel . . . 19
2.4.2.1 Outils de conception . . . 19
2.4.2.2 Outils de développement . . . 20
2.4.2.2.1 Technologies pour le Frontend . . . 20
2.4.2.2.2 Technologies pour le Backend . . . 21
2.4.2.2.3 Outils de gestion des données et des tests . . . 22
2.4.3 Choix technologiques . . . 22
2.4.3.1 Choix du moteur : le Backend . . . 22
2.4.3.2 Choix de l'interface : le Frontend . . . 23
2.5 Notre architecture pour Protoweb . . . 24
2.6 Conclusion . . . 25

3 Release 1 : Mise en place des Fondations Fonctionnelles de la Plateforme 26
3.1 Introduction . . . 26
3.2 Sprint 1 : Mise en place de l'authentification, de la gestion des utilisateurs et des profils 26
3.2.1 Présentation du Sprint 1 . . . 26
3.2.1.1 Sprint Backlog . . . 27
3.2.1.2 Diagramme de cas d'utilisation global du Sprint 1 . . . 27
3.2.1.3 Diagramme de classe du Sprint 1 . . . 28
3.2.2 Raffinement des diagrammes de cas d'utilisation . . . 29
3.2.2.1 Analyse de cas "s'authentifier" . . . 29
3.2.2.1.1 Description textuelle de Cas d'Utilisation : Authentification 29
3.2.2.1.2 Diagramme de Séquence : Authentification . . . 31
3.2.2.2 Analyse de cas " Créer utilisateur " . . . 31
3.2.2.2.1 Description textuelle de Cas d'Utilisation : Créer utilisateur 31
3.2.2.2.2 Diagramme de Séquence : Création d'un utilisateur . . . 32
3.2.2.3 Analyse de cas " Modifier profil " . . . 33
3.2.2.3.1 Description textuelle de cas d'utilisation : Modifier profil . . 33
3.2.2.3.2 Diagramme de Séquence : Changer le Mot de Passe . . . 34
3.2.3 Réalisation du Sprint 1 . . . 35

3.2.4 Phase de Test . . . 37
3.3 Sprint 2 : Conception et gestion des projets ainsi que l'intégration des modèles (templates) . . . 39
3.3.1 Présentation du Sprint 2 . . . 39
3.3.2 Spécification fonctionnelle . . . 39
3.3.2.1 Sprint Backlog . . . 39
3.3.2.2 Diagramme de cas d'utilisation global du Sprint 2 . . . 40
3.3.2.3 Diagramme de classe du Sprint 2 . . . 41
3.3.3 Raffinement des diagrammes de cas d'utilisation . . . 42
3.3.3.1 Analyse de cas " Ajouter un projet à partir d'un modèle" . . . 42
3.3.3.1.1 Description textuelle de Cas d'Utilisation : Ajouter un projet à partir d'un modèle . . . 42
3.3.3.1.2 Diagramme de Séquence : Ajouter un projet à partir d'un modèle . . . 43
3.3.3.2 Analyse de cas " Retirer un modèle de la liste" . . . 44
3.3.3.2.1 Description textuelle de Cas d'Utilisation : Retirer un modèle de la liste . . . 44
3.3.3.2.2 Diagramme de Séquence : Retirer un modèle de la liste . . . 45
3.3.4 Réalisation du Sprint 2 . . . 46
3.3.5 Phase de Test . . . 48
3.4 Sprint 3 : Développement de l'éditeur visuel avec glisser-déposer et personnalisation des composants . . . 50
3.4.1 Présentation du Sprint 3 . . . 50
3.4.2 Spécification fonctionnelle . . . 50
3.4.2.1 Sprint Backlog . . . 50
3.4.2.2 Diagramme de cas d'utilisation global du Sprint 3 . . . 52
3.4.2.3 Diagramme de classe du Sprint 3 . . . 52
3.4.3 Raffinement des diagrammes de cas d'utilisation . . . 53
3.4.3.1 Analyse de cas "Glisser-déposer d'un élément" . . . 53
3.4.3.1.1 Description textuelle de Cas d'Utilisation : Glisser-déposer d'un élément . . . 53
3.4.3.1.2 Diagramme de Séquence : Glisser-déposer d'un élément . . . 54
3.4.3.2 Analyse de cas " Personnalisation d'un élément" . . . 55
3.4.3.2.1 Description textuelle de Cas d'Utilisation : Personnalisation d'un élément . . . 55
3.4.3.2.2 Diagramme de Séquence : Personnalisation d'un élément . . . 56
3.4.4 Réalisation du Sprint 3 . . . 56
3.4.5 Phase de Test . . . 57
3.5 Sprint 4 : Implémentation du partage des projets et du module de commentaires en temps réel . . . 58
3.5.1 Présentation du Sprint 4 . . . 58
3.5.2 Spécification fonctionnelle . . . 58
3.5.2.1 Sprint Backlog . . . 58
3.5.2.2 Diagramme de cas d'utilisation global du Sprint 4 . . . 59
3.5.2.3 Diagramme de classe du Sprint 4 . . . 60
3.5.3 Raffinement des diagrammes de cas d'utilisation . . . 61
3.5.3.1 Analyse de cas "Partager un projet" . . . 62

3.5.3.1.1 Description textuelle de Cas d'Utilisation : Partager un projet 62
3.5.3.1.2 Diagramme de Séquence : Partager un projet 62
3.5.3.2 Analyse de cas "Ajouter un commentaire" 63
3.5.3.2.1 Description textuelle de Cas d'Utilisation : Ajouter un commentaire 63
3.5.3.2.2 Diagramme de Séquence : Ajouter un commentaire 64
3.5.4 Réalisation du Sprint 4 64
3.5.5 Phase de Test 66
3.6 Sprint 5 : Exportation et Obtention du Code Source 68
3.6.1 Présentation du Sprint 5 68
3.6.2 Spécification fonctionnelle 68
3.6.2.1 Sprint Backlog 68
3.6.2.2 Diagramme de cas d'utilisation global du Sprint 5 68
3.6.3 Raffinement des diagrammes de cas d'utilisation 69
3.6.3.1 Analyse de cas "Exporter une maquette" 69
3.6.3.1.1 Description textuelle de Cas d'Utilisation : Exporter une maquette 69
3.6.3.1.2 Diagramme de Séquence : Exportation d'une maquette . . . 70
3.6.3.2 Analyse de cas "obtenir le code source" 70
3.6.3.2.1 Description textuelle de Cas d'Utilisation : Obtenir le code source 71
3.6.3.2.2 Diagramme de Séquence : obtention du code source 71
3.6.3.3 Réalisation du Sprint 5 72
3.6.3.4 Phase de test 73
3.7 Sprint 6 : Conception et développement du tableau de bord statistique 73
3.7.1 Présentation du Sprint 6 73
3.7.2 Diagramme de cas d'utilisation global du Sprint 6 74
3.7.3 Raffinement des diagrammes de cas d'utilisation 74
3.7.3.1 Analyse de cas "Consulter les statistiques" 74
3.7.3.1.1 Description textuelle de Cas d'Utilisation : Consulter les statistiques 74
3.7.4 Diagramme de séquence de la consultation des statistiques 75
3.7.5 Réalisation des Interfaces Graphiques 76
3.7.6 Phase de test - Sprint 6 77
3.8 Conclusion 78

4 Release 2 : Intégration du Chatbot IA et Finalisation de la Plateforme 79
4.1 Introduction 79
4.2 Sprint 1 :assistance protypage via chatbot 79
4.2.1 Présentation du Sprint 1 79
4.2.2 Choix technologiques du Chatbot 80
4.2.2.1 Comparaison des modèles d'IA dans les applications de prototypage et de support 80
4.2.2.2 Les approches : Cloud vs. Auto-hébergement 80
4.2.2.3 Tableau comparatif des approches 80
4.2.2.4 Modèles et déploiement des solutions auto-hébergées 81
4.2.2.5 Notre architecture pour Chatbot 81

4.2.2.6 Décision et justification pour Protoweb 82
4.2.3 Spécification fonctionnelle 82
4.2.3.1 Sprint Backlog 82
4.2.3.2 Diagramme de cas d'utilisation global du Sprint 1 83
4.2.4 Raffinement des diagrammes de cas d'utilisation 83
4.2.4.1 Analyse de cas "Interagir avec le Chatbot" 83
4.2.4.1.1 Description textuelle de Cas d'Utilisation : Interagir avec le Chatbot 83
4.2.5 Réalisation du Sprint 1 84
4.2.6 Phase de Test 86
4.2.6.1 Résultats des tests 87
4.3 Sprint 2 : Exécution des tests finaux et préparation du déploiement 88
4.3.1 Présentation du Sprint 2 88
4.3.2 Spécification fonctionnelle 89
4.3.2.1 Sprint Backlog 89
4.3.3 Processus de Réalisation 89
4.3.3.1 Phase de Stabilisation Pré-Déploiement 89
4.3.3.2 Déploiement sur le Serveur de Test 90
4.3.3.3 Campagne de Tests Finaux et Validation Finale du Projet 90
4.3.3.3.1 Résultats et Validation Finale : 90
4.4 Conclusion 91

Conclusion **92**

Annexes **94**

Bibliographie **98**

Table des figures

1.1 Safran dans le monde.[17] . . . 3
1.2 Logo de Safran. [17] . . . 3
1.3 La solution de prototypage Adobe xD [2] . . . 4
1.4 La solution de prototypage Figma [8]. . . . 5
1.5 Notre solution de prototypage Protoweb. . . . 6
1.6 Le cycle de la méthodologie Agile Scrum [21]. . . . 9

2.1 Schéma des relations entre les domaines de l'Intelligence Artificielle[12]. . . . 10
2.2 Diagramme de cas d'utilisation global. . . . 16
2.3 Diagramme de Gantt du projet détaillé . . . 19
2.4 Logo de Figma. . . . 20
2.5 Logo de Drawio. . . . 20
2.6 Logo de Visual Studio Code [**?**]. . . . 20
2.7 Logo d'Angular [**?**]. . . . 21
2.8 Logo d'IntelliJ IDEA. . . . 21
2.9 Logo de Spring Boot .[16] . . . 21
2.10 Logo de Microsoft SQL Server. [**?**] . . . 22
2.11 Logo de Postman [14]. . . . 22
2.12 Architecture technique de l'application Protoweb. . . . 25

3.1 Diagramme de cas d'utilisation de gestion complète des utilisateurs . . . 28
3.2 Diagramme de classes – Sprint 1 . . . 29
3.3 Diagramme de séquence d'authentification . . . 31
3.4 Diagramme de séquence de création d'un utilisateur . . . 33
3.5 Diagramme de séquence de changer le mot de passe . . . 35
3.6 Interface de Connexion. . . . 35
3.7 Interface de gestion des projets. . . . 36
3.8 Interface de gestion des projets. . . . 36
3.9 Test Postman – Authentification réussie . . . 37
3.10 Test Postman – Création d'un utilisateur. . . . 38
3.11 Erreur d'authentification - Identifiants incorrects . . . 38
3.12 Notification d'erreur - Champs de mot de passe incomplets . . . 38
3.13 Diagramme de cas d'utilisation de gestion complète des projets et modèles. . . . 41
3.14 Structure des classes – Sprint 2. . . . 42
3.15 Diagramme de séquence d'ajouter un projet à partir d'un modèle . . . 44
3.16 Diagramme de séquence de retrait d'un modèle. . . . 46
3.17 Interface de gestion des projets. . . . 46

3.18 Interface de gestion des modèles existants. 47
3.19 Interface d'ajout de projet à partir d'un modèle. 47
3.20 Interface de confirmation pour retirer un projet du statut de modèle. 48
3.21 Test Postman – Nouvelle création de projet 49
3.22 Test Postman – Modification du statut du projet 49
3.23 Notification de succès - Suppression de projet 49
3.24 Notification d'action annulée - Suppression de projet 50
3.25 Notification d'erreur - Champs vide 50
3.26 Diagramme de cas d'utilisation de la gestion des éléments d'une maquette. 52
3.27 Structure des classes – Sprint 3. 53
3.28 Diagramme de séquence de glisser-déposer d'un élément 55
3.29 Diagramme de séquence de personnalisation d'un élément 56
3.30 Interface de l'éditeur avec bibliothèque d'éléments et zone de glisser-déposer. 57
3.31 Composant d'interface dédié à la personnalisation d'un élément. 57
3.32 Diagramme de cas d'utilisation de la collaboration 60
3.33 Structure des classes – Sprint 4. 61
3.34 Diagramme de séquence du partage d'un projet. 63
3.35 Diagramme de séquence d'ajouter d'un commentaire. 64
3.36 Interface de partage d'un projet. 65
3.37 Panneau de commentaires intégré à l'éditeur. 65
3.38 Test Postman – Partage de projet. 66
3.39 Test Postman – Ajout de commentaire. 67
3.40 Interface affichant les droits insuffisants pour la suppression d'un projet par un collaborateur. 67
3.41 Message d'erreur lors d'une tentative de suppression sans les droits requis. 67
3.42 Notification d'erreur - Action non autorisée 68
3.43 Diagramme de cas d'utilisation : Exportation des maquettes et obtention de code source. 69
3.44 Diagramme de séquence – Exportation d'une maquette. 70
3.45 Diagramme de séquence – Obtention de code source. 72
3.46 Gestion des projets – intégration des options d'exportation et génération de code source. 72
3.47 Menu déroulant d'exportation – choix du format PDF, PNG ou JPG. 72
3.48 Exemple de fichiers générés (HTML et CSS). 73
3.49 Diagramme de cas d'utilisation : Consultation du tableau de bord statistique. 74
3.50 Diagramme de séquence de la consultation des statistiques. 75
3.51 Vue d'ensemble du tableau de bord administrateur avec statistiques clés et actions rapides. 76
3.52 Détail de l'interface du tableau de bord administrateur - Liste des derniers utilisateurs inscrits. 76
3.53 Graphiques de répartition des utilisateurs et des projets par département. 77
3.54 Test Postman – Récupération de nombre d'Utilisateurs Actifs sur 24 Heures. 77
3.55 Test Postman –Monitoring des Utilisateurs Actifs. 78

4.1 Architecture d'intégration du Chatbot auto-hébergé dans Protoweb. 82
4.2 Diagramme de cas d'utilisation du Chatbot 83
4.3 Interface du manual prompt. 85
4.4 Interface du Chatbot intégré. 86
4.5 Liste des questions de test. 87

4.6 CLI et Interface Graphique d'Ollama. 88

Liste des tableaux

1.1 Tableau de comparaison simplifié entre Adobe XD et Figma 5
1.2 Comparaison entre l'approche classique et l'approche Agile. 7

2.1 Comparaison des technologies de chatbots. 12
2.2 Affectation des rôles 16
2.3 Backlog du produit (Product Backlog) 17
2.4 Plan de livraison avec sprints 18
2.5 Comparaison simple des technologies pour le Backend 23
2.6 Comparaison simple des technologies pour le Frontend 24

3.1 Backlog du Sprint 1 27
3.2 Description du scénario « Authentification » 30
3.3 Description textuelle du scénario « Créer un utilisateur » 31
3.4 Description du scénario « Changer son mot de passe » 34
3.5 Backlog du Sprint 2 40
3.6 Description du scénario « Ajouter un projet à partir d'un modèle » 43
3.7 Description du scénario « Retirer un modèle de la liste » 45
3.8 Backlog du Sprint 3 51
3.9 Description du scénario « Glisser-déposer et enregistrer des éléments » 54
3.10 Description du scénario « Personnaliser un élément » 55
3.11 Backlog du Sprint 4 59
3.12 Description du scénario « Partager un projet » 62
3.13 Description du scénario « Ajouter un commentaire » 63
3.14 Backlog du Sprint 5 : Détail des User Stories 68
3.15 Description du scénario « Exporter une maquette » 69
3.16 Description du scénario « Obtenir le code source » 71
3.17 Backlog du Sprint 6 74
3.18 Description du scénario « Consulter les statistiques » 75

4.1 Applications de prototypage et modèles d'IA associés 80
4.2 Comparaison des approches Cloud et Auto-hébergée pour le chatbot 81
4.3 Backlog du Sprint 7 82
4.4 Description du scénario « Interagir avec le Chatbot » 83
4.5 Caractéristiques et critères de réussite du chatbot 86
4.6 Backlog du Sprint 8 89
7 Tableau des routes principales de l'API Protoweb 95

Liste des acronymes

Acronyme	Définition
API	Application Programming Interface
NLP	Natural Language Processing
AI	Intelligence Artificielle
ML	Machine Learning
LLMs	Large Language Models
GenAI	Intelligence Artificielle Générative
PNG	Portable Network Graphics
JPG	Joint Photographic Experts Group
US	User Story
VS Code	Visual Studio Code
SQL	Structured Query Language
JSX	JavaScript XML
REST	Representational State Transfer (API REST)
SGBD	Système de Gestion de Base de Données
PDF	Portable Document Format
ITBS	IT Business School
UI/UX	User Interface / User Experience
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
UAT	Test d'acceptation utilisateur
GPT	Generative Pre-trained Transformer
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
IT	Technologies de l'Information
URL	Uniform Resource Locator

Introduction générale

De nos jours, la création d'applications évolue à un rythme soutenu, imposant l'adoption de méthodes de conception toujours plus performantes, intuitives et accessibles, même pour des utilisateurs sans compétences en programmation. Face à cette évolution, de nombreuses solutions de prototypage et de construction d'interfaces par glisser-déposer (drag and drop) ont vu le jour, offrant la possibilité de concevoir visuellement des applications, des sites web ou d'autres systèmes sans écrire de code, ce qui permet de raccourcir considérablement les délais de développement tout en facilitant la collaboration entre équipes techniques et métiers.
Le recours au glisser-déposer joue aujourd'hui un rôle déterminant dans le cycle de développement logiciel, puisqu'il permet de tester et de concrétiser rapidement de nouvelles idées, de rendre la conception d'interfaces accessible aux personnes non spécialistes du développement, d'accélérer la validation des concepts auprès des utilisateurs grâce à des prototypes interactifs et de réduire significativement les coûts liés aux retouches avant la phase de développement final.
C'est dans ce contexte que s'inscrit notre projet, qui vise à développer une plateforme dédiée à la création d'interfaces par glisser-déposer, avec l'ambition de proposer un outil intuitif, flexible et capable de répondre aux besoins de prototypage rapide sans nécessiter de compétences techniques approfondies.
Notre solution permettra ainsi aux utilisateurs de sélectionner et configurer des composants prédéfinis, d'adapter facilement leurs propriétés et comportements, de générer automatiquement une structure fonctionnelle exploitable dans des applications concrètes, et de collaborer en temps réel grâce au partage des maquettes, favorisant le travail d'équipe et les retours immédiats. Le projet sera conduit en plusieurs phases, allant de la conception globale de l'architecture du système jusqu'au développement de l'éditeur visuel et son intégration avec des outils de génération de maquettes interactives et de bases de code, mes responsabilités incluant la mise en place d'un système d'authentification, la gestion et l'administration des comptes utilisateurs, la conception d'une bibliothèque de composants réutilisables, la personnalisation des éléments selon les besoins, le développement des fonctionnalités de partage en temps réel, l'exportation des maquettes au format image et PDF, la génération automatique du code à partir des prototypes, ainsi que l'élaboration de tableaux de bord interactifs.

Chapitre 1

Cadre général du projet

1.1 Introduction

Ce premier chapitre, structuré en trois sections, vise à exposer le cadre général de ce projet. La première section sera dédiée à la présentation de l'organisme d'accueil. La seconde abordera le travail requis, en commençant par l'analyse et l'étude de l'existant, puis par la proposition d'une solution pour répondre à la problématique soulevée. Enfin, la troisième section détaillera la méthodologie adoptée pour mener à bien ce projet, en expliquant les différentes étapes suivies afin d'atteindre les objectifs fixés.

1.2 Présentation de l'organisme d'accueil

Cette section est consacrée à la présentation de l'organisme d'accueil, le groupe Safran, où s'est déroulé notre projet de fin d'études. Nous décrirons sa structure et ses activités au niveau mondial, avant de nous intéresser plus spécifiquement à son entité tunisienne dans le cadre de notre stage.

1.2.1 Présentation générale du groupe Safran

Safran[17] est une entreprise internationale spécialisée dans les technologies de pointe intervenant principalement dans les secteurs de l'aéronautique propulsion, équipements et aménagements, de l'espace et de la défense. Seul ou en collaboration avec des partenaires, le groupe occupe une position de leader à l'échelle mondiale ou européenne dans ses domaines d'activité. Créé en 2003 et présent sur les cinq continents, Safran dispose de plus de 300 sites répartis dans 30 pays et emploie plus de 95 000 personnes, avec un chiffre d'affaires atteignant 24,6 milliards d'euros en 2019.
L'entreprise propose une large gamme d'équipements destinés aux aéronefs des principaux constructeurs, qu'ils soient civils ou militaires. Son expertise repose sur une parfaite maîtrise des technologies avancées, l'utilisation de matériaux innovants et un savoir-faire reconnu.

1.2.2 Présentation du Safran Tunisie

Anciennement connue sous le nom de Zodiac Aerospace, la société Safran s'est implantée en Tunisie en janvier 2003. Depuis, ses anciennes activités ont été réparties entre trois nouvelles entités du groupe : Safran Aerosystems, Safran Cabin et Safran Seats.

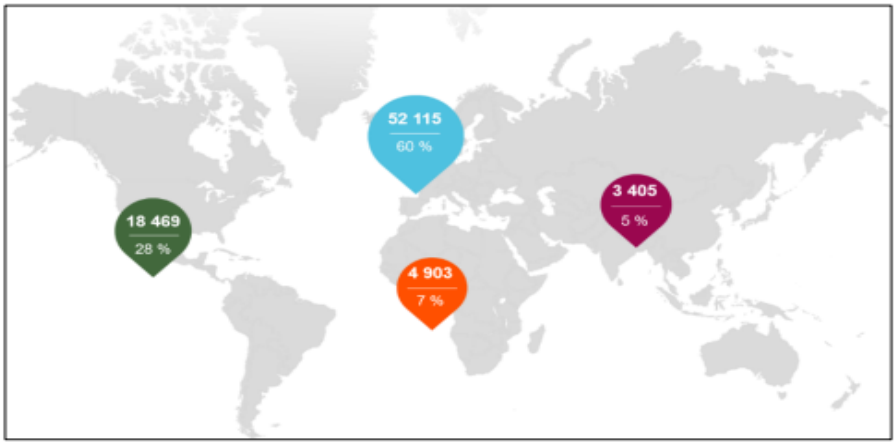


FIGURE 1.1 – Safran dans le monde.[17]

Safran Tunisie compte 2 553 employés répartis sur trois sites : deux situés à Soliman et un à Grombalia. L'entreprise développe principalement la conception de systèmes embarqués, d'équipements aéronautiques ainsi que d'aménagements intérieurs pour avions.
Ce stage de fin d'études s'est déroulé sur le site de Safran Dhari, à Soliman.

FIGURE 1.2 – Logo de Safran. [17]

1.3 Présentation du projet

Après avoir présenté l'organisme d'accueil au sein duquel nous avons réalisé notre projet de fin d'études, nous décrivons le contexte du projet et notre solution proposée. Le stage est effectué dans le cadre de la réalisation du projet de fin d'études, afin d'obtenir le diplôme national d'ingénierie en informatique, spécialité Génie Logiciel à ITBS.

1.3.1 Contexte du projet

La méthode actuelle de conception des interfaces utilisateur chez Safran, bien que fonctionnelle, fait face à un obstacle majeur lors des échanges entre les équipes de design et les clients internes. Ces derniers, n'ayant pas de profil technique, ont souvent du mal à décrire leurs attentes avec précision. Cette difficulté de communication engendre des malentendus et oblige les designers à multiplier les versions de maquettes sur des logiciels experts comme Figma [8] ou Adobe XD [2], ce qui ralentit considérablement les projets et augmente les coûts.

Le défi de ce projet est donc de repenser ce processus. Il s'agit de trouver une solution pour **fluidifier la communication et accélérer la phase de prototypage en la rendant moins dépendante des compétences techniques**. L'ambition est de proposer une plateforme web intuitive qui permettrait à tous les collaborateurs de Safran, même sans connaissances en design, de construire eux-mêmes une première ébauche de leurs interfaces. Ce projet de fin d'études a pour but de développer un tel outil afin de répondre concrètement à ce besoin d'efficacité.

1.3.2 Étude de l'existant

Dans cette partie du chapitre, nous allons présenter le projet, analyser l'existant en repérant ses limites, puis proposer une solution adaptée aux besoins. L'étude de l'existant permet de mieux comprendre les problèmes et de choisir les solutions les plus appropriées.

1.3.2.1 Analyse de l'existant

Dans cette section, nous étudions les solutions et méthodes actuellement utilisées. Cette analyse permet de comprendre comment les besoins sont pris en charge aujourd'hui et d'identifier les limites des approches existantes.
Chez Safran, les équipes de développement et de design ui/ux utilisent principalement des outils comme Figma [8] et Adobe XD [2] pour concevoir des maquettes d'applications web. Le processus commence par des échanges avec les clients internes pour comprendre et recueillir leurs besoins. Dans un premier temps, les designers UI/UX réalisent des esquisses de wireframes sur papier pour donner une première représentation des interfaces. Ces croquis sont ensuite digitalisés sur Figma afin d'être affinés et transformés en maquettes interactives.
Cependant, cette approche présente certaines limites, notamment en termes de précision et de clarté des besoins exprimés, ce qui peut entraîner des incompréhensions ou des allers-retours fréquents.

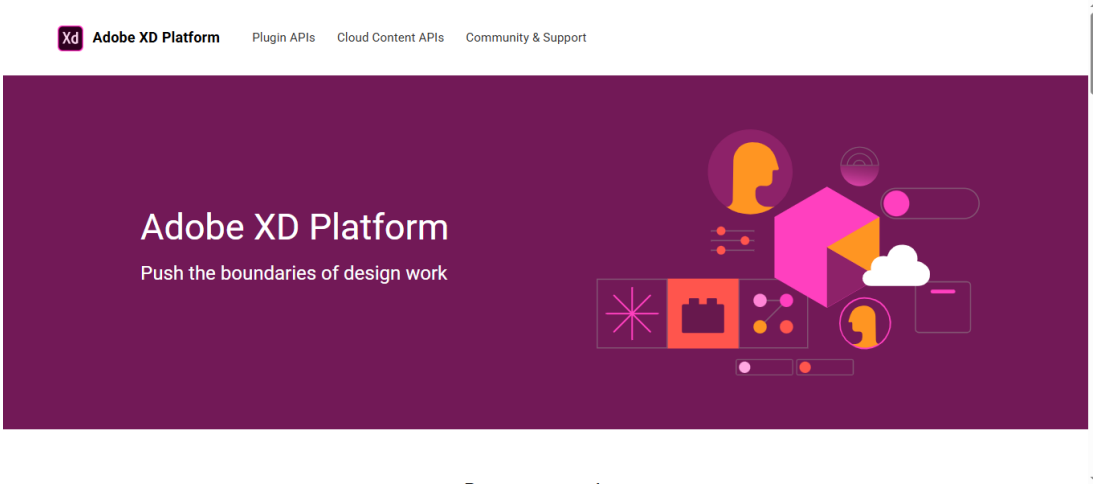


FIGURE 1.3 – La solution de prototypage Adobe xD [2]

1.3.2.2 Critiques de l'existant

Après notre analyse, plusieurs limites des solutions actuelles ont été identifiées. L'une des principales difficultés concerne la formulation des besoins par les clients, qui peinent souvent à exprimer leurs attentes de manière précise et détaillée. Cette imprécision entraîne des ambiguïtés dans la conception

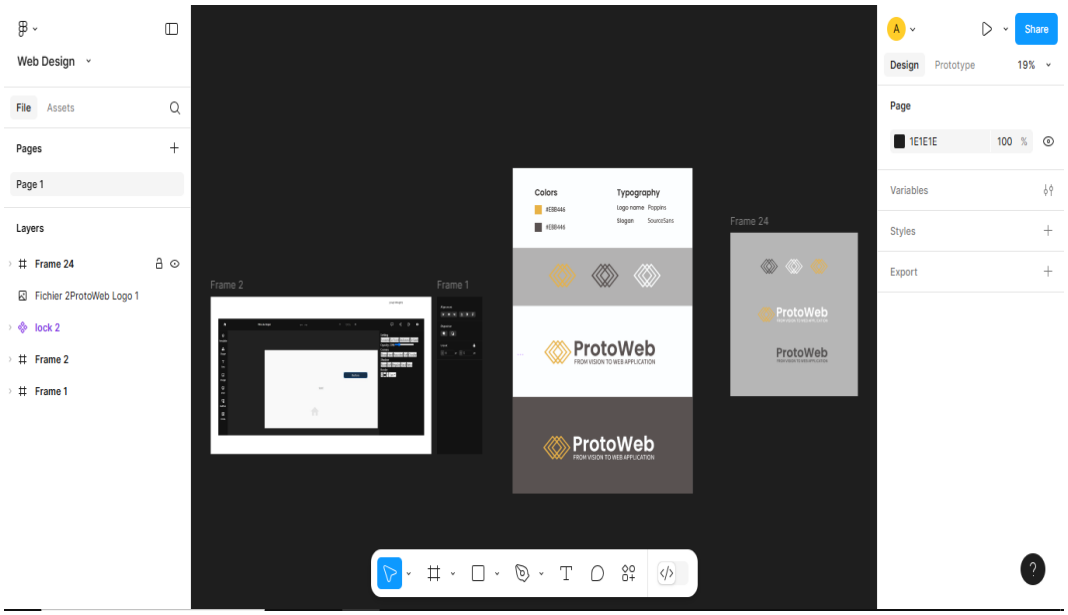


FIGURE 1.4 – La solution de prototypage Figma [8].

des maquettes. Par ailleurs, bien que l'esquisse de wireframes sur papier offre une première visualisation, elle reste statique et nécessite une retranscription manuelle sur des outils comme Figma, augmentant ainsi le risque d'erreurs et de malentendus.
Cette méthode traditionnelle conduit à de nombreux allers-retours entre les équipes techniques et les utilisateurs finaux, rallongeant le processus de prototypage et impactant la productivité. Enfin, l'absence d'un outil interactif, intuitif et accessible pour capturer directement les besoins ralentit la validation des maquettes et mobilise des ressources supplémentaires.
Le tableau ci-dessous présente une analyse comparative entre Adobe XD et Figma, en soulignant les différences majeures dans leurs fonctionnalités, leur approche du design d'interface et les possibilités de collaboration en ligne. Cette analyse se base principalement sur l'article de Zero To Mastery [9].

TABLE 1.1 – Tableau de comparaison simplifié entre Adobe XD et Figma

Outil	**Points forts**	**Points faibles**
Adobe XD	— Très complet, il permet de créer des maquettes très détaillées. — Possibilité de créer des prototypes cliquables qui ressemblent au produit final. — Fonctionne parfaitement avec les autres logiciels Adobe (Photoshop, Illustrator…).	— Difficile à prendre en main pour un débutant. — Il faut une bonne expérience ou une formation pour bien s'en servir. — Beaucoup de fonctions utiles sont payantes (abonnement nécessaire).

Outil	Points forts	Points faibles
Figma	— Idéal pour travailler à plusieurs en même temps. — Très simple pour partager les maquettes et recevoir des commentaires. — On peut ajouter beaucoup d'extensions (plugins) pour avoir plus de fonctions.	— Même s'il est plus simple au début, il faut du temps pour maîtriser toutes ses options. — Moins d'options pour le dessin ou la retouche d'image que les outils Adobe. — Il reste un outil pensé pour des utilisateurs qui connaissent déjà les bases du design.

Ces constats renforcent la nécessité de développer une solution plus flexible, plus rapide, et mieux adaptée aux attentes actuelles.

1.3.2.3 Solution proposée

Pour répondre à ces défis, nous avons conçu Protoweb, une application web innovante permettant de créer des prototypes en mode drag-and-drop, adaptée à tous les utilisateurs, même sans expertise technique. Grâce à cette plateforme, les utilisateurs pourront créer des maquettes interactives en utilisant des composants visuels qu'ils pourront personnaliser facilement (couleurs, tailles, alignement, etc.).
Ce qui distingue Protoweb, c'est son interface intuitive et épurée, idéale pour une prise en main rapide. En plus de la création des maquettes, l'application offre des options d'exportation flexibles, permettant de générer des fichiers au format pdf, image, et de récupérer le code HTML, CSS. De plus, un chatbot intégré apportera une aide contextuelle, guidant l'utilisateur à chaque étape de la conception. Pour les administrateurs, Protoweb offre un tableau de bord complet qui facilite la gestion des comptes utilisateurs, le suivi des projets . Cela permet un contrôle optimal sur le développement des maquettes et un suivi en temps réel de l'avancement des tâches.
En déployant Protoweb, Safran pourra non seulement minimiser les allers-retours avec les clients, mais aussi accélérer le processus de création des interfaces tout en optimisant l'utilisation des ressources humaines et techniques. Cette solution permettra d'améliorer la précision des designs et de réduire considérablement les délais de développement.

Figure 1.5 – Notre solution de prototypage Protoweb.

1.4 Choix méthodologique

Le choix de la méthode de gestion de projet est une étape déterminante qui influence directement le déroulement et le succès du développement. Cette section a pour but de présenter les différentes

approches envisagées, de les comparer, et de justifier la méthodologie finalement adoptée pour la réalisation de Protoweb.

1.4.1 Approches de gestion de projet

Deux grandes philosophies s'opposent en gestion de projet informatique : l'approche classique [28] et l'approche Agile [28].

1.4.1.1 Approche classique

L'approche classique [28], souvent illustrée par le modèle en cascade (Waterfall) ou le cycle en V, est une méthode séquentielle et prédictive. Le projet est découpé en phases distinctes et linéaires (définition des besoins, conception, développement, test, déploiement) qui se succèdent sans retour en arrière. Cette méthode exige une définition complète et détaillée de tous les besoins dès le début du projet, consignée dans un cahier des charges figé. Elle convient aux projets dont les exigences sont stables et parfaitement connues à l'avance.

1.4.1.2 Approche Agile

À l'inverse, l'approche Agile[28] est une philosophie qui privilégie la flexibilité, la collaboration et l'apport de valeur continue. Née en réaction aux lourdeurs des méthodes traditionnelles, elle repose sur des cycles de développement courts et itératifs. Plutôt que de suivre un plan rigide, l'Agilité encourage l'adaptation au changement. Son fondement est le **Manifeste Agile**, qui met en avant quatre valeurs clés : les individus et leurs interactions, des logiciels opérationnels, la collaboration avec le client et l'adaptation au changement.

1.4.1.3 Étude comparative entre l'approche classique et l'approche agile

Pour visualiser les différences fondamentales entre ces deux approches, le tableau suivante présente une comparaison sur plusieurs critères essentiels.

Table 1.2 – Comparaison entre l'approche classique et l'approche Agile.

Critère	**Approche Classique**	**Approche Agile**
Flexibilité	Faible. Le changement est difficile et coûteux une fois une phase terminée.	Élevée. Le changement est bienvenu et intégré à chaque itération.
Planning	Entièrement défini au début du projet. Très détaillé.	Adaptatif. Planifié par itérations courtes (Sprints).
Client	Impliqué principalement au début (besoins) et à la fin (recette).	Impliqué en continu tout au long du projet. Collaboration permanente.
Livraison	Une seule livraison du produit final à la fin du projet.	Livraisons fréquentes de versions fonctionnelles du produit.

1.4.1.4 Choix d'approche de gestion du projet

Compte tenu de la nature de notre projet, Protoweb, l'approche classique s'est révélée inadaptée. La problématique principale étant la difficulté des utilisateurs à exprimer leurs besoins, il était impossible d'établir un cahier des charges complet et définitif au départ.
Par conséquent, **nous avons choisi l'approche Agile**. Sa nature itérative et collaborative était la plus à même de répondre à ce défi. Elle nous a permis de construire la solution brique par brique, en validant chaque étape avec les parties prenantes, garantissant ainsi que le produit final corresponde véritablement à leurs attentes.

1.4.2 Méthodologie Agile adoptée : Scrum

Au sein de la grande famille des méthodes Agiles (comme Kanban ou Extreme Programming), notre choix s'est porté sur le cadre de travail **Scrum** [21]. C'est l'une des implémentations les plus populaires et structurées de l'Agile, particulièrement efficace pour des projets de développement complexes avec des équipes de petite taille.
Le cadre Scrum organise le travail en cycles de développement à durée fixe appelés **Sprints**, durant généralement de deux à quatre semaines. Chaque Sprint a pour objectif de produire un incrément potentiellement livrable du produit.

1.4.2.1 Rôles clés dans Scrum.

Scrum définit trois rôles principaux, chacun avec des responsabilités claires :

- **Le Product Owner :** Il est le représentant des clients et des utilisateurs. Sa mission est de définir la vision du produit et de gérer la liste des fonctionnalités à développer (le Product Backlog) en les priorisant pour maximiser la valeur livrée.
- **Le Scrum Master :** Il est le garant du cadre Scrum. Il ne gère pas l'équipe, mais s'assure qu'elle dispose de tout ce dont elle a besoin pour être productive. Il anime les réunions et aide à lever les obstacles.
- **L'équipe de développement :** C'est une équipe auto-organisée et pluridisciplinaire qui réalise le travail. Elle est responsable de la conception, du développement et du test des fonctionnalités durant chaque Sprint.

1.4.2.2 Artefacts et événements de Scrum.

Le processus s'articule autour d'outils (artefacts) et de réunions (événements) qui assurent la transparence et l'amélioration continue :

- **Le Product Backlog :** C'est la liste unique et priorisée de tout ce qui est souhaité dans le produit.
- **Le Sprint Backlog :** C'est la liste des tâches que l'équipe de développement s'engage à réaliser durant un Sprint.
- **La Mêlée quotidienne (Daily Scrum) :** Une réunion très courte (15 minutes) chaque jour où l'équipe se synchronise sur l'avancement et les obstacles.
- **La Revue de Sprint (Sprint Review) :** À la fin du Sprint, l'équipe présente ce qui a été accompli aux parties prenantes pour recueillir leurs retours.

— **La Rétrospective de Sprint (Sprint Retrospective) :** Juste après la revue, l'équipe se réunit pour réfléchir à ce qui a bien ou mal fonctionné durant le Sprint, dans le but de s'améliorer pour le suivant.

L'adoption de Scrum nous a fourni une structure claire pour appliquer les principes Agiles de manière efficace, comme l'illustre la figure suivante.

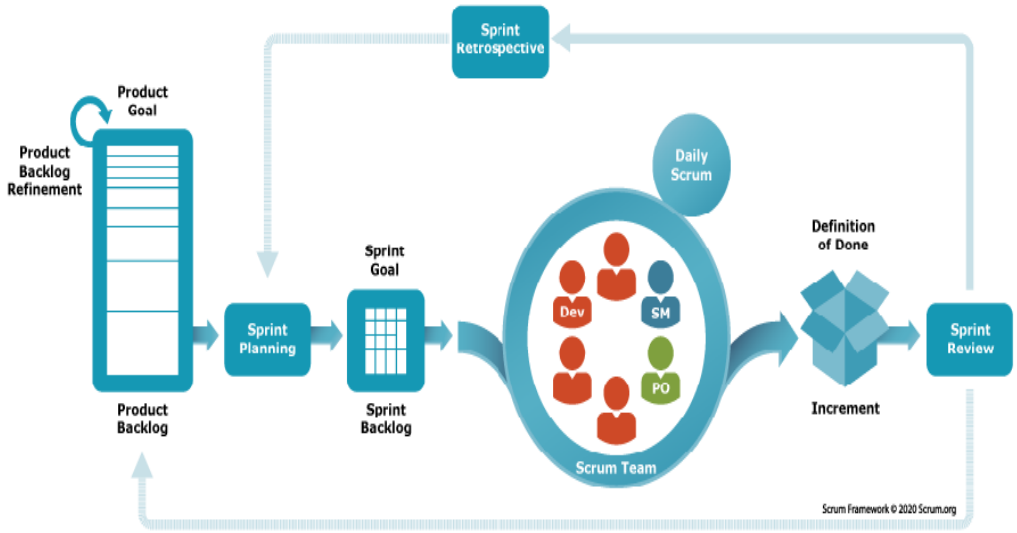


FIGURE 1.6 – Le cycle de la méthodologie Agile Scrum [21].

1.5 Conclusion

Ce chapitre nous a offert une vision globale en introduisant l'organisme d'accueil, en analysant l'existant, en présentant la solution envisagée et en justifiant notre approche méthodologique.

Chapitre 2

État de l'art et Spécification des besoins

2.1 Introduction

Ce chapitre présente l'état de l'art des technologies et méthodes utilisées pour les systèmes interactifs et les agents conversationnels. Il permet de situer notre projet par rapport aux solutions existantes et de justifier nos choix techniques.
Nous abordons également la conception du système, en identifiant les acteurs, les besoins fonctionnels et non fonctionnels, et en présentant les diagrammes qui décrivent l'organisation et le fonctionnement du projet.
Cette étude fournit une base solide pour comprendre les choix techniques, planifier le développement et structurer l'application de manière claire et cohérente.

2.2 État de l'art

La présente section constitue un état de l'art dédié aux architectures technologiques qui animent les agents conversationnels contemporains. Cette analyse s'avère indispensable pour contextualiser les choix de conception de notre solution, particulièrement en ce qui concerne l'intégration d'un chatbot d'assistance, une composante clé de Protoweb. Nous y examinerons ainsi les soubassements théoriques issus de l'Intelligence Artificielle qui rendent ces systèmes opérants.

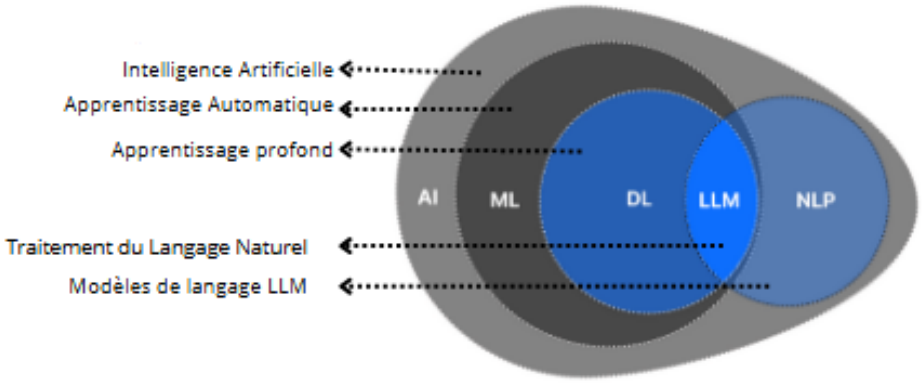


FIGURE 2.1 – Schéma des relations entre les domaines de l'Intelligence Artificielle[12].

2.2.1 Intelligence Artificielle

L'intelligence Artificielle (IA) [19] est une branche de l'informatique qui cherche à créer des machines capables de réaliser des tâches demandant normalement de l'intelligence humaine. Une partie importante de l'IA s'appelle l'apprentissage automatique, ou Machine Learning.

2.2.2 Apprentissage Automatique

Le Machine Learning (ML) [19] ne repose pas sur des instructions fixes, mais sur l'entraînement des algorithmes avec beaucoup de données. Cela permet aux machines de reconnaître des modèles et de faire des prédictions. Il existe trois principaux types d'apprentissage :

1. **Apprentissage supervisé :** Le modèle apprend à partir de données déjà étiquetées.
2. **Apprentissage non supervisé :** Le modèle découvre des structures cachées dans des données non étiquetées.
3. **Apprentissage par renforcement :** Le modèle apprend en essayant différentes actions dans un environnement et en recevant des récompenses ou des pénalités. C'est utilisé dans les jeux ou la robotique.

2.2.3 Apprentissage profond

Le Deep Learning (DL), ou apprentissage profond, est une branche de l'apprentissage automatique qui utilise des réseaux de neurones comportant plusieurs couches. Chaque couche apprend à extraire des caractéristiques de plus en plus complexes à partir des données, ce qui permet au modèle de reconnaître des motifs sophistiqués.
Cette approche est particulièrement adaptée au traitement du langage naturel et à la génération de texte, car elle permet aux modèles d'apprendre automatiquement les relations entre les mots, les phrases et le contexte global. Les modèles de langage de type LLM (Large Language Models) reposent sur le Deep Learning pour produire des réponses cohérentes, pertinentes et proches du langage humain.

2.2.4 Traitement du Langage Naturel

Le Natural Language Processing (NLP)[15] est une discipline qui permet aux ordinateurs de comprendre, d'analyser et de générer le langage humain. Il combine informatique et linguistique pour transformer le texte ou la parole en données exploitables.

Le Natural Language Processing peut utiliser des approches classiques basées sur des règles ou des approches statistiques, selon le type de projet, et rend possible une interaction intelligente entre l'humain et la machine, tout en facilitant l'exploitation des données textuelles.

2.2.5 Chatbots et leurs différentes approches

Un chatbot [6], ou agent conversationnel, est un programme qui imite une conversation humaine. Au fil du temps, leur fonctionnement est devenu de plus en plus complexe.

2.2.5.1 Types de chatbots

On peut classer les chatbots en trois types selon la technologie qu'ils utilisent :

1. **Chatbots à règles [3] :** Les plus simples, qui suivent des instructions précises, souvent sous forme d'arbres de décision. Ils répondent uniquement aux questions prévues et ont du mal avec des demandes imprévues.
2. **Chatbots utilisant le Machine Learning [3] :** Ces chatbots peuvent comprendre l'intention de l'utilisateur même si la question est formulée différemment. Ils deviennent plus performants avec le temps en s'entraînant sur de nouvelles conversations.
3. **Chatbots basés sur IA générative [3] :** La génération la plus récente de chatbots, capable de produire des réponses naturelles et adaptées au contexte, presque comme si elles étaient écrites par un humain.

2.2.5.2 Comparaison des Approches de Chatbots

Le tableau suivant met en évidence les distinctions principales entre les différentes catégories de chatbots et justifie le choix d'une approche reposant sur l'IA générative pour notre projet [7].

Table 2.1 – Comparaison des technologies de chatbots.

Critère	**Basé sur des règles**	**Basé sur le ML classique**	**Basé sur l'IA Générative (LLMs)**
Flexibilité	Très limitée. Suit un script précis.	Moyenne. Peut comprendre plusieurs formulations.	Très élevée. Gère des sujets variés et des conversations complexes.
Qualité de la conversation	Robotiques et répétitives.	Naturelle mais souvent courte.	Fluide, cohérente et proche du langage humain.
Coût de développement	Faible au départ, mais coûteux à maintenir pour ajouter de nouvelles règles.	Élevé, car il faut des données et de l'expertise pour l'entraînement.	Variable : rapide via API, plus complexe si hébergé soi-même.
Connaissance	Limitée aux informations programmées.	Limitée aux données utilisées pour l'entraînement.	Large si modèle généraliste, ou spécialisée si ajusté.

2.2.6 Intelligence Artificielle Générative

L'intelligence artificielle générative [31] désigne les systèmes capables de produire du contenu original, tel que du texte, des images ou du code, au lieu de se limiter à l'analyse ou la prédiction de données. Cette approche est particulièrement utilisée dans les chatbots, améliorant la fluidité et le réalisme des interactions avec les utilisateurs.

2.2.7 Modèles de Langage Génératifs (LLMs)

Les Large Language Models [29] sont des architectures de deep learning spécialisées dans le traitement et la génération de texte. En étant entraînés sur d'importants corpus textuels, ils acquièrent une

compréhension des structures grammaticales, du sens des phrases et du contexte. Dans les chatbots, les LLMs [29] permettent de produire des réponses pertinentes et adaptées, offrant une expérience conversationnelle naturelle.

2.2.8 Outils et Plateformes pour le Développement de Chatbots IA

Un chatbot [6] basé sur un grand modèle de langage peut être utilisé via des API cloud ou en hébergeant soi-même des modèles open source. Des outils comme Ollama [13] permettent de lancer facilement ces modèles sur un ordinateur ou un serveur. Cela donne un meilleur contrôle sur les données et l'infrastructure, ce qui est important dans un cadre industriel.

2.3 Spécification des besoins

2.3.1 Identification des acteurs

Un acteur représente l'entité, qu'il s'agisse d'une personne, d'un processus ou d'un élément externe, qui interagit avec un système en assumant un rôle spécifique. Dans le cadre de notre projet, nous identifierons les acteurs suivants :

- **Administrateur :**
  C'est l'acteur principal, c'est lui qui crée les comptes et gère les différentes fonctionnalités du système.
- **Designer** :
  C'est un acteur secondaire de l'application, qui dispose des mêmes fonctionnalités qu'un Utilisateur, avec en plus la capacité de gérer les templates.
- **Utilisateur** :
  Il s'agit d'un acteur clé de notre application, capable d'exprimer ses besoins en concevant une maquette ou un projet à l'aide d'un système de glisser-déposer (Drag and Drop).

2.3.2 Besoins Fonctionnels

Cette partie décrit ce que notre application, Protoweb, doit concrètement permettre de faire. Nous avons listé les fonctionnalités clés qui répondent aux problèmes que nous avons identifiés chez Safran.

- **Lancer un nouveau projet :**
  Pour bien commencer, l'utilisateur aura le choix entre deux approches. Soit il part d'une toile vierge pour laisser libre cours à sa créativité, soit il sélectionne un modèle déjà préparé par un designer pour accélérer son travail et suivre une structure existante.
- **Créer et gérer les modèles (pour les Designers) :**
  Cette fonction est réservée aux Designers. Un designer conçoit une maquette comme un projet normal. Une fois satisfait, il peut "promouvoir" ce projet en tant que modèle, le rendant disponible pour tous. Il garde également le contrôle pour retirer ses modèles de la liste.
- **Créer des interfaces en glissant-déposant :**
  L'idée de base est simple. L'utilisateur doit pouvoir prendre des éléments comme des boutons ou des titres depuis une liste, et les faire glisser sur sa page pour construire sa maquette. Pas besoin de coder, tout se fait à la souris.

— **Personnaliser chaque détail :**
Une fois qu'un élément est sur la page, l'utilisateur doit pouvoir cliquer dessus et changer tout ce qu'il veut : la couleur, la taille du texte, ce qui est écrit dessus, etc. L'interface doit lui donner le contrôle total sur l'apparence.

— **Sauvegarder le travail :**
Pour ne rien perdre, la création d'un nouveau projet est sauvegardée tout de suite. Ensuite, pour les modifications, c'est l'utilisateur qui décide quand enregistrer en cliquant sur un bouton dédié. C'est un choix pour lui donner plus de contrôle.

— **Travailler à plusieurs et gérer les droits :**
Un utilisateur qui crée un projet doit pouvoir inviter ses collègues. Il pourra alors choisir pour chaque personne si elle a le droit de modifier la maquette (accès en écriture) ou si elle peut seulement la regarder (accès en lecture seule).

— **Discuter directement sur le design :** Pour que les retours soient clairs et efficaces, les collaborateurs pourront commenter directement sur la maquette. La discussion se passe ainsi au bon endroit, sur le travail lui-même, ce qui met fin aux longues chaînes d'e-mails et aux risques de malentendus.

— **Exporter les maquettes :**
Une fois le travail terminé, il faut pouvoir le partager. L'application doit donc permettre d'exporter la maquette en image image PNG, JPG ou en PDF, pour l'envoyer par mail ou l'inclure dans une présentation.

— **Obtenir le code source :**
Pour faire gagner du temps aux développeurs, l'application doit être capable de "traduire" la maquette visuelle en code HTML,Css. Le développeur récupère ainsi une base de travail déjà prête.

— **Avoir une aide intégrée :**
Pour que personne ne se sente perdu, un chatbot sera disponible directement dans l'application. Il pourra répondre aux questions des utilisateurs sur le fonctionnement de l'outil.

— **Suivre l'activité de la plateforme :**
Un administrateur doit pouvoir consulter un tableau de bord. Il y retrouvera des chiffres clés pour comprendre l'usage réel de l'application : le nombre de projets en cours, les modèles les plus populaires, ou encore le nombre d'utilisateurs actifs.

— **Gérer les utilisateurs et les rôles :**
Un administrateur aura la charge de créer les comptes et de définir qui est "Utilisateur" ou "Designer", chacun ayant des possibilités différentes dans l'application.

2.3.3 Besoins non fonctionnels

Les besoins non fonctionnels représentent les exigences implicites auxquelles le système doit se conformer. Ils contribuent à l'amélioration de la qualité des services offerts par le site. Parmi ces exigences, on peut citer :

- **Sécurité :**

L'application doit garantir la confidentialité des projets et la protection des données des utilisateurs. Les accès doivent être sécurisés par un système d'authentification robuste, et des rôles (Administrateur, Designer, Utilisateur) doivent être mis en place pour contrôler les permissions sur les différentes fonctionnalités.

- **Performance :**

Le temps de réponse du système doit être rapide. Le chargement d'une page Web dans le navigateur ne doit pas dépasser une certaine limite dans des conditions normales.

- **Ergonomie :**

L'application doit être intuitive et agréable à utiliser, avec des interfaces harmonieuses et une disposition bien organisée pour une meilleure expérience utilisateur.

- **Disponibilité :**

Le système doit être accessible et fonctionnel aux utilisateurs du site .

- **Maintenabilité :**

Le code de l'application doit être bien structuré et organisé afin de faciliter la maintenance, permettant ainsi d'assurer les mises à jour et d'intégrer d'éventuelles modifications.

2.3.4 Vue architecturale des cas d'utilisations

Le cas d'utilisation global a pour objectif de décrire le comportement fonctionnel du système.
la figure suivante en offre une illustration.

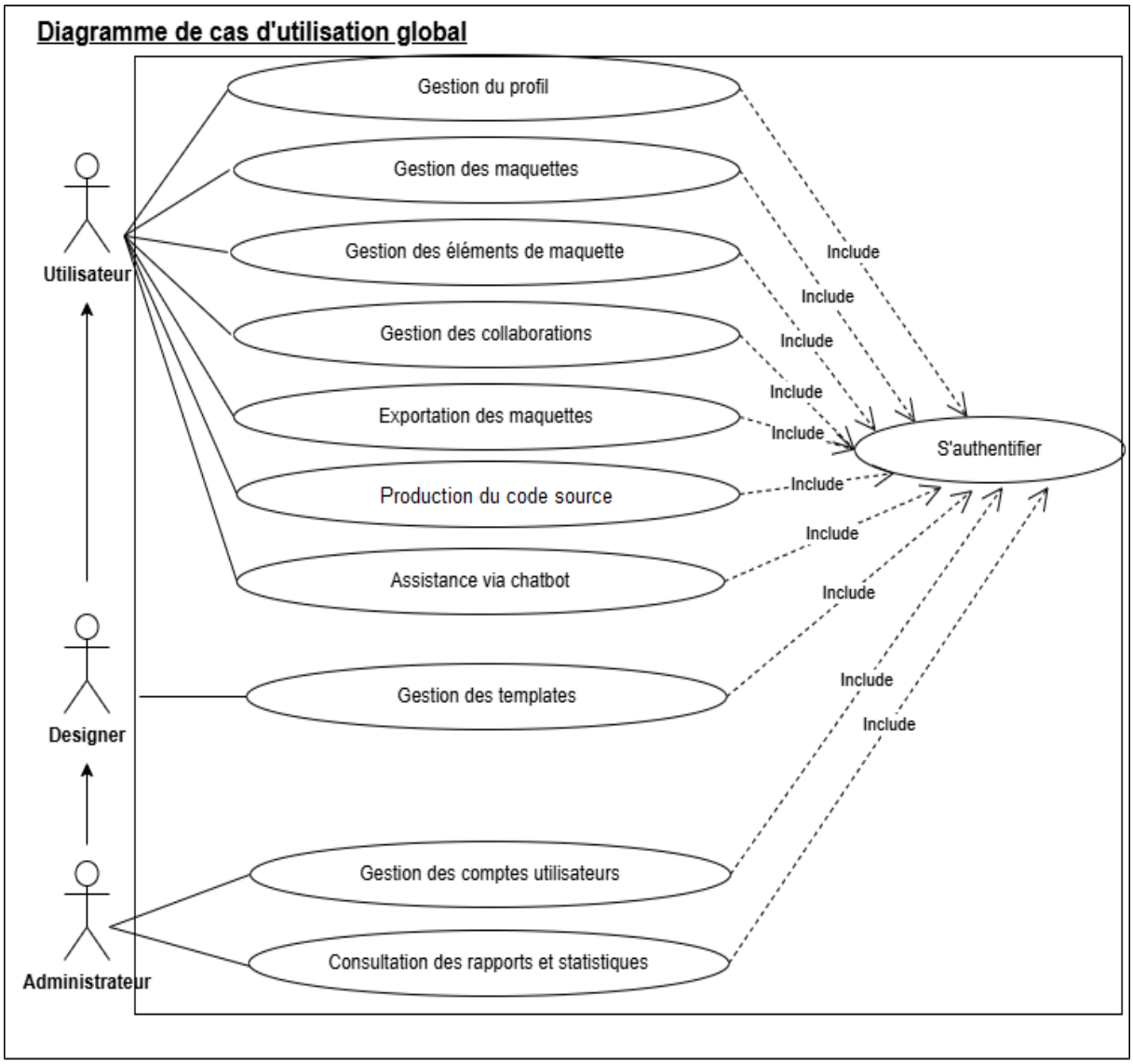


FIGURE 2.2 – Diagramme de cas d'utilisation global.

2.3.5 Planification

2.3.5.1 Equipe et rôle

Puisque la méthodologie Scrum [21] a été retenue, comme expliqué dans le chapitre 1, nous allons maintenant introduire notre équipe Scrum.

TABLE 2.2 – Affectation des rôles

Rôle	Membre
Scrum Master	Mohamed Hakim Trabelsi
Product Owner	Wajdi Chebane
Équipe de développement	Ala Nsir

2.3.5.2 Product Backlog

Le backlog du produit contient un ensemble des caractéristiques (fonctionnalités ou besoins techniques) qui constituent le produit souhaité .

Chaque élément du backlog est directement lié à un cas d'utilisation du système (voir figure suivante). Cette correspondance permet d'assurer que les besoins identifiés sont bien traduits en fonctionnalités concrètes dans le développement

Table 2.3 – Backlog du produit (Product Backlog)

User Story	Priorité	Description
US 1	1	En tant qu'Admin, UX Designer ou utilisateur, je peux m'authentifier en toute sécurité.
US 2	5	En tant qu'Admin, je peux gérer les comptes des utilisateurs (UX Designer ou Utilisateur).
US 3	4	En tant qu'Admin, UX Designer ou Utilisateur, je peux gérer et personnaliser les composants.
US 4	3	En tant qu'Admin, UX Designer ou Utilisateur, je peux glisser et déposer les composants.
US 5	2	En tant qu'Admin, UX Designer ou Utilisateur, je peux gérer les projets.
US 6	7	En tant que Designer, mon but est de préparer des modèles de projets (templates). Je dois pouvoir les créer et les mettre à jour facilement, pour que les autres utilisateurs n'aient plus qu'à choisir un modèle pour commencer un nouveau projet.
US 7	9	En tant qu'Admin, UX Designer ou Utilisateur, je peux collaborer avec d'autres utilisateurs et échanger des commentaires en temps réel.
US 8	11	En tant qu'Admin, UX Designer ou Utilisateur, j'ai la possibilité d'interagir avec le chatbot.
US 9	10	En tant qu'Admin, je peux consulter les statistiques.
US 10	6	En tant qu'Admin, UX Designer ou Utilisateur, je souhaite pouvoir consulter et modifier les informations de mon profil.
US 11	8	En tant qu'Admin, UX Designer ou Utilisateur, je peux exporter une maquette et obtenir le code source.

2.3.5.3 Processus de livraison

Le projet sera réalisé en deux grandes phases, réparties entre le 3 février et le 3 août. Chaque phase correspond à une release du produit, permettant d'ajouter progressivement les fonctionnalités et d'aboutir à une application pleinement opérationnelle.

1- Première release (Février – Juillet)
Cette phase vise à mettre en place le noyau principal de l'application web ainsi que les fonctionnalités essentielles pour la gestion et la création de projets. Les User Stories seront développées au sein de

sprints planifiés selon leur complexité, certains sprints durant 2, 3 ou 4 semaines.

2- Deuxième release (Juillet – Août)
La deuxième phase a pour objectif d'ajouter les fonctionnalités avancées, comme l'intégration de l'IA et du chatbot, d'optimiser les performances, de corriger les anomalies et de préparer l'application pour son déploiement sur un serveur de test. Les sprints sont également ajustés selon la charge de travail des User Stories.

Le tableau suivant présente le détail des sprints, leurs dates, leur durée et les User Stories associées à chaque étape de livraison.

Table 2.4 – Plan de livraison avec sprints

Release	Sprint	Durée / Dates	User Stories
Release 1	Sprint 1	4 semaines (03/02–01/03)	Authentification (US 1), Gestion des utilisateurs par l'administrateur (US 5), Gestion du profil utilisateur (US 10)
Release 1	Sprint 2	4 semaines (02/03–29/03)	Création et gestion des projets (US 5), Gestion des modèles par les designers (US 6)
Release 1	Sprint 3	4 semaines (30/03–26/04)	Implémentation du glisser-déposer (US 4), Personnalisation des composants (US 3)
Release 1	Sprint 4	4 semaines (27/04–24/05)	Partage de projets et commentaires en temps réel (US 7)
Release 1	Sprint 5	4 semaines (25/05–21/06)	Exportation de la maquette au code source (US 11)
Release 1	Sprint 6	2 semaines (22/06–05/07)	Développement du tableau de bord statistique (US 9)
Release 2	Sprint 7	3 semaines (06/07–26/07)	Sélection et intégration d'un modèle de langage adapté (US 8)
Release 2	Sprint 8	1 semaine (27/07–03/08)	Test et déploiement

2.3.5.4 Chronogramme de Planification

Le projet est structuré en 8 sprints, répartis sur une période totale de 6 mois, allant du 03 février 2025 au 03 août 2025.
La première release s'étend sur 5 mois et couvre les sprints 1 à 6, incluant les fonctionnalités de base comme l'authentification, la gestion des utilisateurs, la création et la personnalisation des projets, le partage collaboratif, l'exportation en code source et l'intégration d'un tableau de bord statistique.
La seconde release, d'une durée d'1 mois, regroupe les sprints 7 et 8 et se concentre sur l'intégration d'un modèle de langage adapté ainsi que la phase finale de test et de déploiement. La planification détaillée de chaque sprint, avec ses objectifs, sa durée et ses dates, est présentée dans le tableau ci-après.

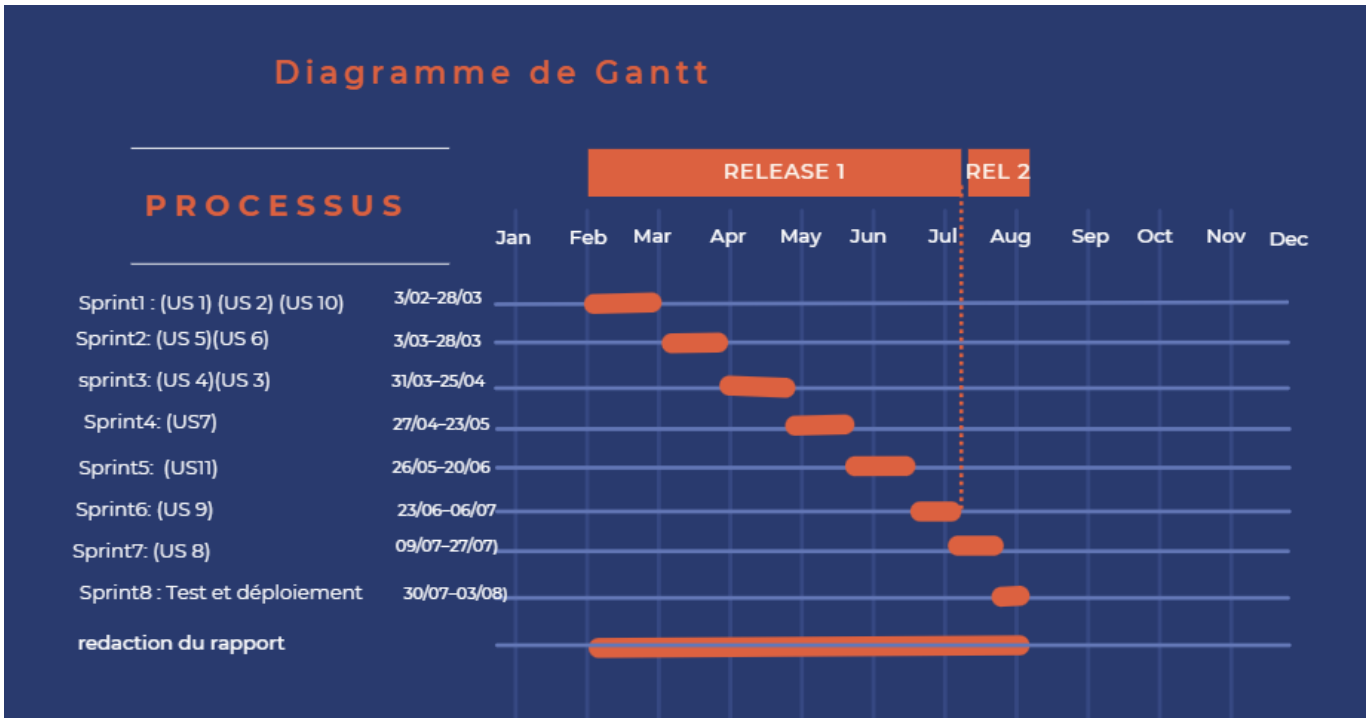


FIGURE 2.3 – Diagramme de Gantt du projet détaillé

2.4 Environnement de travail

Pour réaliser le développement de notre application, nous avons utilisé divers outils matériels et logiciels. Cette section expose l'environnement technologique retenu ainsi que les ressources matérielles mises en œuvre.

2.4.1 Environnement matériel

Nous avons utilisé un ordinateur de développement ASUS X415 de configurations matérielles suivantes :
- Système d'exploitation : Windows 10 Famille
- 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz 2.42 GHz
- RAM : 8 Go.
- Disque dur : 512 Go SSD.

2.4.2 Environnement logiciel

Pour mener à bien ce projet, nous avons dû sélectionner une série d'outils et de technologies. Chaque choix a été fait pour répondre à un besoin précis, que ce soit pour dessiner les maquettes, écrire le code ou gérer nos données. Voici la liste des logiciels que nous avons utilisés.

2.4.2.1 Outils de conception

Avant d'écrire la moindre ligne de code, il a fallu poser nos idées et concevoir l'aspect visuel de l'application.

Pour créer les maquettes de Protoweb, notre choix s'est porté sur **Figma**. C'est un outil très pratique pour dessiner les interfaces et simuler les interactions entre les écrans, le tout en ligne et de manière collaborative.

FIGURE 2.4 – Logo de Figma.

Drawio nous a servi pour tout ce qui est visuel en dehors de l'application elle-même. C'est avec cet outil que nous avons réalisé les schémas et diagrammes que vous trouvez dans ce rapport.

FIGURE 2.5 – Logo de Drawio.

2.4.2.2 Outils de développement

2.4.2.2.1 Technologies pour le Frontend

C'est l'ensemble des outils qui nous ont permis de construire la partie de l'application que l'utilisateur voit et manipule dans son navigateur.

Pour écrire le code, nous avons utilisé **vscode**. C'est un éditeur de texte léger, rapide et plein d'extensions utiles qui nous ont facilité le travail au quotidien.

FIGURE 2.6 – Logo de Visual Studio Code [**?**].

L'application côté Utilisateur a été entièrement bâtie avec **Angular (v13)**. Ce framework de Google nous a fourni une structure solide pour créer une application web moderne et réactive, où les pages se chargent sans rafraîchissement complet.

FIGURE 2.7 – Logo d'Angular [**?**].

2.4.2.2.2 Technologies pour le Backend

C'est le moteur de notre application, la partie invisible qui gère toute la logique et les données.

Pour le développement en Java, nous avons choisi **IntelliJ IDEA**. C'est un environnement de développement très puissant qui nous a beaucoup aidés à écrire un code propre et à trouver les erreurs rapidement.

FIGURE 2.8 – Logo d'IntelliJ IDEA.
[26]

Le backend lui-même a été développé avec **Spring Boot (v2.7)**. Ce framework nous a fait gagner un temps précieux en simplifiant énormément la configuration d'un serveur web en Java.

FIGURE 2.9 – Logo de Spring Boot .[16]

2.4.2.2.3 Outils de gestion des données et des tests

Enfin, il nous fallait des outils pour stocker les informations et pour vérifier que notre API fonctionnait correctement.

Toutes les données de Protoweb (utilisateurs, projets, etc.) sont stockées dans une base de données **Microsoft SQL Server**. C'est une solution robuste et fiable pour gérer des données relationnelles.

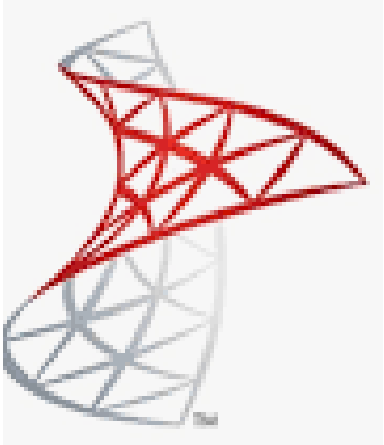


FIGURE 2.10 – Logo de Microsoft SQL Server. [**?**]

Avant de connecter le frontend au backend, nous avons testé chaque route de notre api avec **Postman**. Cet outil nous a permis d'envoyer des requêtes et de vérifier les réponses du serveur pour nous assurer que tout fonctionnait comme prévu.

FIGURE 2.11 – Logo de Postman [14].

2.4.3 Choix technologiques

Choisir les bons outils est très important pour qu'un projet réussisse. Pour Protoweb, nous ne voulions pas seulement des technologies à la mode, mais des outils solides et adaptés au monde de l'entreprise. Voici pourquoi nous avons pris Angular, Spring Boot et sql Server, après avoir regardé ce qui existait d'autre.

2.4.3.1 Choix du moteur : le Backend

Le backend, c'est le cerveau de l'application. Il doit être fiable et sécurisé.

Table 2.5 – Comparaison simple des technologies pour le Backend

Outil	**Spring Boot (notre choix)**	**Node.js**	**.NET**
Comment il est fait ?	Utilise Java, un langage très strict. Ça nous oblige à écrire un code propre et ça évite beaucoup de bugs plus tard, surtout sur un gros projet.	Utilise JavaScript, qui est plus souple. C'est très rapide mais ça peut rendre le code plus difficile à maintenir sur le long terme.	Utilise C#, un langage de Microsoft, très similaire à Java. C'est aussi un choix très solide et fiable.
Et pour la sécurité ?	Intègre un outil très puissant et connu, **Spring Security**, qui nous a permis de bien protéger les accès des utilisateurs.	Il faut assembler plusieurs petites librairies externes pour gérer la sécurité, ce qui peut être plus complexe à gérer.	Offre de très bons outils de sécurité, directement fournis par Microsoft.

Pourquoi on a choisi Spring Boot ? Pour un projet chez Safran, la **sécurité** et la **fiabilité** étaient nos priorités numéro 1. Spring Boot nous a offert la solution la plus complète et la plus sûre. C'est un outil connu pour sa robustesse, parfait pour une application d'entreprise.

2.4.3.2 Choix de l'interface : le Frontend

Le frontend, c'est tout ce que l'utilisateur voit et clique. Il doit être agréable à utiliser et facile à construire.

Table 2.6 – Comparaison simple des technologies pour le Frontend

Outil	Angular (notre choix)	React	Vue.js
Comment ça marche ?	C'est comme une "boîte à outils complète". Il nous donne un cadre de travail strict, une sorte de "plan" à suivre. C'est très utile en équipe.	C'est plus une "bibliothèque" qu'une boîte à outils. C'est très flexible, mais ça veut dire qu'on doit choisir et assembler beaucoup d'outils nous-mêmes.	C'est un peu entre les deux. Il est plus simple à apprendre qu'Angular, mais moins complet au départ.
Le langage utilisé	Utilise **TypeScript**, une version de JavaScript plus sûre qui aide à éviter les erreurs de code avant même de lancer l'application.	Utilise surtout jsx. On peut y ajouter TypeScript, mais ce n'est pas obligatoire.	Comme React, on peut utiliser TypeScript mais ce n'est pas la base.

Pourquoi on a choisi Angular ? Angular nous a donné une **structure claire et solide**. Pour une application aussi complète que Protoweb, c'était important d'avoir un cadre pour que le projet reste propre et facile à modifier plus tard. Le fait qu'il utilise TypeScript par défaut était un grand plus pour la qualité du code.

2.5 Notre architecture pour Protoweb

Pour construire notre application Protoweb, nous devions choisir une structure solide, facile à maintenir et performante. Nous sommes partis du modèle classique à trois niveaux (3-Tier) [**?**], mais nous l'avons personnalisé avec les technologies que nous avons réellement utilisées pour le projet.
Le schéma de la figure **??** montre comment les différentes briques de notre système sont organisées.

— **La partie visible (Frontend) :**
C'est l'interface que l'utilisateur voit et manipule dans son navigateur. Nous l'avons développée en **Angular** [**?**]. C'est cette partie qui gère l'affichage, le glisser-déposer, et qui envoie les requêtes au serveur via une Api rest.

— **Le cerveau de l'application (Backend) :**
C'est ici que se trouve toute l'intelligence de Protoweb. Développé en **Spring Boot** [16], le backend s'occupe de la logique métier : qui a le droit de faire quoi, comment les projets sont sauvegardés, la gestion des comptes, etc. Il fonctionne comme un serveur qui répond aux demandes du frontend.

— **La mémoire du système (Base de données) :**
Pour stocker toutes les informations de manière durable (les utilisateurs, les projets, les composants), nous avons mis en place un **sgbd**Microsoft SQL Server [**?**]. Le backend est le seul à pouvoir discuter directement avec cette base pour lire ou enregistrer des données.

Le grand avantage de cette structure est que nous avons pu travailler sur le frontend et le backend en parallèle, sans que l'un ne bloque l'autre.

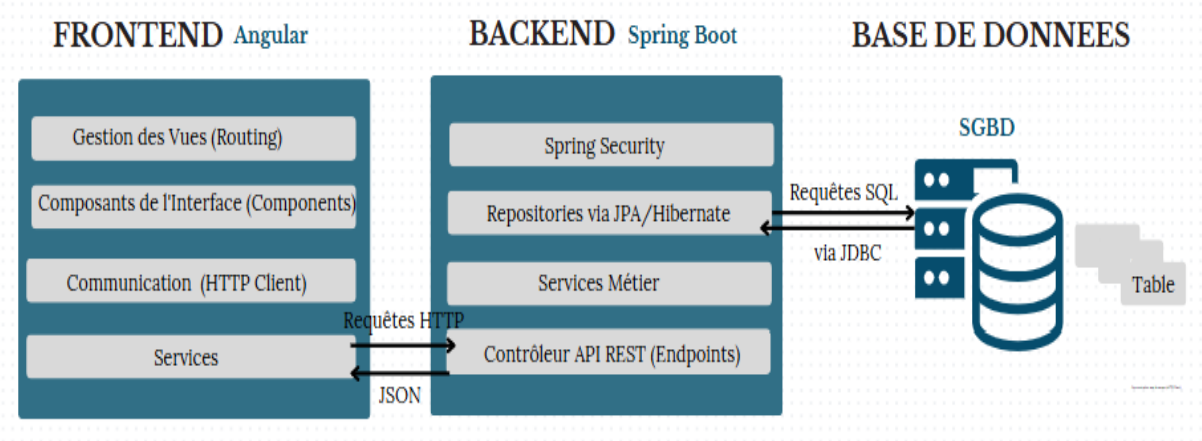


FIGURE 2.12 – Architecture technique de l'application Protoweb.

2.6 Conclusion

Ce chapitre a posé les fondations techniques de notre projet. Nous avons commencé par identifier les acteurs et spécifier en détail les besoins fonctionnels et non fonctionnels, ce qui nous a permis de définir le périmètre de la solution. La planification Agile via le Product Backlog et la définition des rôles de l'équipe Scrum ont ensuite structuré notre démarche de développement. Enfin, la présentation des environnements de travail et de l'architecture à trois niveaux a clarifié les choix technologiques qui soutiennent notre application Protoweb.

Chapitre 3

Release 1 : Mise en place des Fondations Fonctionnelles de la Plateforme

3.1 Introduction

Cette première release constitue la base du développement de notre application web. Elle regroupe l'ensemble des fonctionnalités essentielles permettant d'assurer l'authentification, la gestion des utilisateurs et des profils, la création et l'administration des projets, ainsi que les premiers outils de collaboration et de personnalisation.
Conformément à la planification définie, la Release 1 s'est déroulée sur une période de cinq mois et a été structurée en six Sprints successifs. Le premier Sprint a introduit l'authentification, la gestion des utilisateurs par l'administrateur ainsi que la gestion des profils. Le deuxième Sprint a été consacré à la création et à la gestion des projets ainsi qu'à la mise en place des modèles (templates). Le troisième Sprint a porté sur le développement de l'éditeur visuel intégrant les fonctionnalités de glisser-déposer et de personnalisation des composants. Le quatrième Sprint a introduit le partage des projets et les commentaires en temps réel. Le cinquième Sprint a permis l'exportation des maquettes en code source. Enfin, le sixième et dernier Sprint, plus court, a été dédié au développement du tableau de bord statistique.
Ainsi, chaque Sprint a apporté une avancée fonctionnelle concrète et a contribué à la construction progressive et structurée du produit

3.2 Sprint 1 : Mise en place de l'authentification, de la gestion des utilisateurs et des profils

3.2.1 Présentation du Sprint 1

Le premier Sprint joue un rôle stratégique dans la mise en place de la plateforme Protoweb, en posant les fondations du module utilisateur. Il a permis d'intégrer les mécanismes essentiels d'authentification, la gestion des comptes par les administrateurs ainsi que la personnalisation des profils par les utilisateurs. Ce cycle de développement a assuré une base fiable et sécurisée pour les futures fonctionnalités, en instaurant les premiers jalons de sécurité, d'organisation et d'expérience utilisateur. L'ensemble de ces travaux a permis de construire un socle fonctionnel complet, garantissant la cohérence et la robustesse du système tout en centralisant la gestion des utilisateurs et en offrant

une expérience personnalisée à chaque acteur de la plateforme. subsectionSpécification fonctionnelle Cette section décrit en détail les fonctionnalités à implémenter lors du Sprint 1, en précisant les besoins des utilisateurs, les interactions attendues avec le système, ainsi que les exigences de sécurité et de gestion des comptes.

3.2.1.1 Sprint Backlog

Ce Sprint a été planifié avec un backlog ambitieux, regroupant toutes les User Stories liées à l'authentification, la gestion des utilisateurs par l'administrateur, et la gestion du profil utilisateur. L'objectif était de livrer un module utilisateur complet et fonctionnel dès le premier cycle de développement.

Table 3.1 – Backlog du Sprint 1

User Story	**Description**	**Complexité (Points)**	**Estimation (jours)**
US 1.1	En tant qu'utilisateur, je veux disposer d'une interface de connexion claire (champs Identifiant/Mot de passe) pour pouvoir saisir mes identifiants et accéder à l'application.	5	3
US 1.2	En tant que développeur, je veux mettre en place une API sécurisée qui valide les identifiants et protège les mots de passe.	8	4
US 2.1	En tant qu'Admin, je veux une page qui liste tous les utilisateurs enregistrés.	1	3
US 2.2	En tant qu'Admin, je veux pouvoir créer un nouveau compte utilisateur.	3	2
US 2.3	En tant qu'Admin, je veux pouvoir modifier les informations d'un utilisateur existant.	3	2
US 2.4	En tant qu'Admin, je veux pouvoir supprimer un utilisateur dont le compte n'est plus nécessaire.	3	2
US 2.5	En tant qu'Admin, je veux pouvoir rechercher un utilisateur en combinant plusieurs critères.	5	2
US 10.1	En tant qu'utilisateur, je veux pouvoir modifier mon nom et ma photo de profil.	5	3
US 10.2	En tant qu'utilisateur, je veux pouvoir changer mon mot de passe.	3	4

3.2.1.2 Diagramme de cas d'utilisation global du Sprint 1

Les interactions clés des différents acteurs avec le système ont été minutieusement modélisées. Le diagramme de cas d'utilisation consolidé ci-dessous présente une vue d'ensemble des fonctionnalités

implémentées, illustrant la manière dont les administrateurs et les utilisateurs interagissent avec les modules d'authentification, de gestion des utilisateurs et de gestion de profil.

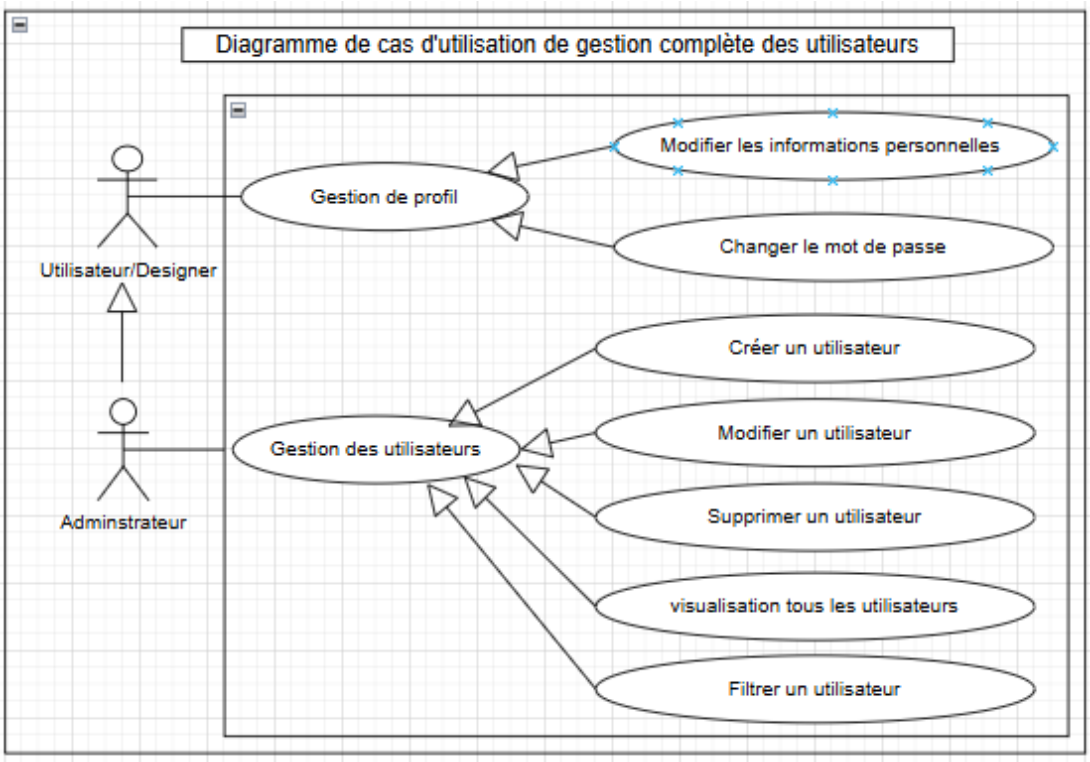


FIGURE 3.1 – Diagramme de cas d'utilisation de gestion complète des utilisateurs

Ce diagramme met en évidence deux grandes composantes : La gestion des utilisateurs (uniquement accessible à l'administrateur pour la visualisation, création, modification, suppression et recherche) et la gestion du profil (accessible à tous les utilisateurs pour modifier leurs informations personnelles et leur mot de passe).

3.2.1.3 Diagramme de classe du Sprint 1

La conception architecturale de ce Sprint est centrée autour de la robustesse et de l'extensibilité, avec la classe Utilisateur comme entité centrale. Ce diagramme de classes illustre la structure des données et les relations entre les composants essentiels du système utilisateur.

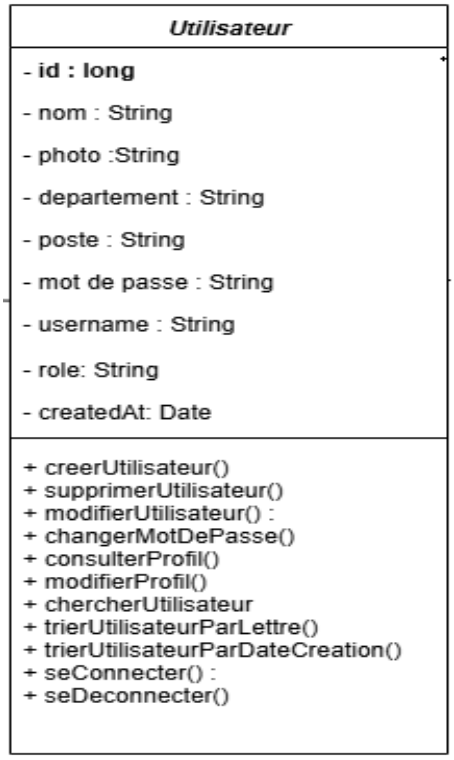


FIGURE 3.2 – Diagramme de classes – Sprint 1

Cette figure présente le diagramme de classes simplifié pour le Sprint 1, focalisé sur l'entité Utilisateur. Elle détaille les attributs essentiels tels que l'identifiant, le nom, les informations de profil, ainsi que le rôle. La section des opérations met en évidence les fonctionnalités clés liées à la gestion des utilisateurs, incluant la création, la modification, la suppression et l'authentification, reflétant ainsi les bases fonctionnelles mises en place lors de ce premier Sprint.

3.2.2 Raffinement des diagrammes de cas d'utilisation

Pour mieux comprendre les interactions, voici les descriptions détaillées de quelques cas d'utilisation représentatifs de ce Sprint.
Pour détailler les interactions dynamiques, les diagrammes de séquence suivants illustrent les flux d'informations et les échanges entre les différents composants du système lors des opérations critiques d'authentification et de gestion de profil.

3.2.2.1 Analyse de cas "s'authentifier"

Cette analyse présente le processus détaillé par lequel un utilisateur accède à la plateforme, en mettant en évidence les étapes d'authentification, les validations requises et les interactions entre l'interface et le système backend.

3.2.2.1.1 Description textuelle de Cas d'Utilisation : Authentification

Ce scénario décrit le processus standard par lequel tout utilisateur se connecte à la plateforme.

Table 3.2 – Description du scénario « Authentification »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Authentification de l'utilisateur
Acteur	Utilisateur, Designer, Admin
Objectif	Permettre à l'utilisateur de se connecter à la plateforme.
Pré-condition	L'utilisateur se trouve sur la page de connexion (View Login).
Post-condition	L'utilisateur est authentifié et redirigé vers l'interface Projets.
Scénario nominal	1. L'utilisateur saisit l'identifiant et le mot de passe dans le formulaire de l'interface `View Login`. 2. L'interface `View Login` effectue une **Validation (champs non vides)**. 3. **Si la saisie est invalide :** L'interface `View Login` affiche un **Message d'erreur** à l'utilisateur. Le processus s'arrête ici pour cette tentative. 4. **Si la saisie est valide :** L'interface `View Login` envoie une requête `login()` au `LoginController`. 5. Le `LoginController` appelle la méthode pour **Vérifier identifiants** `:User`. 6. LoginController interroge la base de données pour valider les identifiants et retourne une réponse (`true/false`) via le service. 7. **Si les identifiants sont invalides :** Le `LoginController` renvoie une **Erreur (Données non valides)** à la `View Login`. 8. L'interface `View Login` affiche le **Message d'erreur** à l'utilisateur. 9. **Si les identifiants sont valides :** Le `LoginController` renvoie un message de **Succès (authentification OK)** à la `View Login`. 10. L'interface `View Login` effectue une **Redirection vers Projets**.
Scénario d'échec	— **Saisie invalide :** L'utilisateur laisse des champs vides. La `View Login` détecte l'erreur et affiche directement un "Message d'erreur" sans communiquer avec le backend. — **Identifiants incorrects :** Après une saisie valide, les identifiants sont jugés invalides. La `View Login` affiche un "Message d'erreur".

3.2.2.1.2 Diagramme de Séquence : Authentification

Ce diagramme illustre le déroulement de la connexion d'un utilisateur au système.

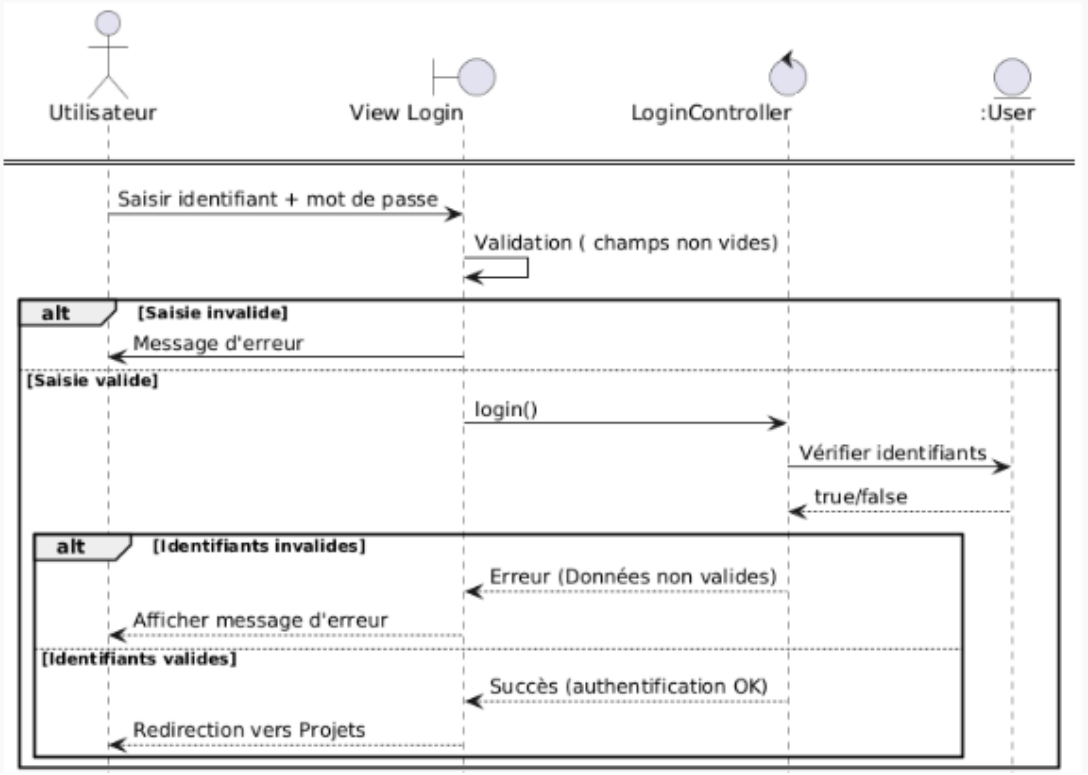


FIGURE 3.3 – Diagramme de séquence d'authentification

3.2.2.2 Analyse de cas " Créer utilisateur "

Cette analyse explique la procédure suivie par un administrateur pour ajouter un nouvel utilisateur, en mettant en lumière les étapes de saisie, de validation et de communication avec le système.

3.2.2.2.1 Description textuelle de Cas d'Utilisation : Créer utilisateur

Ce scénario décrit la procédure pour un administrateur afin d'ajouter un nouveau collaborateur à la plateforme.

TABLE 3.3 – Description textuelle du scénario « Créer un utilisateur »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Créer un nouvel utilisateur
Acteur	Administrateur
Pré-condition	L'administrateur est authentifié et se trouve sur la page de gestion des utilisateurs
Post-condition	Un nouveau compte utilisateur est créé dans le système.

Élément du cas d'utilisation	Détails
Scénario nominal	1. L'administrateur clique sur le bouton « + ajouter une ligne » dans le tableau. 2. Le formulaire intégré à la ligne apparaît pour saisir les informations du nouvel utilisateur. 3. ViewUser vérifie les champs obligatoires et la validité des données. 4. Si les données sont valides, ViewUser envoie la requête `createUser()` au `UserController`. 5. Le `UserController` traite la requête et crée l'utilisateur dans la base de données via l'entité `User`. 6. Le `UserController` retourne le résultat de l'opération à `ViewUser`. 7. `ViewUser` met à jour le tableau et affiche un message de succès.
Scénario d'échec	1. Si les champs du formulaire sont invalides, `ViewUser` affiche immédiatement un message d'erreur et bloque l'envoi. 2. Si l'insertion en base échoue (doublon, erreur serveur...), le `UserController` renvoie une erreur.

3.2.2.2.2 Diagramme de Séquence : Création d'un utilisateur

Ce diagramme détaille les étapes pour un administrateur créant un nouveau compte utilisateur.

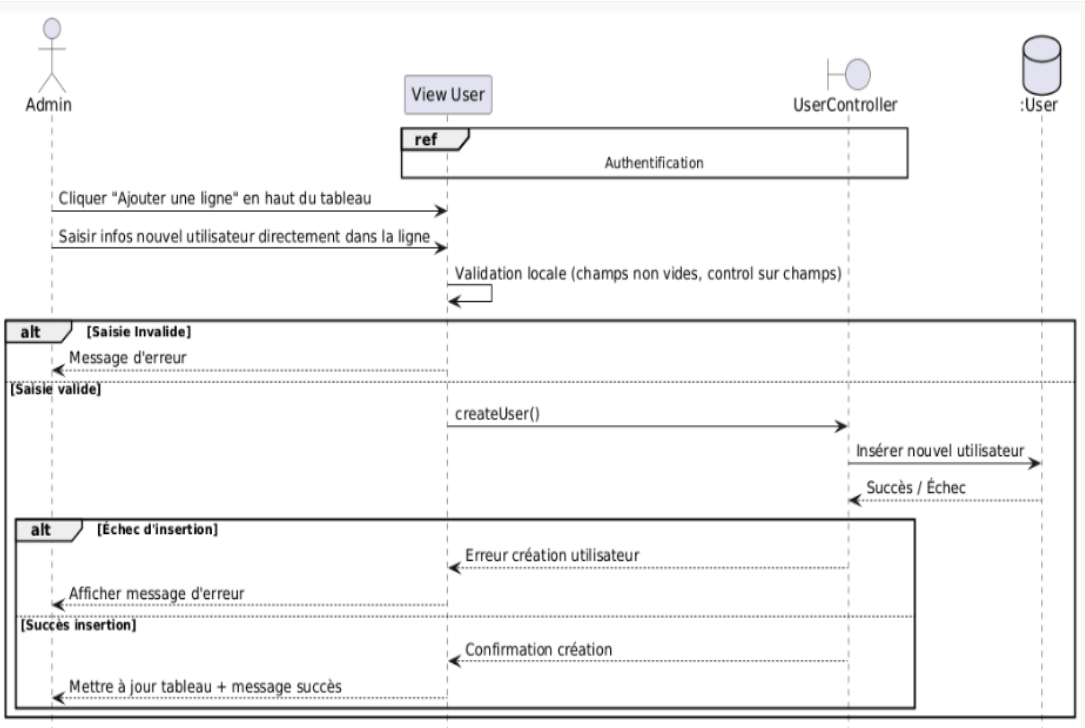


FIGURE 3.4 – Diagramme de séquence de création d'un utilisateur

3.2.2.3 Analyse de cas " Modifier profil "

Cette analyse illustre la manière dont un utilisateur peut mettre à jour ses informations personnelles et son mot de passe, en précisant les contrôles effectués côté interface et les interactions avec le backend.

3.2.2.3.1 Description textuelle de cas d'utilisation : Modifier profil

Ce scénario explique comment un utilisateur peut mettre à jour ses informations personnelles.

Table 3.4 – Description du scénario « Changer son mot de passe »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Changer son mot de passe
Acteur	Admin, Utilisateur, Designer
Pré-condition	L'utilisateur est authentifié et a accédé à la page « Paramètres » de son profil.
Post-condition	Le mot de passe de l'utilisateur est mis à jour dans la base de données et le changement est immédiatement visible.
Scénario nominal	1. L'utilisateur navigue vers la page de son profil. 2. Le système affiche les informations actuelles. 3. L'utilisateur saisit le nouveau mot de passe et la confirmation. 4. Le View Profil valide la saisie (champs non vides, correspondance des mots de passe). 5. Si la saisie est valide, envoie la requête `changePassword()` au `UserController`. 6. Le backend (`UserController`) met à jour le mot de passe dans l'entité `User`. 7. Le `UserController` renvoie le résultat au View Profil. 8. Le View Profil affiche un message de confirmation en cas de succès et met à jour l'affichage.
Scénario d'échec	1. Si les champs sont vides ou que la confirmation ne correspond pas, View Profil bloque l'envoi et affiche un message d'erreur. 2. Si le `UserController` rencontre un problème lors de la mise à jour, un message d'erreur est affiché à l'utilisateur.

3.2.2.3.2 Diagramme de Séquence : Changer le Mot de Passe

Ce diagramme montre les interactions lors de la modification du mot de passe par un utilisateur.

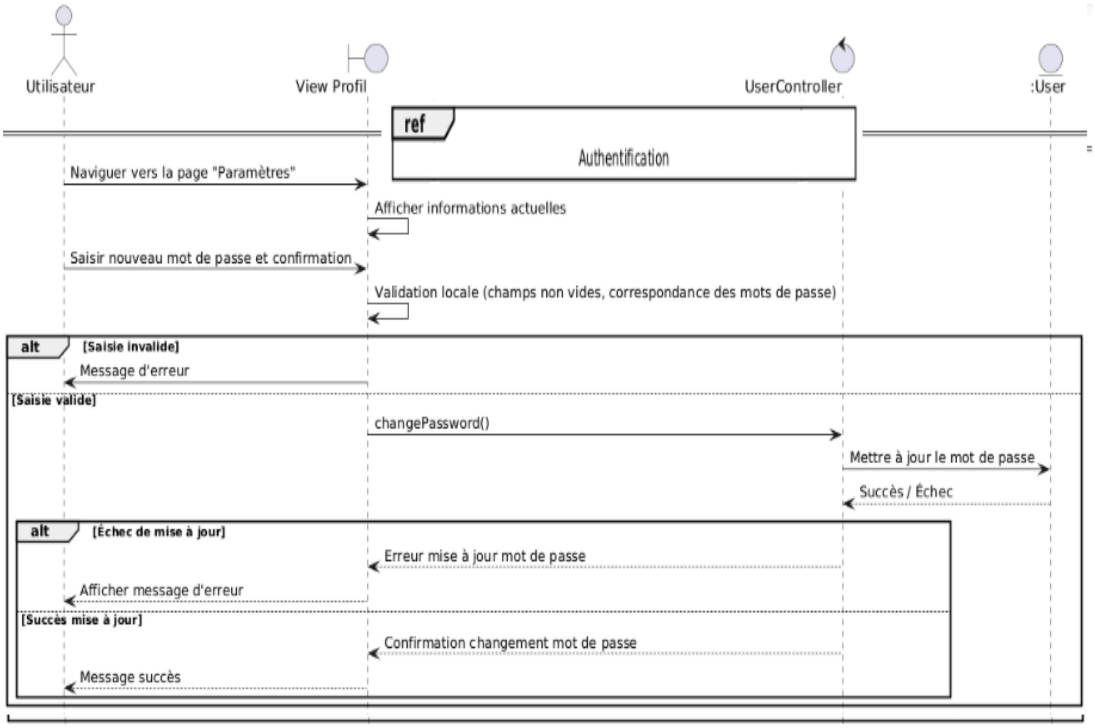


FIGURE 3.5 – Diagramme de séquence de changer le mot de passe

3.2.3 Réalisation du Sprint 1

La première phase de développement a donné lieu à la mise en œuvre des interfaces fondamentales de la plateforme. Ces écrans sont essentiels pour l'authentification, la gestion des utilisateurs par les administrateurs et la personnalisation des profils, établissant une base sécurisée et fonctionnelle.

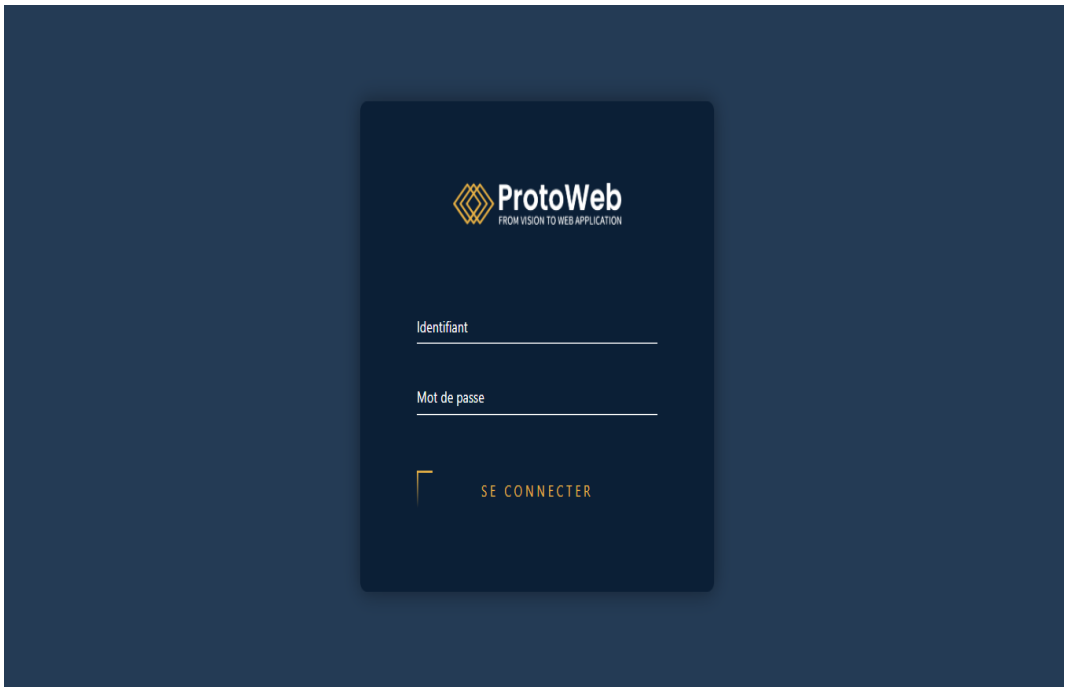


FIGURE 3.6 – Interface de Connexion.

Ceci est l'interface utilisateur pour la connexion, où les utilisateurs saisissent leurs identifiants et mots de passe pour accéder à l'application.

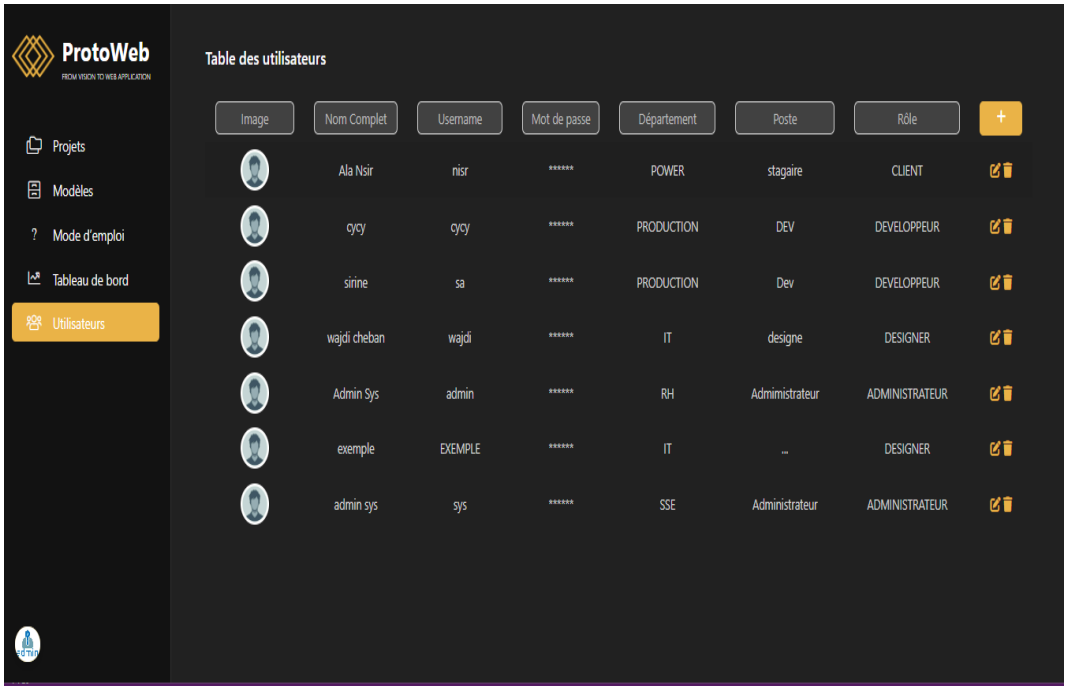


FIGURE 3.7 – Interface de gestion des projets.

Cette interface reflète une conception centrée sur l'utilisateur, offrant une vue claire de tous les comptes avec des options d'édition et de suppression accessibles, ainsi qu'un bouton dédié à la création de nouveaux utilisateurs. Elle représente l'aboutissement des user stories de ce Sprint, garantissant une gestion efficace et centralisée.

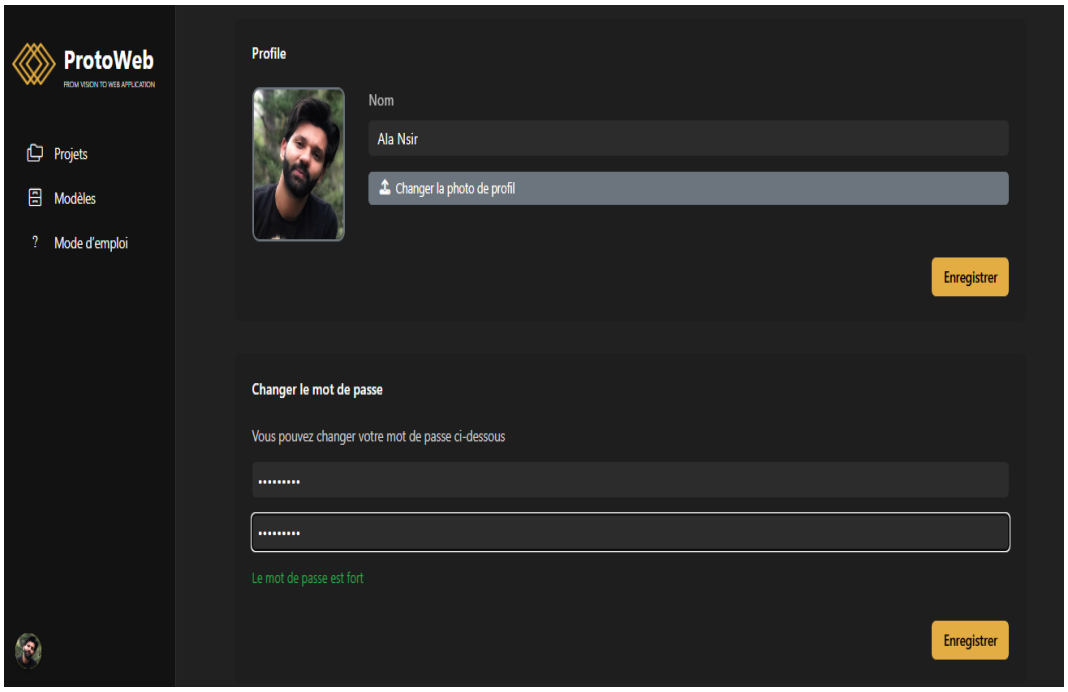


FIGURE 3.8 – Interface de gestion des projets.

Cet écran permet à l'utilisateur de gérer ses informations personnelles, notamment la modification de son nom, le changement de sa photo de profil et la mise à jour de son mot de passe.

3.2.4 Phase de Test

La première phase de développement a donné lieu à la mise en œuvre des interfaces fondamentales de la plateforme. Ces écrans sont essentiels pour l'authentification, la gestion des utilisateurs par les administrateurs et la personnalisation des profils, établissant une base sécurisée et fonctionnelle. La phase de test de ce premier Sprint consolidé a été exhaustive, visant à valider l'intégralité du module utilisateur, depuis l'authentification jusqu'à la gestion des profils et des comptes par les administrateurs.
Une attention particulière a été portée sur la sécurité, la robustesse de l'API et la convivialité des interfaces. Des tests unitaires et d'intégration ont été systématiquement appliqués à chaque fonctionnalité.
Pour l'authentification, des scénarios de succès (connexion avec différents rôles) et d'échec (identifiants incorrects, champs vides) ont été testés.
La gestion des utilisateurs par l'administrateur a impliqué la vérification complète des opérations CRUD : création d'utilisateurs avec divers rôles et données, consultation de la liste, modification de leurs informations, suppression de comptes, et recherche avec différentes combinaisons de critères pour assurer la précision et la performance.
Enfin, la gestion du profil utilisateur a été validée en testant la modification du nom, le téléchargement de photos de profil, et le changement de mot de passe (y compris les scénarios d'échec comme des mots de passe non concordants ou des fichiers images invalides).
L'outil Postman a été un élément clé de cette phase, utilisé pour simuler les requêtes vers l'API et vérifier les réponses du backend.
.

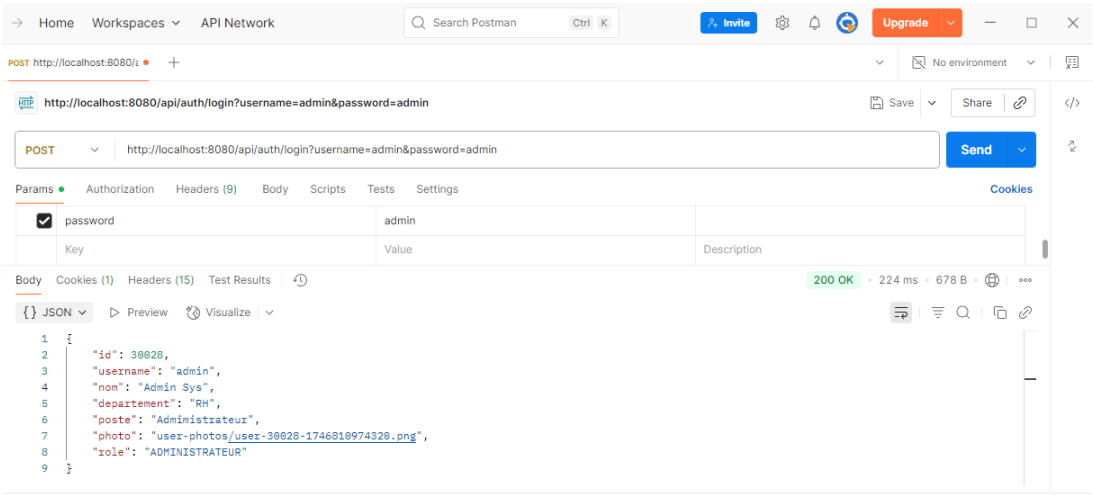


FIGURE 3.9 – Test Postman – Authentification réussie

Cette figure illustre une requête POST réussie via Postman pour l'authentification. Elle montre la soumission d'identifiants valides et la réception d'une réponse HTTP 200 OK.

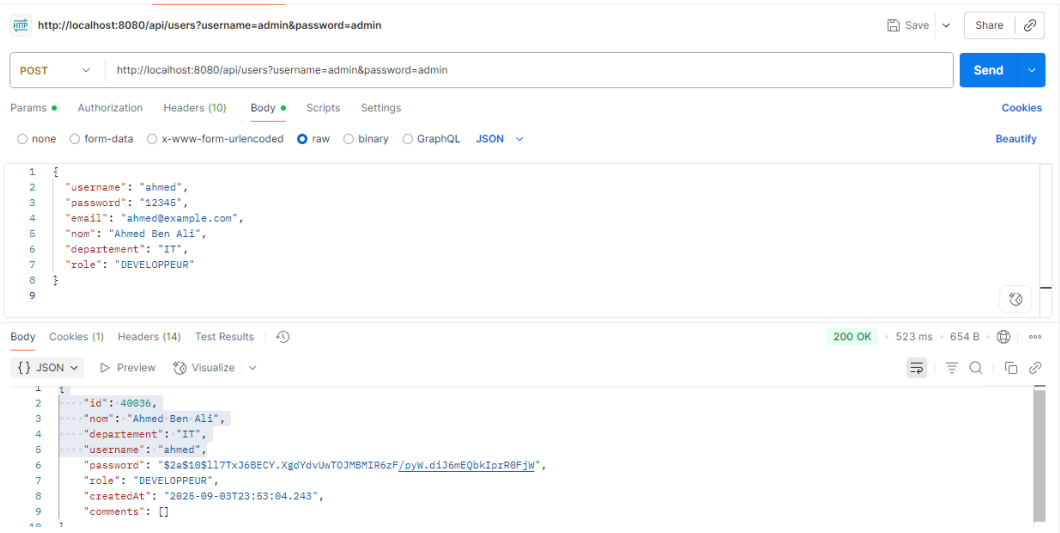


FIGURE 3.10 – Test Postman – Création d'un utilisateur.

Cette figure présente un exemple de requête Postman validant la création d'un utilisateur. Elle confirme le bon fonctionnement de l'API, l'intégration des logiques métier et la communication efficace avec le backend.

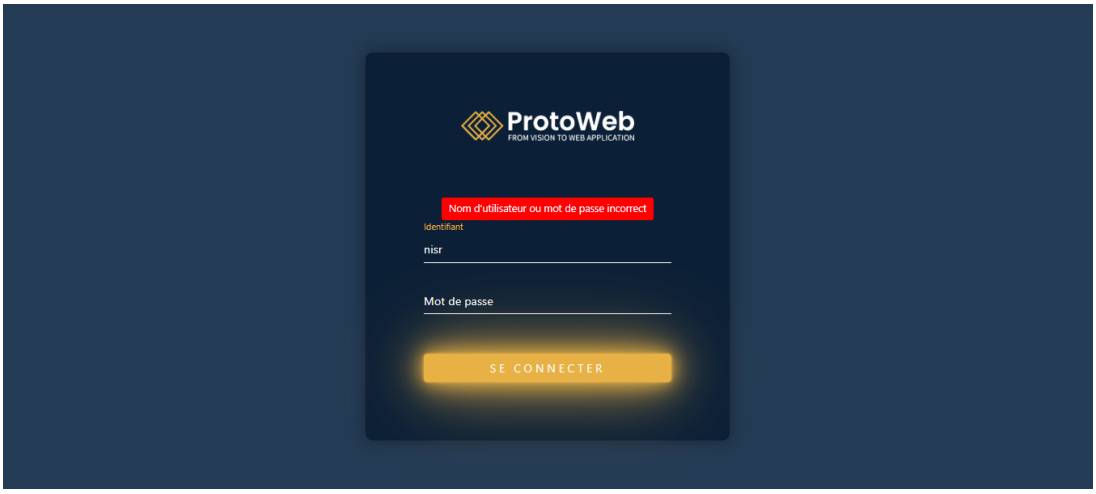


FIGURE 3.11 – Erreur d'authentification - Identifiants incorrects

Cette image prouve que le système gère correctement les tentatives de connexion avec des identifiants invalides, affichant un message d'erreur clair pour informer l'utilisateur de l'échec de l'authentification.

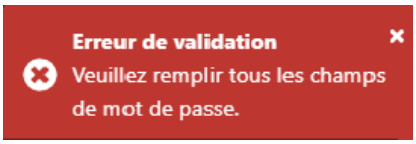


FIGURE 3.12 – Notification d'erreur - Champs de mot de passe incomplets

Cette notification d'erreur confirme que la validation côté client fonctionne, en alertant l'utilisateur de l'obligation de remplir tous les champs de mot de passe avant soumission.

3.3 Sprint 2 : Conception et gestion des projets ainsi que l'intégration des modèles (templates)

3.3.1 Présentation du Sprint 2

Le Sprint 2 vise à enrichir la plateforme Protoweb en introduisant la gestion complète des projets et l'intégration des modèles (templates). Il constitue la suite logique après l'authentification des utilisateurs, en leur offrant un espace de travail permettant de créer, organiser et personnaliser leurs projets de manière intuitive. Ce cycle de développement a permis aux utilisateurs de créer de nouveaux projets à partir de zéro ou en utilisant des modèles existants, tandis que les designers ont pu transformer leurs projets en modèles réutilisables pour simplifier et accélérer la création future. L'objectif principal de ce Sprint était de structurer l'organisation du contenu utilisateur et de favoriser la réutilisation des composants, tout en garantissant une expérience fluide et efficace pour tous les acteurs de la plateforme.

3.3.2 Spécification fonctionnelle

Cette section détaille les fonctionnalités attendues pour ce Sprint, en précisant les comportements du système pour chaque scénario d'utilisation et les interactions entre les utilisateurs et les modules de gestion de projets et de modèles.

3.3.2.1 Sprint Backlog

Ce Sprint a été planifié avec un backlog ambitieux, regroupant toutes les User Stories liées à la gestion des projets par l'utilisateur et à la gestion des modèles par les designers et leur utilisation. L'objectif était de livrer un module de gestion de contenu complet et fonctionnel.

Table 3.5 – Backlog du Sprint 2

User Story	Description	Complexité (Points)	Estimation (jours)
US5.1	En tant qu'utilisateur, je souhaite pouvoir créer un nouveau projet à partir d'une page blanche ou d'un modèle existant.	5	2
US5.2	En tant qu'utilisateur, je veux voir la liste de tous mes projets existants sous forme de cartes avec un aperçu visuel.	8	3
US5.3	En tant qu'utilisateur, je veux pouvoir modifier les informations de mes projets (nom, description).	3	2
US5.4	En tant qu'utilisateur, je veux avoir la possibilité de supprimer définitivement un projet dont je n'ai plus besoin.	3	2
US5.5	En tant qu'utilisateur, je veux pouvoir rechercher un projet par son nom et trier la liste (par nom A-Z, les plus récents, les plus anciens) afin de retrouver rapidement un projet spécifique.	5	2
US6.1	En tant que Designer, je veux pouvoir marquer un de mes projets comme "modèle" pour le rendre accessible à tous les autres utilisateurs.	5	2
US6.2	En tant que Designer, je veux pouvoir retirer le statut de "modèle" à un de mes projets si je ne souhaite plus qu'il soit utilisé comme base.	3	2
US6.3	En tant qu'utilisateur, je veux voir une galerie de tous les modèles disponibles.	5	3
US6.4	En tant qu'utilisateur, lorsque je crée un projet à partir d'un modèle, je veux que le système crée une copie indépendante, que je puisse modifier sans affecter le modèle original.	8	2

3.3.2.2 Diagramme de cas d'utilisation global du Sprint 2

Les interactions clés des différents acteurs avec le système ont été modélisées, se concentrant sur la gestion des projets et l'utilisation des modèles. Les diagrammes de cas d'utilisation ci-dessous présentent une vue d'ensemble des fonctionnalités implémentées, illustrant la manière dont les acteurs (Utilisateur, Designer) interagissent avec les modules de gestion de projet et de modèles.

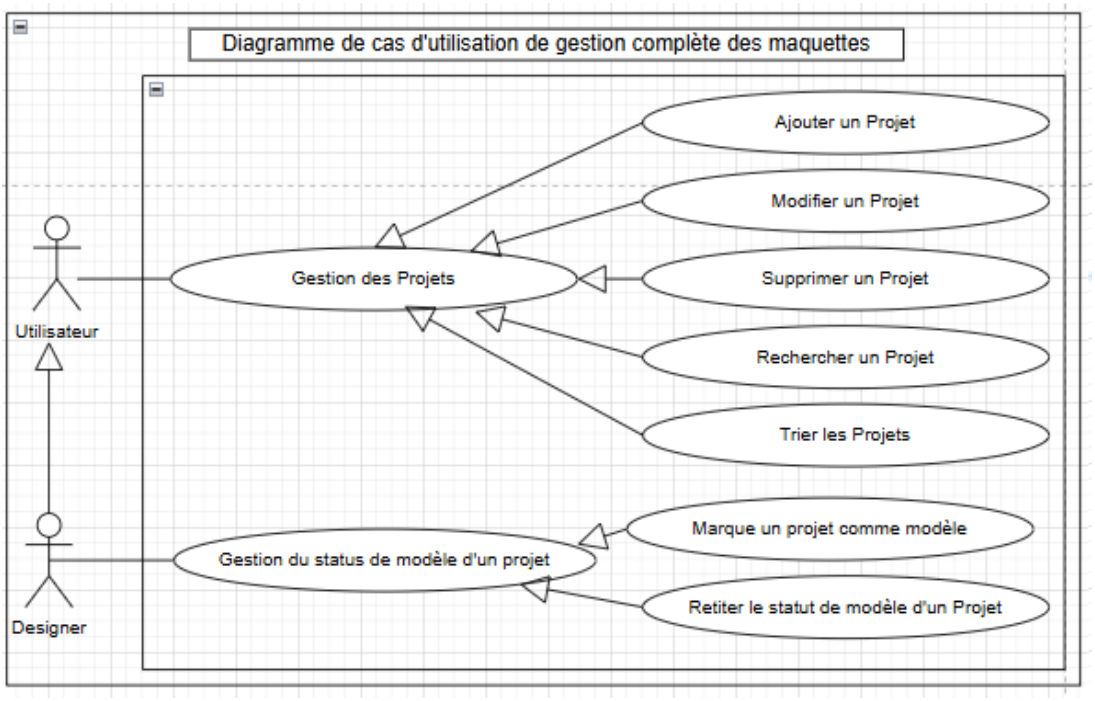


FIGURE 3.13 – Diagramme de cas d'utilisation de gestion complète des projets et modèles.

Ces diagrammes mettent en évidence deux composantes principales : la Gestion des Projets qui permet aux utilisateurs de créer, visualiser, modifier et supprimer leurs maquettes, et la Gestion des Modèles qui habilite les Designers à transformer leurs projets en templates publics.

3.3.2.3 Diagramme de classe du Sprint 2

Le diagramme de classes présenté illustre la structure des entités et leurs relations au sein du système pour ce Sprint. Il met en évidence les interactions entre les classes Utilisateur et Maquette, notamment l'association "Créer" qui permet à un utilisateur de générer plusieurs maquettes. Cette organisation est fondamentale pour supporter les fonctionnalités de gestion de projets et de création de maquettes développées.

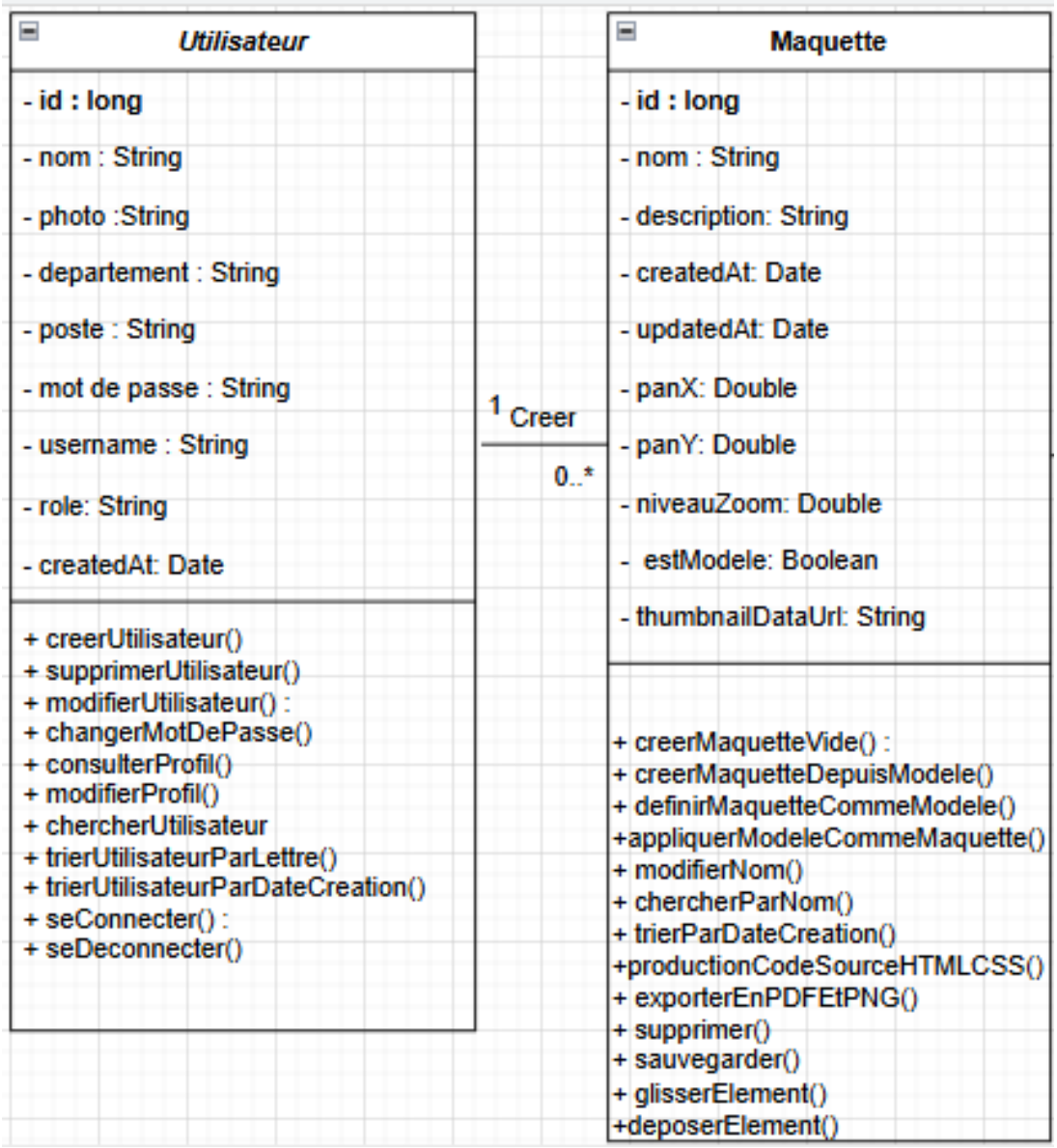


FIGURE 3.14 – Structure des classes – Sprint 2.

3.3.3 Raffinement des diagrammes de cas d'utilisation

Pour mieux comprendre les interactions, voici les descriptions détaillées de quelques cas d'utilisation représentatifs de ce Sprint, suivies de leurs diagrammes de séquence illustrant les flux d'informations et les échanges entre les différents composants du système.

3.3.3.1 Analyse de cas " Ajouter un projet à partir d'un modèle"

Cette analyse décrit le processus par lequel un utilisateur peut créer un nouveau projet en se basant sur un modèle existant, facilitant ainsi la réutilisation rapide de maquettes.

3.3.3.1.1 Description textuelle de Cas d'Utilisation : Ajouter un projet à partir d'un modèle

Ce cas d'utilisation détaille le processus permettant à un utilisateur d'initier un nouveau projet en se basant sur un modèle prédéfini, facilitant ainsi le démarrage rapide de nouvelles maquettes.

Table 3.6 – Description du scénario « Ajouter un projet à partir d'un modèle »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Ajouter un projet à partir d'un modèle
Acteur	Admin, Client, Designer
Pré-condition	L'acteur est authentifié et se trouve sur la page « Projets » ou sur la page « Modèles ».
Post-condition	Un nouveau projet, copie du modèle sélectionné, est créé et ajouté à la liste des projets de l'acteur. L'acteur est redirigé vers l'interface d'édition de ce nouveau projet.
Scénario nominal	1. Le système affiche la page « Projets » avec l'option *Commencer à partir d'un modèle*. 2. L'acteur clique sur l'option ou sélectionne un modèle dans la galerie. 3. Le système présente une galerie des modèles disponibles. 4. L'acteur sélectionne un modèle de son choix. 5. Le système ouvre une fenêtre modale et propose un nom par défaut. 6. L'acteur peut modifier le nom, ajouter une description optionnelle et clique sur « Créer ». 7. L'interface (VueProjet) envoie une requête `createProjectFromTemplate()` au `ProjectController` . 8. Le contrôleur crée une copie du modèle en base et l'associe à l'utilisateur. 9. Le contrôleur retourne le résultat. 10. Le système redirige l'acteur vers l'éditeur du nouveau projet et affiche une notification de succès.
Scénario d'échec	— Si aucun modèle n'est disponible, l'option est désactivée ou affiche « Aucun modèle disponible ». — Si l'acteur annule la création, retour à la page « Projets » sans création. — En cas d'échec en base, le système affiche un message d'erreur approprié.

3.3.3.1.2 Diagramme de Séquence : Ajouter un projet à partir d'un modèle

Ce diagramme illustre le déroulement de la création d'un projet basé sur un modèle.

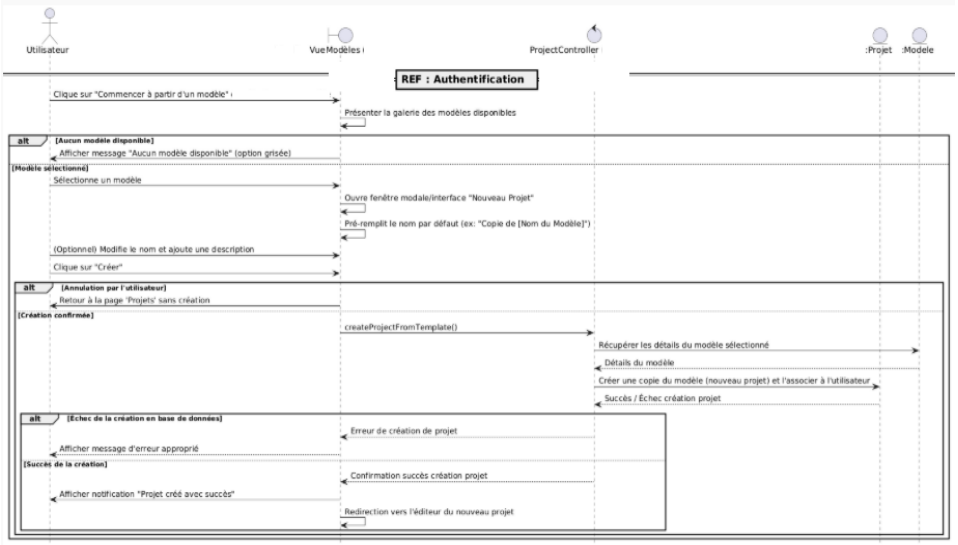


FIGURE 3.15 – Diagramme de séquence d'ajouter un projet à partir d'un modèle

3.3.3.2 Analyse de cas " Retirer un modèle de la liste"

Cette analyse présente le scénario où un designer retire le statut de modèle d'un projet, le rendant privé et non accessible aux autres utilisateurs.

3.3.3.2.1 Description textuelle de Cas d'Utilisation : Retirer un modèle de la liste Ce cas d'utilisation décrit le processus par lequel un designer retire le statut de "modèle" à l'un de ses projets, le rendant ainsi indisponible pour les autres utilisateurs et le transformant en un projet privé.

Table 3.7 – Description du scénario « Retirer un modèle de la liste »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Retirer un modèle de la liste
Acteur	Designer
Pré-condition	L'acteur est authentifié et est le propriétaire d'un projet qui a été préalablement marqué comme modèle.
Post-condition	Le projet n'apparaît plus dans la galerie des modèles disponibles. Il redevient un projet privé, visible uniquement par son propriétaire et les personnes avec qui il est partagé.
Scénario nominal	— Le designer se rend sur la page "Modèles". — Il localise le projet modèle qu'il souhaite retirer et clique sur le menu d'options (icône "...") associé. — Le designer sélectionne l'option "Retirer comme modèle". — Le système demande une confirmation pour éviter toute action accidentelle. — Le designer confirme son choix. — L'interface VueModèles envoie une requête `removeTemplateStatus()` au `ProjetController`. — Le contrôleur traite la requête, met à jour le statut du projet dans la base de données. — Le contrôleur retourne le résultat de l'opération. — Le projet est immédiatement retiré de la galerie publique des modèles. VueModèles met à jour l'affichage et une notification de succès s'affiche.

3.3.3.2.2 Diagramme de Séquence : Retirer un modèle de la liste

Ce diagramme montre les interactions entre le Designer et le système lors de la suppression du statut de modèle pour un projet.

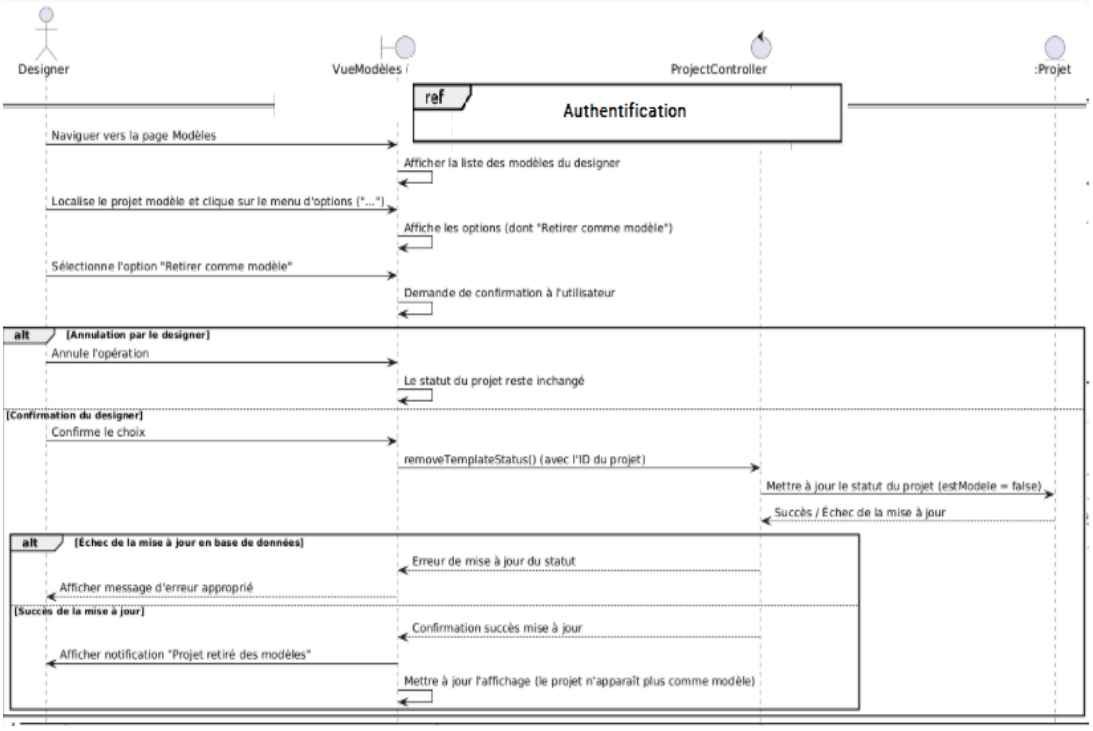


FIGURE 3.16 – Diagramme de séquence de retrait d'un modèle.

3.3.4 Réalisation du Sprint 2

À la fin de ce Sprint, plusieurs interfaces clés ont été livrées, offrant aux utilisateurs une expérience fluide pour la gestion de leurs projets et l'exploitation des modèles.

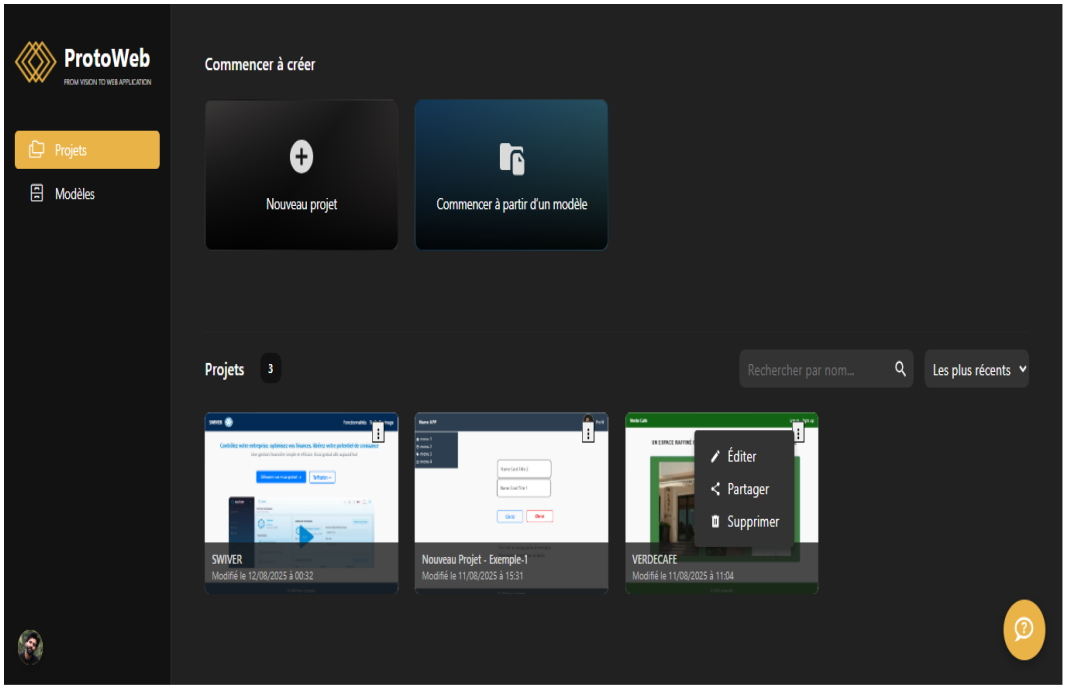


FIGURE 3.17 – Interface de gestion des projets.

Cette figure représente l'interface principale de gestion des projets, où les utilisateurs peuvent visualiser, accéder et interagir avec leurs différentes maquettes.

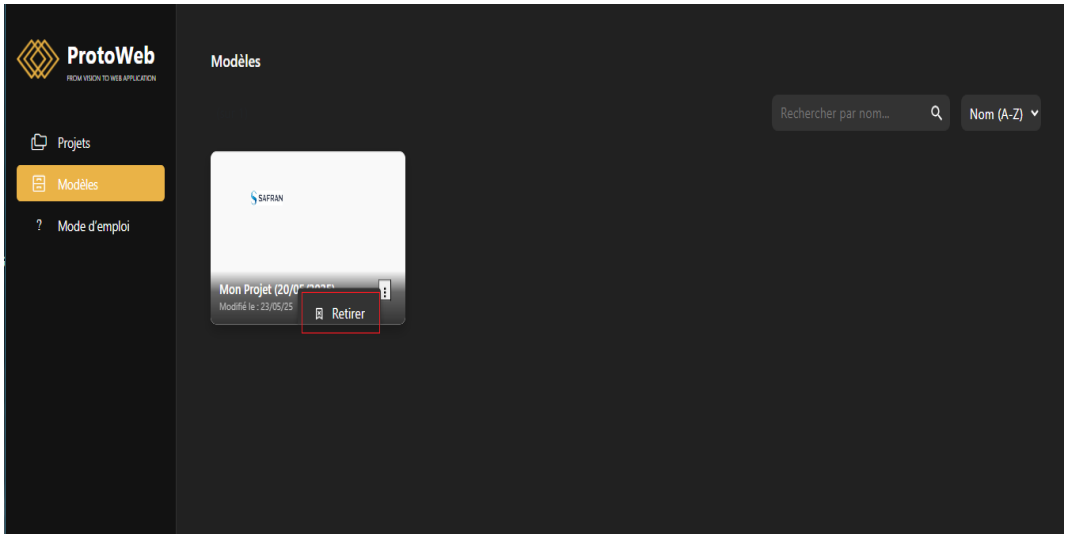


FIGURE 3.18 – Interface de gestion des modèles existants.

Cette interface présente une collection de modèles prêts à l'emploi, accessibles à tous les utilisateurs pour faciliter la création de nouveaux projets.

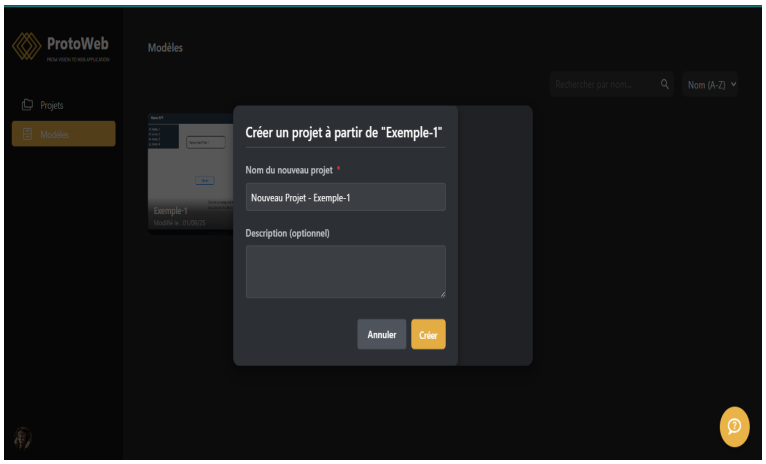


FIGURE 3.19 – Interface d'ajout de projet à partir d'un modèle.

Cette figure illustre la fenêtre modale de "Création d'un projet à partir d'un modèle existant". L'utilisateur peut y définir le nom du nouveau projet, potentiellement basé sur un modèle sélectionné, et ajouter une description facultative avant de valider la création.

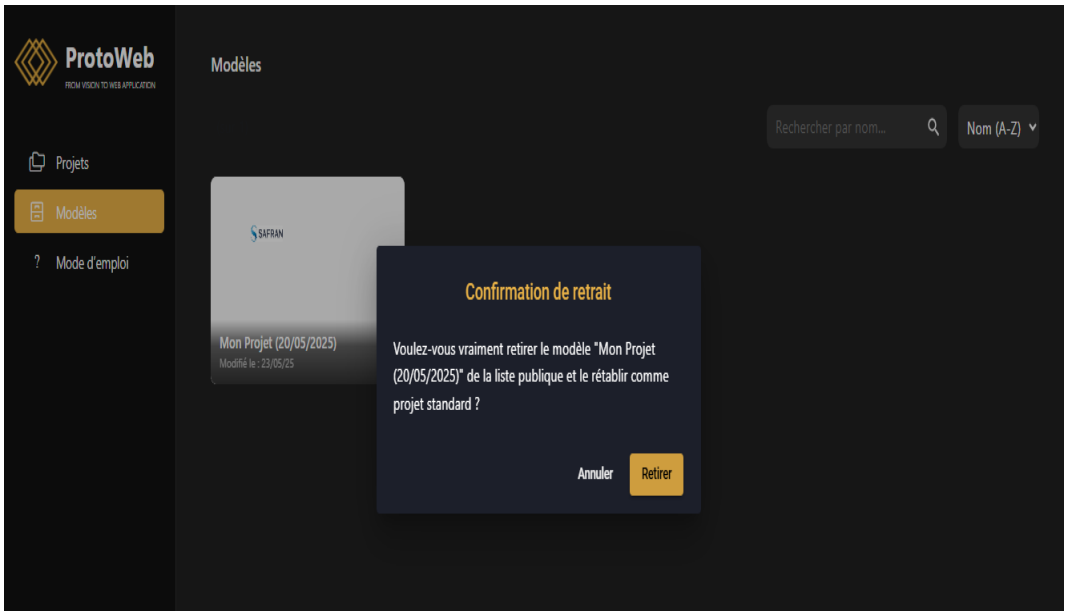


FIGURE 3.20 – Interface de confirmation pour retirer un projet du statut de modèle.

Cette figure présente une boîte de dialogue de "Confirmation de retrait". Elle apparaît lorsqu'un designer souhaite retirer un projet du statut de modèle public, le rétablissant en tant que projet standard, et demande une confirmation avant de finaliser l'action.

3.3.5 Phase de Test

La phase de test de ce deuxième Sprint a été menée de manière approfondie pour valider l'intégralité du module de gestion de projets et de modèles. Une attention particulière a été portée sur la création et la manipulation des projets, l'exactitude de l'affichage des listes, et la gestion du statut de modèle. Des tests unitaires et d'intégration ont été appliqués à chaque fonctionnalité.
Pour la gestion des projets, des scénarios de succès (création à partir de zéro, modification des informations, suppression, recherche et tri) et d'échec (champs obligatoires vides, erreurs de suppression) ont été testés. La gestion des modèles a nécessité la vérification de la capacité d'un designer à marquer un projet comme modèle, à le visualiser dans une galerie, et à le retirer, ainsi que la capacité des autres utilisateurs à créer des copies indépendantes à partir de ces modèles.
L'outil Postman a continué d'être un pilier de cette phase, utilisé pour simuler les requêtes vers l'API backend et vérifier la robustesse et la justesse des réponses concernant la création, la modification et la gestion des statuts de projet et de modèle.

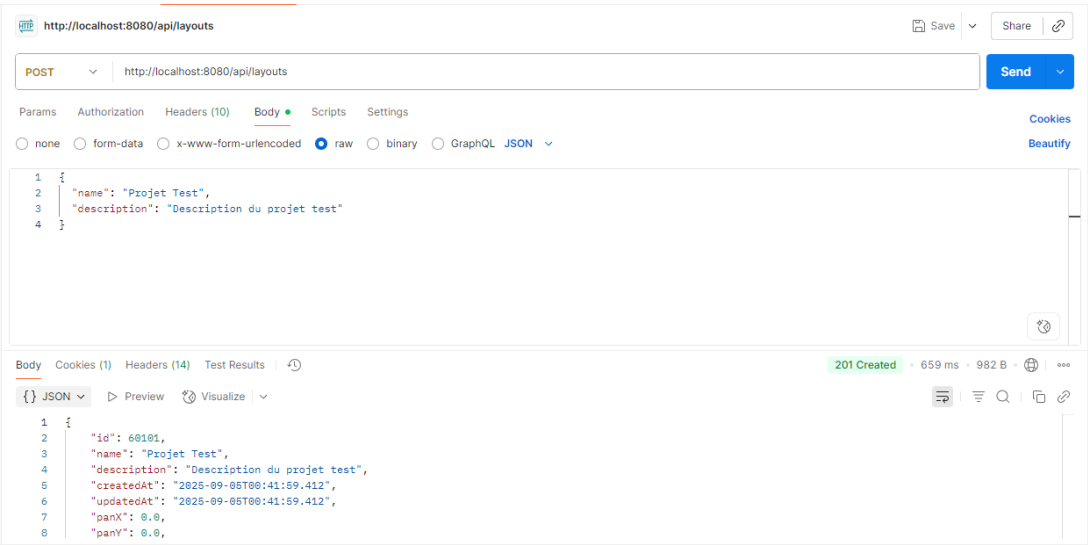


FIGURE 3.21 – Test Postman – Nouvelle création de projet

Cette figure illustre une requête POST réussie via Postman pour la création d'un nouveau projet. Elle montre la soumission des détails du projet et la réception d'une réponse HTTP 201 Created, confirmant la bonne intégration avec l'API.

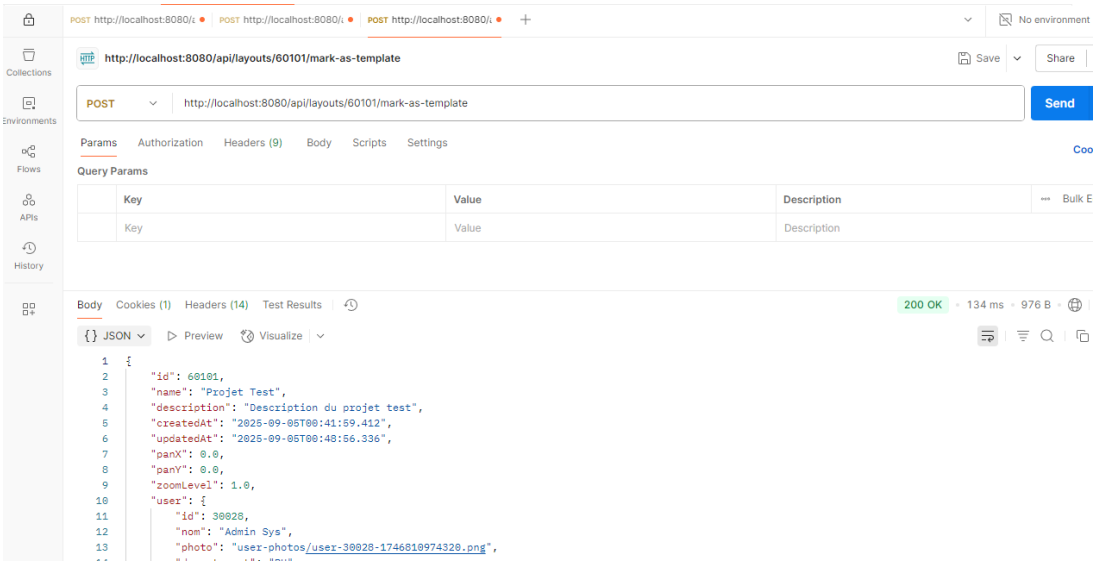


FIGURE 3.22 – Test Postman – Modification du statut du projet

Cette figure présente un exemple de requête Postman validant la modification du statut d'un projet en "modèle". Elle confirme le bon fonctionnement de l'API pour les mises à jour et la gestion des modèles.

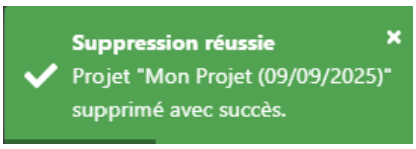


FIGURE 3.23 – Notification de succès - Suppression de projet

Cette notification confirme visuellement la réussite de la suppression d'un projet, assurant à l'utilisateur que l'action a été prise en compte et qu'il n'y a pas eu de perte de données inattendue.

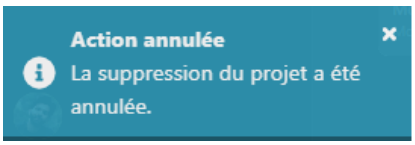


FIGURE 3.24 – Notification d'action annulée - Suppression de projet

Cette notification confirme le bon fonctionnement de la gestion des annulations, assurant que les utilisateurs peuvent interrompre des opérations potentiellement destructrices sans perte de données.

FIGURE 3.25 – Notification d'erreur - Champs vide

Cette notification confirme une erreur de validation lors de la modification du nom du projet, car le champ reste vide.

3.4 Sprint 3 : Développement de l'éditeur visuel avec glisser-déposer et personnalisation des composants

3.4.1 Présentation du Sprint 3

Ce troisième Sprint constitue une étape majeure de la Release 1, car il marque le développement de l'éditeur visuel, le cœur de l'application Protoweb. L'objectif était de permettre aux utilisateurs de créer et de manipuler des maquettes de manière intuitive. Ce Sprint a été divisé en deux axes principaux : l'implémentation de la fonctionnalité de glisser-déposer pour positionner les éléments, et la mise en place des outils de personnalisation pour modifier leurs propriétés. Cela inclut la définition d'une bibliothèque de composants, la gestion de leur positionnement sur une zone de travail, et la modification en temps réel de leurs attributs visuels et fonctionnels via un panneau de propriétés contextuel.

3.4.2 Spécification fonctionnelle

3.4.2.1 Sprint Backlog

Le backlog du Sprint 3 a été conçu pour couvrir à la fois l'implémentation du glisser-déposer et et la personnalisation des composants. Il regroupe toutes les User Stories essentielles pour établir un éditeur visuel fonctionnel et interactif.

Table 3.8 – Backlog du Sprint 3

User Story	**Description**	**Complexité (Points)**	**Estimation (jours)**
US 4.1	En tant que développeur, je veux définir une bibliothèque de composants (boutons, textes, formes, templates...) dans un fichier de données (components-data.ts), chacun avec ses propriétés initiales (taille, couleur, etc.), pour fournir une base d'éléments prêts à l'emploi.	8	3
US 4.2	En tant qu'utilisateur, je veux une interface de bibliothèque claire où les composants sont classés par type (Template, Shape, Text...) et où je peux rechercher un élément par son nom, afin de trouver rapidement ce dont j'ai besoin.	5	3
US 4.3	En tant qu'utilisateur, je veux pouvoir glisser un composant depuis la bibliothèque et le déposer sur la zone de travail, pour commencer à assembler visuellement ma maquette.	13	4
US 3.1	En tant qu'utilisateur, je veux pouvoir cliquer sur un élément dans la zone de travail pour le sélectionner, ce qui doit afficher ses propriétés modifiables dans un panneau dédié.	5	2
US 3.2	Une fois un élément sélectionné, je veux pouvoir modifier ses propriétés visuelles (couleur, taille, texte, typographie, bordure, opacité...) via des contrôles intuitifs et voir les changements s'appliquer en temps réel sur l'élément.	13	5
US 3.3	En tant qu'utilisateur, je veux disposer d'outils pour aligner l'élément sélectionné sur la page et pour gérer sa superposition (avancer, reculer, mettre au premier plan), afin de peaufiner la mise en page.	8	1

User Story	Description	Complexité (Points)	Estimation (jours)
US 3.4	En tant qu'utilisateur, je souhaite pouvoir redimensionner un élément directement sur la maquette. Pour cela, je veux que des poignées de redimensionnement apparaissent sur les coins et les bords de l'élément sélectionné, me permettant de l'agrandir ou de le réduire avec la souris, pour un ajustement visuel et intuitif.	8	2

3.4.2.2 Diagramme de cas d'utilisation global du Sprint 3

Les diagrammes de cas d'utilisation ci-dessous ont été affinés pour refléter les interactions des utilisateurs avec l'éditeur visuel. Ils présentent une vue d'ensemble des fonctionnalités clés, notamment la gestion des éléments par glisser-déposer et leur personnalisation. Ces diagrammes illustrent comment les administrateurs, clients et designers peuvent interagir avec les modules de création et de modification d'éléments dans l'éditeur.

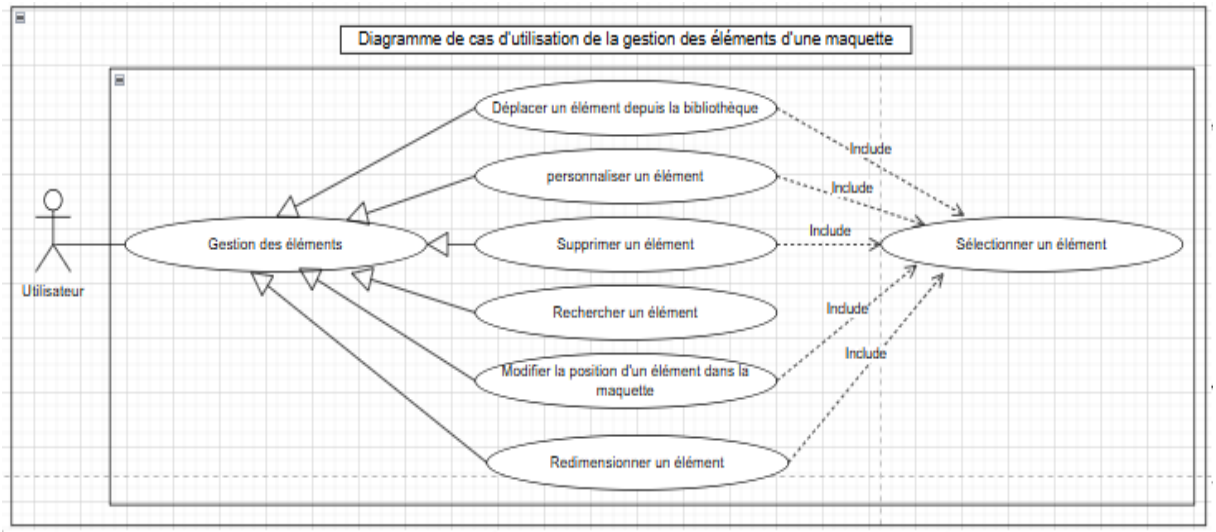


FIGURE 3.26 – Diagramme de cas d'utilisation de la gestion des éléments d'une maquette.

Ces diagrammes mettent en évidence deux composantes principales : la gestion des éléments par glisser-déposer depuis une bibliothèque vers la zone de travail, et la personnalisation de ces éléments via un panneau de propriétés dynamique et des manipulations directes (redimensionnement).

3.4.2.3 Diagramme de classe du Sprint 3

Le diagramme de classes ci-dessous illustre l'organisation et les relations entre les entités clés introduites lors du Sprint 3. Il met en évidence la structuration des composants (Element, Propriete) pour la bibliothèque et leur gestion dans le contexte de l'éditeur de maquettes, ainsi que les classes nécessaires à la gestion de leurs propriétés dynamiques.

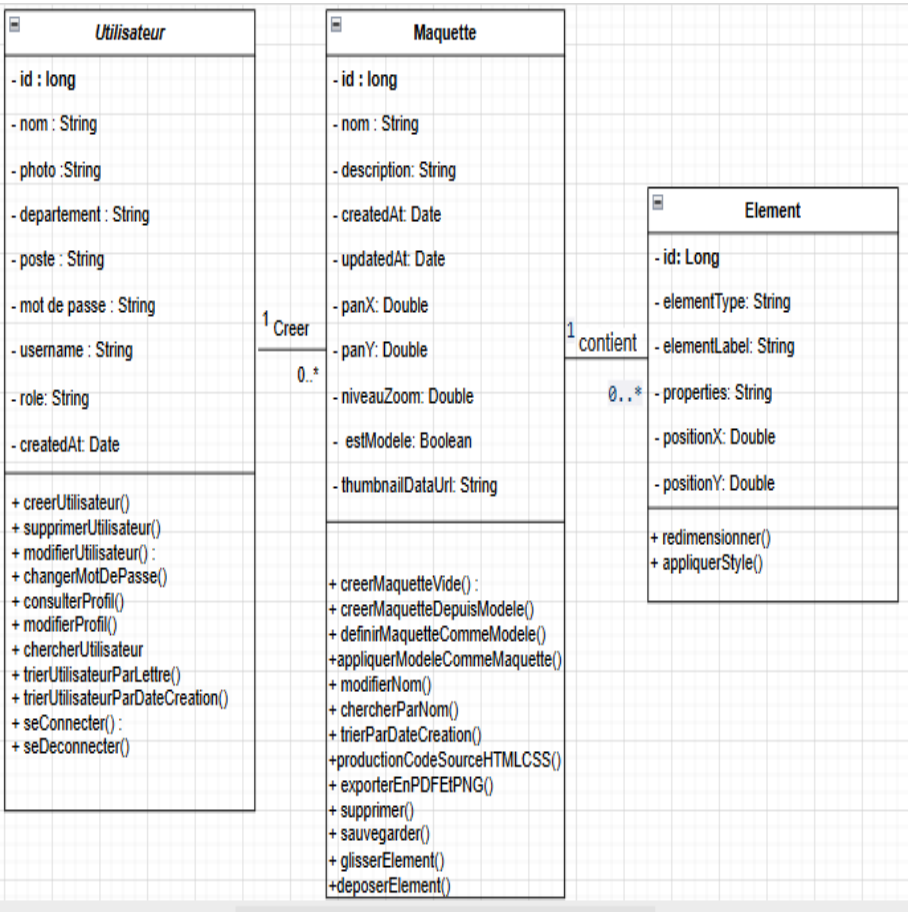


FIGURE 3.27 – Structure des classes – Sprint 3.

Cette figure présente le diagramme de classes pour le Sprint 3, focalisé sur l'entité Element (représentant les composants de la maquette) et ses Proprietes associées. Elle détaille les attributs essentiels de chaque élément (ID, type, position, taille, styles) et les opérations liées à leur création, sélection, modification et suppression, reflétant ainsi les bases fonctionnelles de l'éditeur visuel. Raffinement des diagrammes de cas d'utilisation

3.4.3 Raffinement des diagrammes de cas d'utilisation

Pour mieux comprendre les interactions, voici les descriptions détaillées de quelques cas d'utilisation représentatifs de ce Sprint, suivies de leurs diagrammes de séquence illustrant les flux d'informations et les échanges entre les différents composants du système.

3.4.3.1 Analyse de cas "Glisser-déposer d'un élément"

3.4.3.1.1 Description textuelle de Cas d'Utilisation : Glisser-déposer d'un élément

Ce cas d'utilisation décrit les étapes suivies par un utilisateur pour sélectionner un composant depuis la bibliothèque et l'ajouter visuellement sur la zone de travail, ce qui constitue la base de la construction de la maquette.

Table 3.9 – Description du scénario « Glisser-déposer et enregistrer des éléments »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Glisser-déposer et enregistrer des éléments sur la maquette
Acteur	Utilisateur, Designer, Admin
Pré-condition	L'utilisateur est authentifié et se trouve sur l'interface de l'éditeur de projet.
Post-condition	Le projet est enregistré avec les composants ajoutés/modifiés, ou un message d'erreur est affiché en cas d'échec de l'enregistrement.
Scénario nominal	1. Le système (VueÉditeur) affiche la bibliothèque de composants à l'utilisateur. 2. L'utilisateur glisse un composant depuis la bibliothèque. 3. Le système (VueÉditeur) ajoute visuellement le composant dans la zone de travail (temporairement). 4. L'utilisateur lâche le composant dans la zone de travail (Dépôt valide). 5. Le composant apparaît dans la zone de travail. 6. L'utilisateur clique sur le bouton "Enregistrer". 7. La VueÉditeur envoie une requête saveProject() au l'EelementController, incluant les composants placés et leurs positions. 8. L'EelementController persiste les composants et leurs positions dans la base de données via l'entité Element. 9. L'EelementController retourne un message de succès. 10. La VueÉditeur affiche une notification "Projet enregistré avec succès".
Scénario d'échec	— **Dépôt en dehors de la zone :** Si l'utilisateur relâche le composant en dehors de la zone de travail, aucun composant n'est ajouté et l'opération de glisser-déposer est annulée. — **Échec d'enregistrement :** Si la persistance des composants et de leurs positions échoue, le EelementController retourne une erreur ("impossible d'enregistrer le projet"). La VueÉditeur affiche alors un message d'erreur à l'utilisateur.

3.4.3.1.2 Diagramme de Séquence : Glisser-déposer d'un élément

Ce diagramme illustre le déroulement du processus de glisser-déposer d'un composant sur la zone de travail.

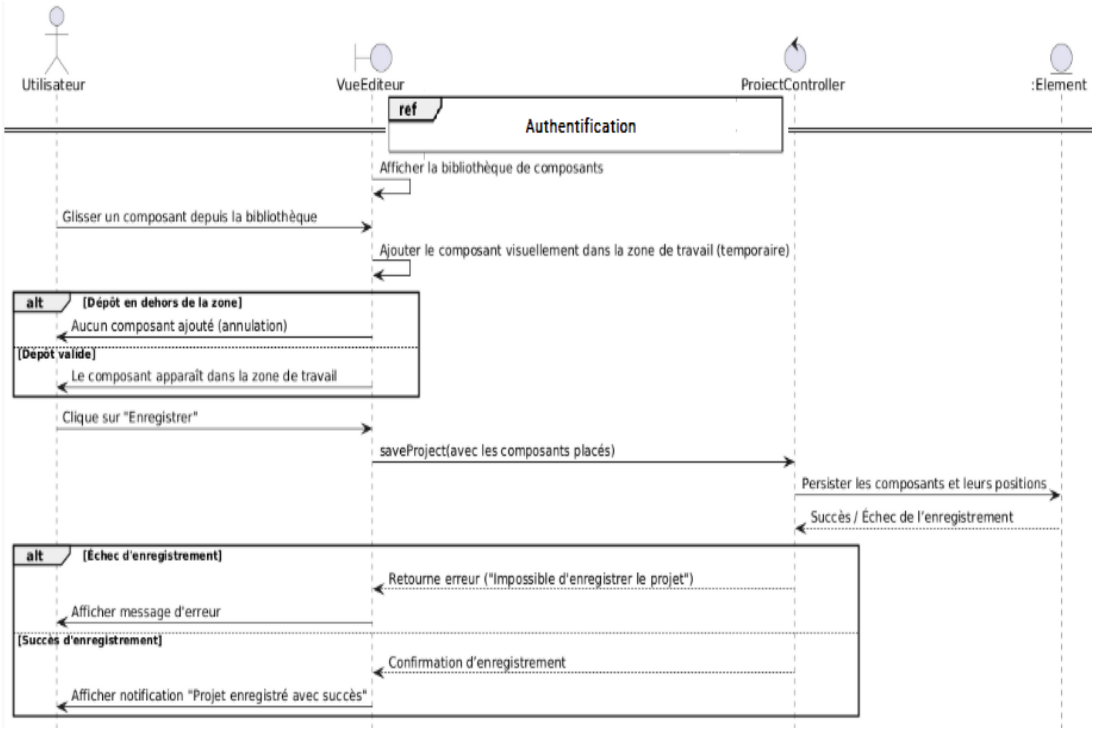


FIGURE 3.28 – Diagramme de séquence de glisser-déposer d'un élément

3.4.3.2 Analyse de cas " Personnalisation d'un élément"

3.4.3.2.1 Description textuelle de Cas d'Utilisation : Personnalisation d'un élément

Ce cas d'utilisation explique les étapes par lesquelles un utilisateur modifie un composant qu'il a placé sur la zone de travail, que ce soit par le panneau latéral ou par manipulation directe à la souris.

TABLE 3.10 – Description du scénario « Personnaliser un élément »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Modifier un élément
Acteur	Utilisateur (Admin, Client, Designer)
Pré-condition	L'utilisateur est authentifié, se trouve sur l'éditeur de projet et au moins un élément est présent dans la zone de travail.
Post-condition	Les modifications appliquées à l'élément (propriétés visuelles ou dimensions) sont sauvegardées dans le projet si l'opération réussit.

Élément du cas d'utilisation	Détails
Scénario nominal	1. L'utilisateur sélectionne un élément dans la zone de travail. 2. Le système affiche un contour avec des poignées de redimensionnement et ouvre le panneau de configuration. 3. L'utilisateur modifie l'élément soit via le panneau (ex. couleur, texte, position), soit par manipulation directe avec les poignées. 4. Le système met immédiatement à jour l'affichage de l'élément dans la zone de travail. 5. L'utilisateur clique sur le bouton *« Enregistrer »*. 6. Le système envoie les changements au contrôleur, qui tente de persister les modifications dans l'entité `ElementEntity`. 7. Selon le résultat (succès ou échec), une notification est affichée à l'utilisateur.
Scénario alternatif	Si l'utilisateur clique sur le fond de la zone de travail (hors d'un élément), le système désélectionne l'objet actif, supprime le contour et les poignées, et ferme le panneau de configuration.

3.4.3.2.2 Diagramme de Séquence : Personnalisation d'un élément

Ce diagramme montre les interactions entre l'utilisateur et le système lors de la personnalisation d'un élément sur la zone de travail.

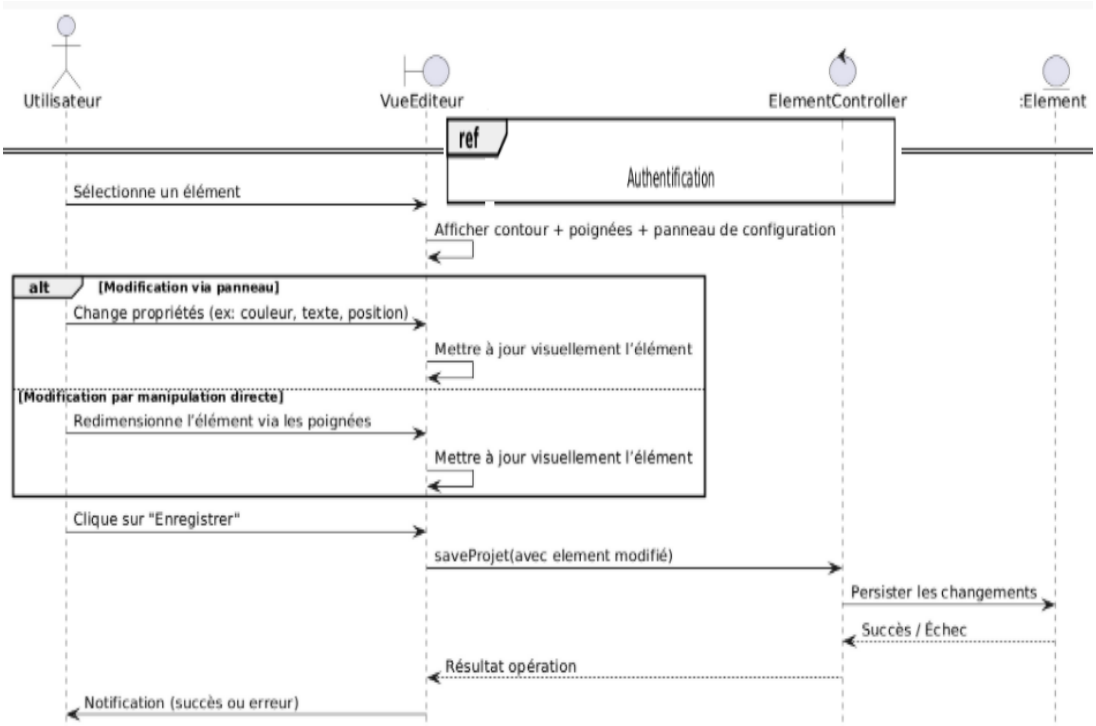


FIGURE 3.29 – Diagramme de séquence de personnalisation d'un élément

3.4.4 Réalisation du Sprint 3

Les interfaces utilisateur réalisées durant ce Sprint sont le cœur de l'expérience de l'éditeur visuel, combinant la bibliothèque de composants avec la zone de travail interactive et le panneau de person-

nalisation.

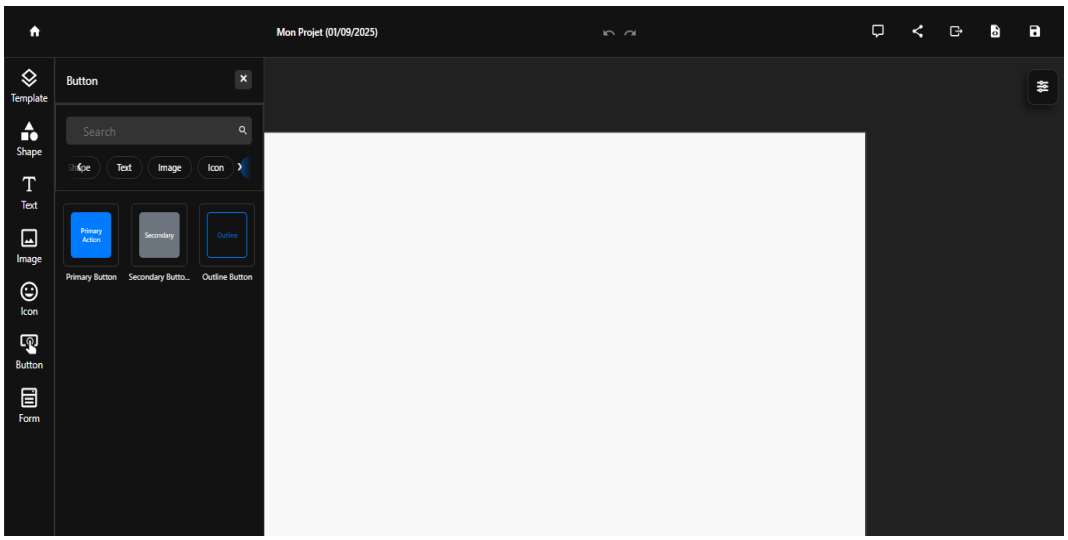


FIGURE 3.30 – Interface de l'éditeur avec bibliothèque d'éléments et zone de glisser-déposer.

Cette figure représente l'interface principale de l'éditeur, où la bibliothèque de composants est organisée sur le côté gauche, et la zone de travail permet le glisser-déposer des éléments pour assembler visuellement la maquette.

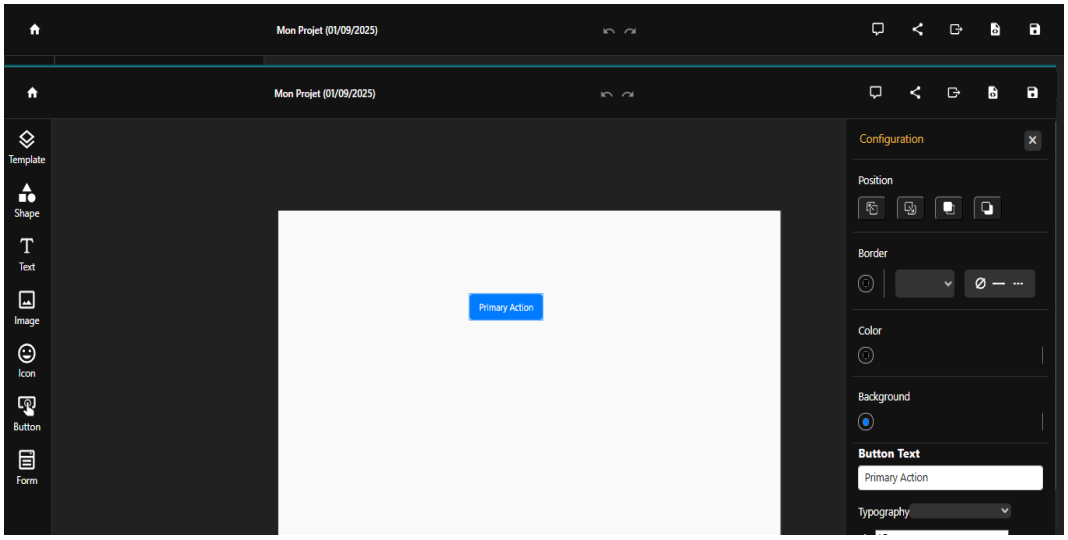


FIGURE 3.31 – Composant d'interface dédié à la personnalisation d'un élément.

Cette interface illustre concrètement la personnalisation en temps réel des composants. Les modifications apportées via le panneau latéral (taille, couleur, texte, etc.) se répercutent instantanément sur l'élément sélectionné sur la zone de travail, offrant une expérience utilisateur fluide et immédiate.

3.4.5 Phase de Test

La phase de test de ce troisième Sprint a été exhaustive, visant à valider l'intégralité des fonctionnalités de l'éditeur visuel, à savoir le glisser-déposer des éléments et leur personnalisation.

Une attention particulière a été portée sur la fluidité des interactions, la précision du positionnement et du redimensionnement, ainsi que la réactivité des mises à jour des propriétés.
Des tests unitaires et d'intégration ont été systématiquement appliqués à chaque fonctionnalité. Pour le glisser-déposer, des scénarios de succès (déplacement de divers composants sur différentes zones de la maquette) et d'échec (relâchement en dehors de la zone de travail) ont été testés pour garantir la robustesse de l'interaction. Concernant la personnalisation, les tests ont couvert la sélection d'éléments, la modification de toutes les propriétés visuelles (couleur, taille, texte, alignement, etc.) via le panneau dédié, et le redimensionnement direct à l'aide des poignées. Les mises à jour en temps réel sur la maquette ont été vérifiées pour chaque modification. De plus, les fonctionnalités d'alignement et de gestion des couches ont été validées pour assurer un contrôle précis de la mise en page.
Ces validations ont permis de s'assurer que l'éditeur offre une expérience utilisateur intuitive et fonctionnelle pour la construction et la modification des maquettes.

3.5 Sprint 4 : Implémentation du partage des projets et du module de commentaires en temps réel

3.5.1 Présentation du Sprint 4

Le quatrième Sprint vise à transformer Protoweb en un espace de travail collaboratif, en permettant aux utilisateurs de partager leurs projets avec d'autres collaborateurs, de gérer les droits d'accès et de faciliter les échanges directs via un système de commentaires en temps réel. Ce cycle de développement a permis de mettre en place la fonctionnalité de partage de projets avec attribution de rôles spécifiques (visualisation ou modification) et de développer un module de commentaires intégré à l'éditeur, offrant aux collaborateurs la possibilité de laisser des retours et des discussions contextuelles sur les maquettes. L'ensemble de ces travaux contribue à renforcer la collaboration et la communication au sein des équipes de conception.

3.5.2 Spécification fonctionnelle

3.5.2.1 Sprint Backlog

Le backlog du Sprint 4 a été conçu pour couvrir toutes les User Stories nécessaires à l'implémentation du partage de projets et du module de commentaires. Il s'articule autour des besoins des utilisateurs pour collaborer efficacement.

Table 3.11 – Backlog du Sprint 4

User Story	Description	Complexité (Points)	Estimation (jours)
$US_{7.1}$	En tant que propriétaire de projet, je veux pouvoir inviter d'autres utilisateurs à collaborer sur mes projets en recherchant des utilisateurs existants et en leur envoyant des invitations avec un niveau de permission initial ("peut voir" ou "peut modifier"), afin d'initier le partage.	8	4
$US_{7.2}$	En tant que propriétaire de projet, je veux pouvoir visualiser la liste des collaborateurs actuels sur un projet, et avoir la possibilité de modifier leurs permissions (passer de "peut voir" à "peut modifier" et inversement) ou de révoquer entièrement leur accès, afin de garder un contrôle total sur mon projet.	8	4
$US_{7.3}$	En tant qu'utilisateur collaborateur, je veux pouvoir accéder aux projets qui m'ont été partagés, avec les droits correspondants à ma permission (lecture seule si "peut voir", édition si "peut modifier"), afin de participer ou consulter les maquettes.	5	4
$US_{7.4}$	En tant qu'utilisateur, je veux pouvoir ouvrir un panneau latéral de commentaires sur la maquette, visualiser l'historique des discussions par ordre chronologique, et savoir qui a posté chaque commentaire, afin de suivre les échanges.	8	4
$US_{7.5}$	En tant qu'utilisateur, je veux pouvoir rédiger et poster de nouveaux commentaires dans le panneau dédié, et voir mon message apparaître immédiatement dans le fil de discussion, afin de contribuer aux échanges en temps réel.	8	4

3.5.2.2 Diagramme de cas d'utilisation global du Sprint 4

Le diagramme de cas d'utilisation ci-dessous reflètent les interactions des utilisateurs avec les nouvelles fonctionnalités de collaboration. Il offre une vue d'ensemble des processus de partage de projets et de gestion des commentaires.

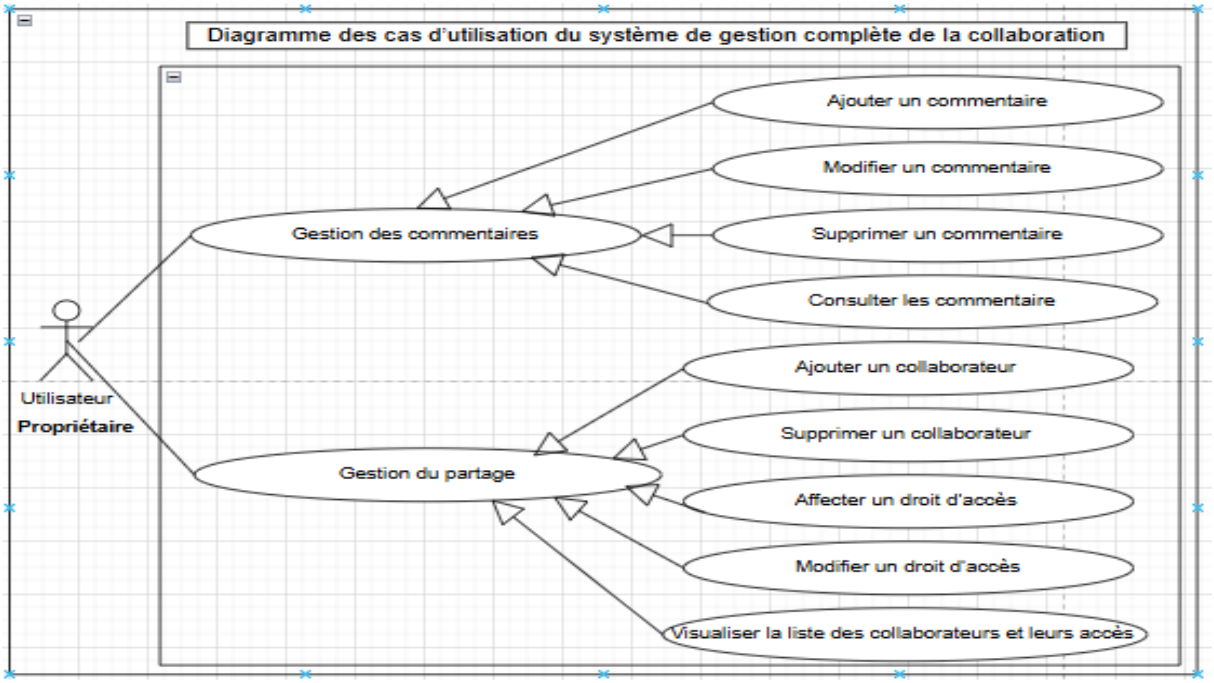


FIGURE 3.32 – Diagramme de cas d'utilisation de la collaboration

Ce diagramme met en évidence les cas d'utilisation liés à l'invitation de collaborateurs, à la gestion de leurs permissions et à la possibilité d'ajouter des commentaires sur la maquette, illustrant comment les différents acteurs interagissent avec le système de collaboration.

3.5.2.3 Diagramme de classe du Sprint 4

Le diagramme de classes ci-dessous illustre l'organisation et les relations entre les entités clés introduites ou modifiées lors du Sprint 4. Il met en évidence la structuration des classes nécessaires à la gestion des partages de projets, des permissions et des commentaires.

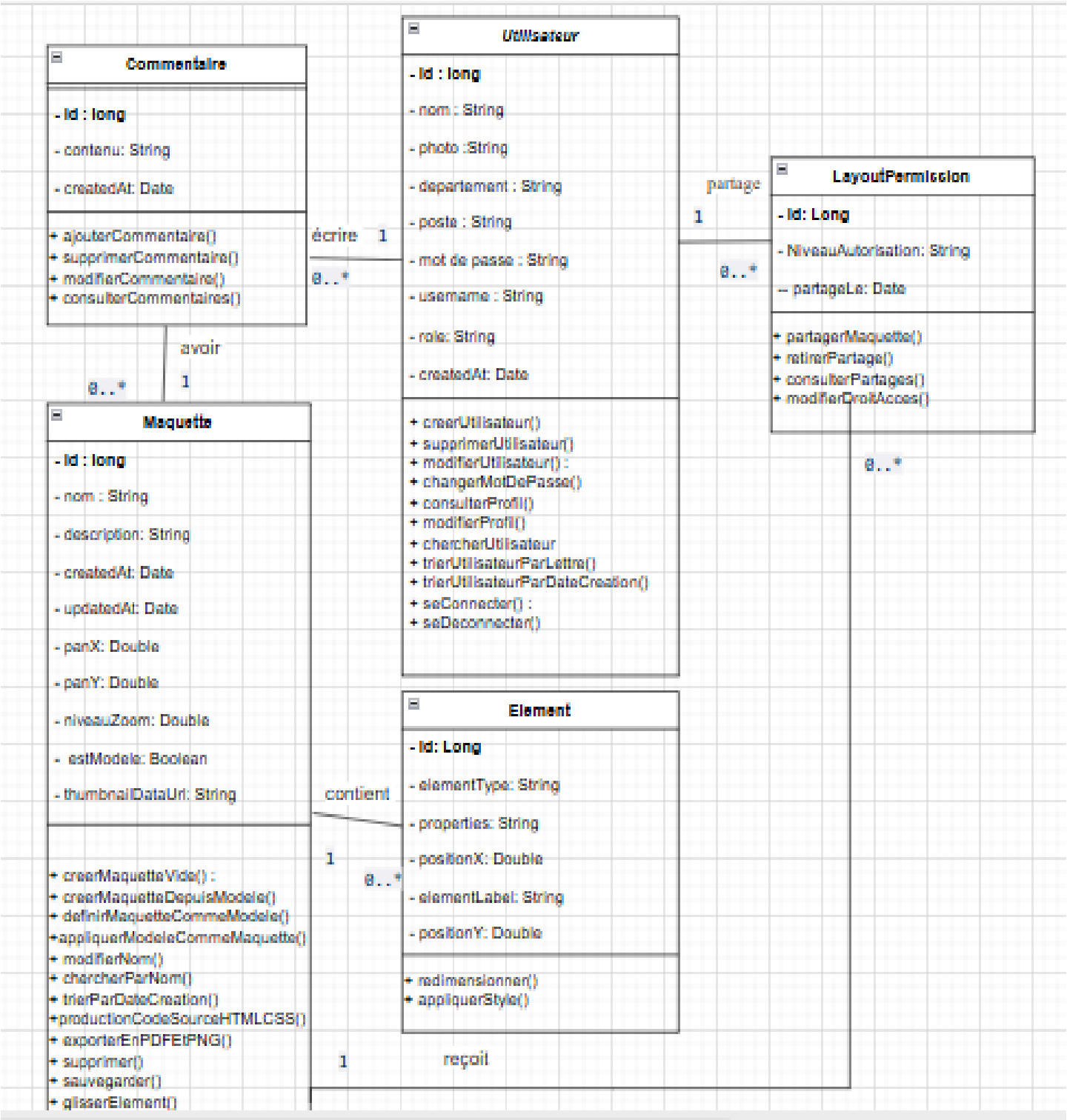


FIGURE 3.33 – Structure des classes – Sprint 4.

3.5.3 Raffinement des diagrammes de cas d'utilisation

Pour mieux comprendre les interactions, voici les descriptions détaillées de quelques cas d'utilisation représentatifs de ce Sprint, suivies de leurs diagrammes de séquence illustrant les flux d'informations et les échanges entre les différents composants du système.

3.5.3.1 Analyse de cas "Partager un projet"

3.5.3.1.1 Description textuelle de Cas d'Utilisation : Partager un projet

Ce cas d'utilisation détaille comment le propriétaire d'un projet peut inviter un collaborateur et lui attribuer des permissions spécifiques.

Table 3.12 – Description du scénario « Partager un projet »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Partager un projet avec un collaborateur
Acteur	Propriétaire du projet
Pré-condition	L'acteur est authentifié et est propriétaire du projet.
Post-condition	Le collaborateur invité a désormais accès au projet avec les droits définis, ou un message d'erreur est affiché en cas d'échec.
Scénario nominal	1. L'acteur clique sur "Partager" depuis l'éditeur. 2. La 'ShareProjectModal' affiche la liste des collaborateurs actuels et un champ de recherche pour ajouter un nouvel utilisateur. 3. L'acteur sélectionne un utilisateur et définit le niveau de permission (ex : "peut voir" ou "peut modifier"). 4. L'acteur clique sur "Inviter". 5. La 'ShareProjectModal' envoie la requête 'shareProject()' au 'ProjectController'. 6. Le 'ProjectController' interagit avec 'ProjetPermission' pour persister la collaboration et les permissions. 7. 'ProjetPermission' retourne le résultat (succès ou erreur) au 'ProjectController'. 8. La 'ShareProjectModal' met à jour la liste des collaborateurs et affiche un message de succès ou d'erreur à l'acteur.
Scénario alternatif	— **Utilisateur introuvable :** Si l'utilisateur recherché n'existe pas, la 'ShareProjectModal' affiche "Utilisateur introuvable". — **Erreur de persistance :** Si 'ProjetPermission' rencontre un problème lors de l'enregistrement, un message "Erreur lors du partage. Réessayez." est affiché.

3.5.3.1.2 Diagramme de Séquence : Partager un projet

Ce diagramme illustre le déroulement du processus de partage d'un projet et de l'attribution des permissions à un collaborateur.

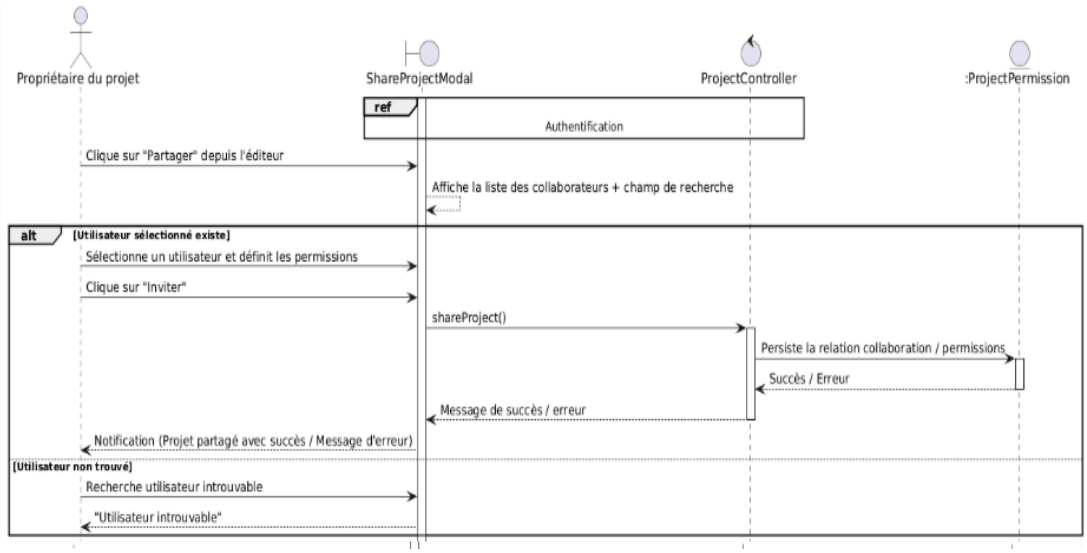


FIGURE 3.34 – Diagramme de séquence du partage d'un projet.

3.5.3.2 Analyse de cas "Ajouter un commentaire"

3.5.3.2.1 Description textuelle de Cas d'Utilisation : Ajouter un commentaire

Ce cas d'utilisation détaille le processus par lequel un utilisateur partage son avis ou pose une question directement sur une maquette, afin de faciliter la communication et la collaboration au sein de l'équipe projet.

TABLE 3.13 – Description du scénario « Ajouter un commentaire »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Ajouter un commentaire sur la maquette
Acteur	Utilisateur ayant accès au projet
Pré-condition	L'acteur est authentifié et se trouve sur l'interface de l'éditeur.
Post-condition	Le commentaire est enregistré.
Scénario nominal	1. L'acteur clique sur l'icône "Commentaires" dans l'éditeur. 2. La 'VueCommentPanel' affiche les commentaires existants. 3. L'acteur saisit un message dans la zone de texte. 4. L'acteur clique sur "Poster". 5. La 'VueCommentPanel' envoie la requête 'addComment()' au 'ProjetController'. 6. Le 'ProjetController' persiste le commentaire dans 'CommentEntity'. 7. Le 'ProjetController' retourne le commentaire enregistré à la 'VueCommentPanel'. 8. La 'VueCommentPanel' met à jour la liste des commentaires et vide le champ de saisie.

Élément du cas d'utilisation	Détails
Scénario d'échec	— **Champ vide :** Si le champ de texte est vide, 'VueCommentPanel' affiche "Le commentaire ne peut pas être vide." et empêche l'envoi. — **Erreur de persistance :** Si l'enregistrement échoue, 'ProjetController' renvoie une erreur et 'VueCommentPanel' affiche "Impossible de poster le commentaire. Réessayez."

3.5.3.2.2 Diagramme de Séquence : Ajouter un commentaire

Ce diagramme montre les interactions entre l'utilisateur et le système lors de l'ajout d'un commentaire sur la maquette.

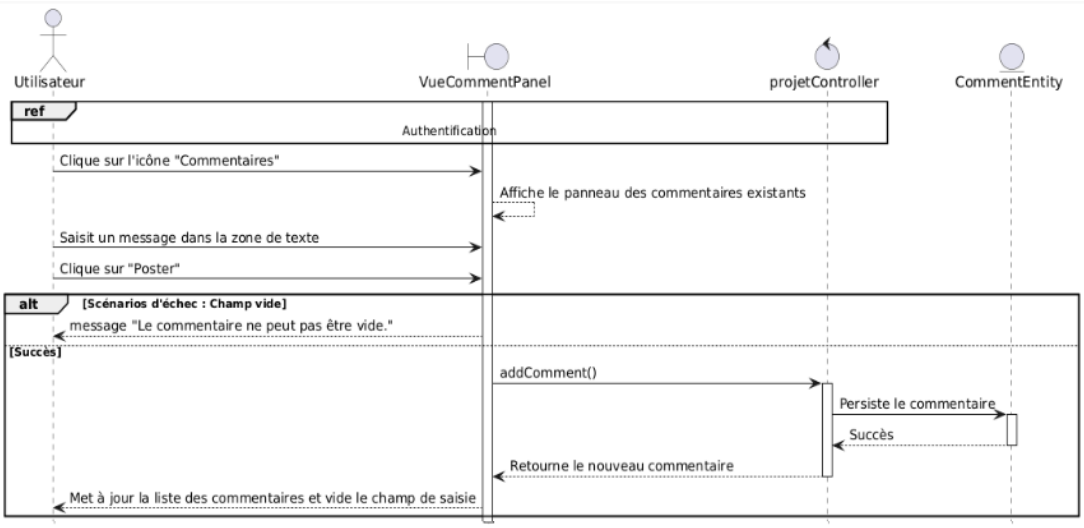


FIGURE 3.35 – Diagramme de séquence d'ajouter d'un commentaire.

3.5.4 Réalisation du Sprint 4

Les interfaces utilisateur réalisées durant ce Sprint concrétisent les fonctionnalités de collaboration, offrant des outils intuitifs pour le partage et la discussion.

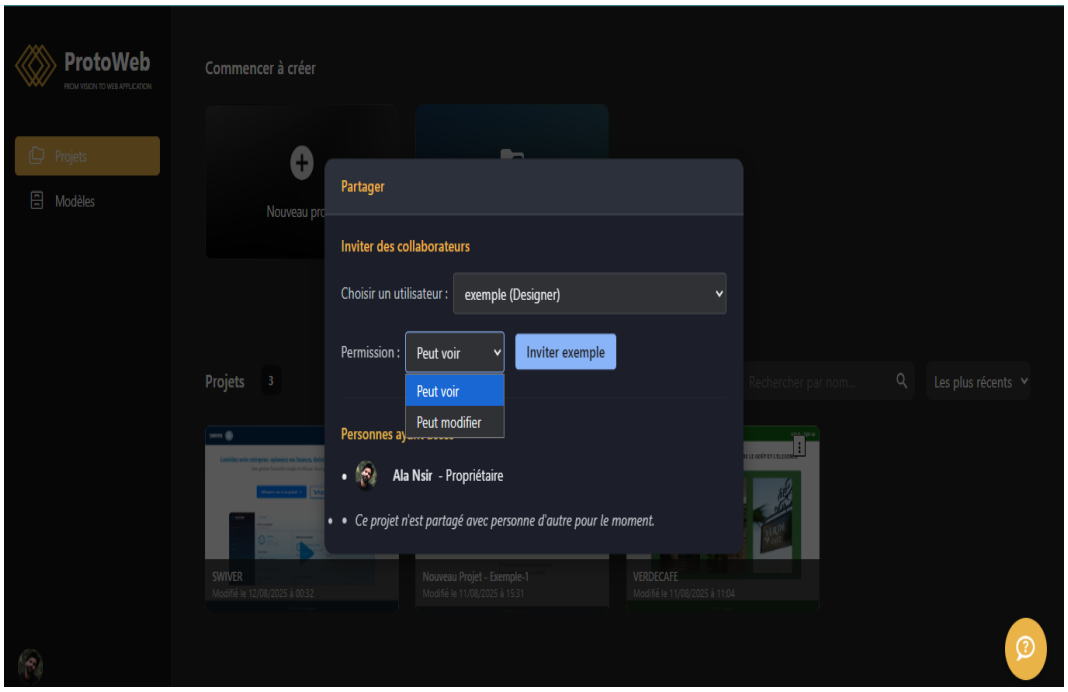


FIGURE 3.36 – Interface de partage d'un projet.

Cette figure représente la modale de partage, où le propriétaire du projet peut inviter des collaborateurs en recherchant des utilisateurs et en leur attribuant des permissions de "visualisation" ou de "modification". Elle permet également de voir et de gérer la liste des collaborateurs existants.

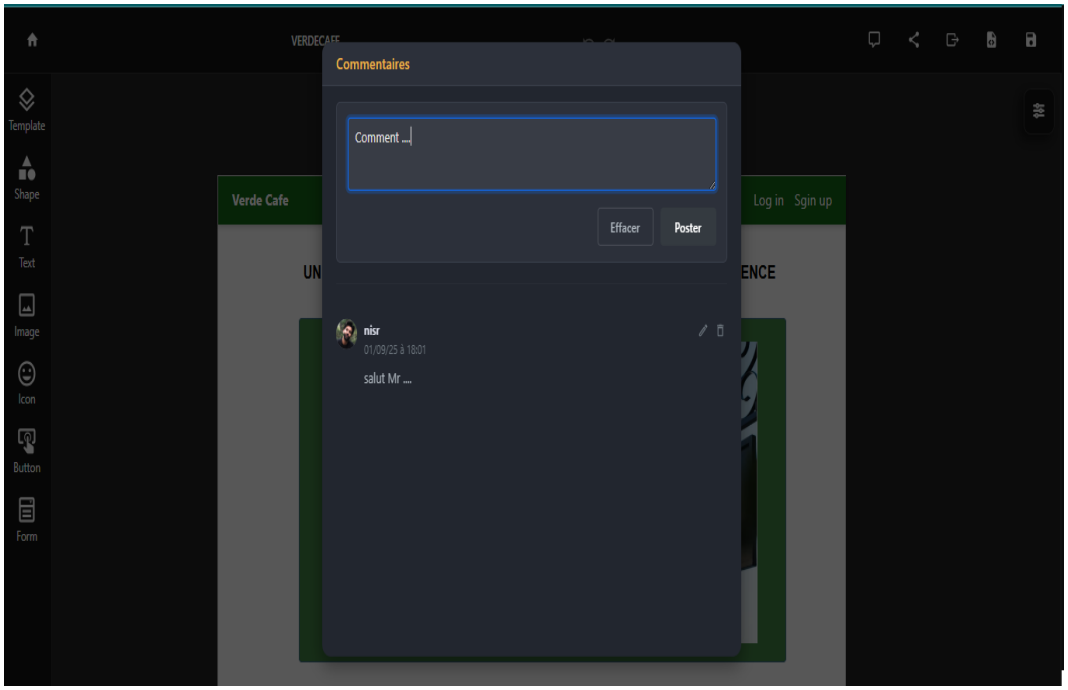


FIGURE 3.37 – Panneau de commentaires intégré à l'éditeur.

Cette interface illustre le panneau latéral des commentaires, directement intégré à l'éditeur visuel. Il permet aux utilisateurs de saisir leurs messages, de les poster et de visualiser l'historique des discussions en temps réel, facilitant ainsi les retours contextuels sur la maquette.

3.5.5 Phase de Test

La phase de test de ce quatrième Sprint a été rigoureuse, visant à valider l'intégralité des fonctionnalités de collaboration et de communication en temps réel.
Une attention particulière a été portée sur :

— **Le partage de projets et la gestion des permissions :** Des scénarios de test ont couvert l'invitation de nouveaux collaborateurs (avec différentes permissions), la modification de leurs droits, la révocation d'accès, et la gestion des cas d'erreur (utilisateur déjà invité, utilisateur introuvable). La vérification de l'accès correct (lecture seule ou modification) pour les utilisateurs invités a été primordiale.

— **Le module de commentaires en temps réel :** Les tests ont inclus l'ajout de commentaires par différents utilisateurs, la vérification de l'affichage en temps réel pour tous les collaborateurs connectés au projet, la gestion des commentaires vides, et la persistance des commentaires après rafraîchissement ou reconnexion. La fluidité de l'interface et la pertinence des notifications ont également été évaluées.

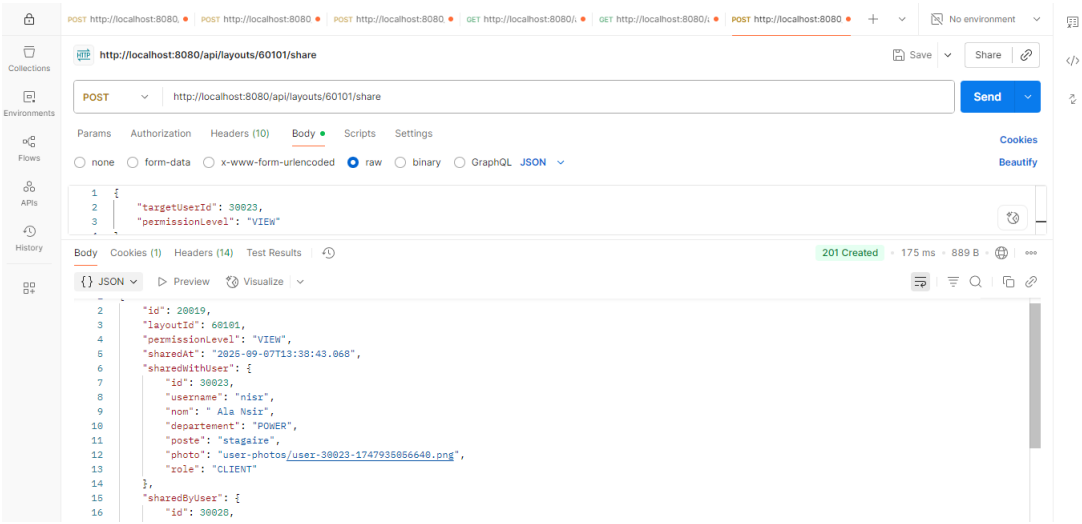


FIGURE 3.38 – Test Postman – Partage de projet.

Cette figure illustre une requête Postman validant l'invitation d'un collaborateur avec des permissions spécifiques. Elle confirme le bon fonctionnement de l'API de partage en renvoyant un message de succès.

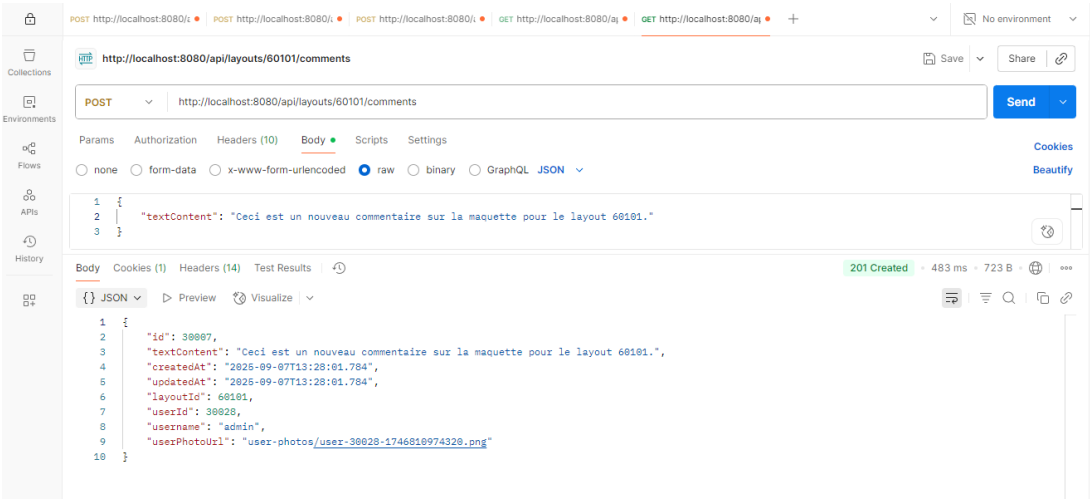


FIGURE 3.39 – Test Postman – Ajout de commentaire.

Cette figure présente un exemple de requête Postman permettant à un utilisateur d'ajouter un commentaire. Elle confirme la bonne persistance du commentaire dans la base de données et son affichage dans le projet.

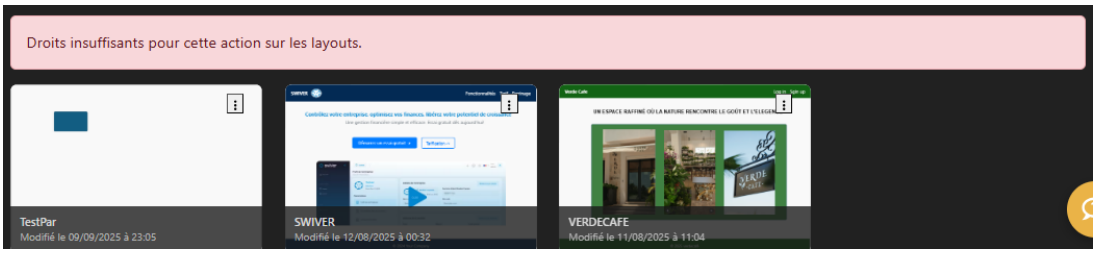


FIGURE 3.40 – Interface affichant les droits insuffisants pour la suppression d'un projet par un collaborateur.

Cette figure illustre un cas de test où un collaborateur tente de supprimer un projet. Le message d'erreur "Droits insuffisants pour cette action sur les layouts" confirme que le système de gestion des permissions fonctionne correctement, n'autorisant que le propriétaire du projet à réaliser cette action sensible.

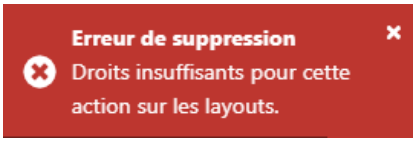


FIGURE 3.41 – Message d'erreur lors d'une tentative de suppression sans les droits requis.

Cette figure démontre le mécanisme de protection mis en place, où un utilisateur ne disposant pas des droits de propriété tente de supprimer un projet, résultant en un message explicite "Droits insuffisants pour cette action sur les layouts".

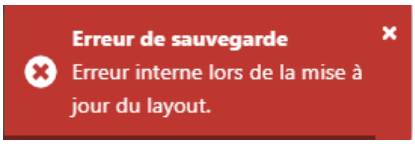


FIGURE 3.42 – Notification d'erreur - Action non autorisée

Cette figure illustre le mécanisme de protection : un utilisateur disposant uniquement des droits de lecture tente de modifier la maquette, mais en est empêché faute de droits de propriété.

3.6 Sprint 5 : Exportation et Obtention du Code Source

3.6.1 Présentation du Sprint 5

Le cinquième Sprint a introduit deux fonctionnalités clés pour Protoweb : l'exportation des maquettes dans différents formats (PDF, PNG, JPG) afin de faciliter la communication et la documentation des projets, et l'obtention automatique de fichiers HTML et CSS pour accélérer la transition entre la conception et l'implémentation. Ces fonctionnalités ont été intégrées de manière simple et intuitive dans l'interface utilisateur, puis testées et validées sur une période de quatre semaines pour assurer leur robustesse et leur fiabilité. L'ensemble de ces travaux a permis d'améliorer la fluidité du processus entre design et développement, tout en offrant un gain de temps significatif pour les utilisateurs.

3.6.2 Spécification fonctionnelle

Cette section détaille les fonctionnalités prévues pour le Sprint 5, en précisant le backlog, les cas d'utilisation, ainsi que les différents diagrammes illustrant leur fonctionnement.

3.6.2.1 Sprint Backlog

Le tableau ci-dessous présente les User Stories définies pour le Sprint 5, en précisant leur description, leur complexité ainsi que leur estimation en jours.

TABLE 3.14 – Backlog du Sprint 5 : Détail des User Stories

User Story	Description	Complexité (Points)	Estimation (jours)
US 11.1	Exporter la maquette en PDF pour faciliter la documentation et le partage.	5	3
US 11.2	Exporter la maquette en PNG et JPG pour des usages graphiques variés.	5	3
US 11.3	Obtenir le code source (HTML/CSS) de la maquette afin de faciliter le passage au développement.	13	14

3.6.2.2 Diagramme de cas d'utilisation global du Sprint 5

Le diagramme suivant illustre de manière globale les interactions entre les utilisateurs et le système pour les fonctionnalités d'exportation des maquettes et d'obtention du code source.

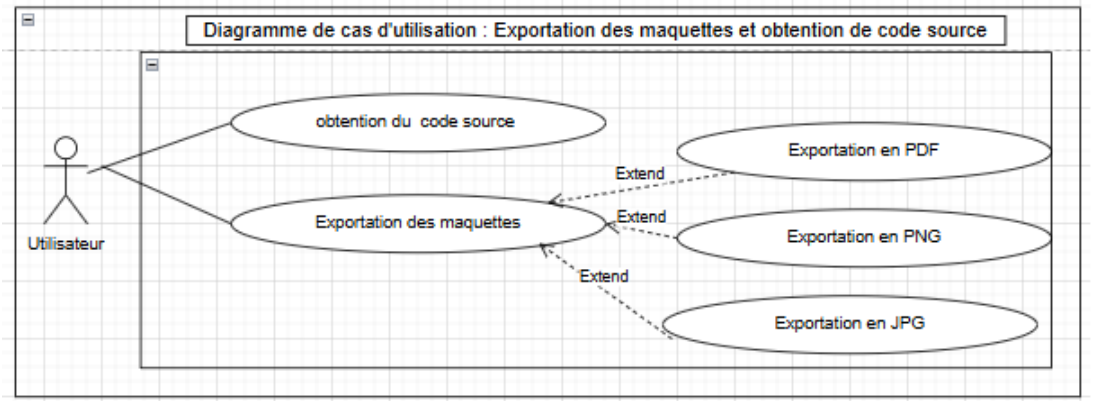


FIGURE 3.43 – Diagramme de cas d'utilisation : Exportation des maquettes et obtention de code source.

3.6.3 Raffinement des diagrammes de cas d'utilisation

Cette section présente une analyse détaillée des principaux cas d'utilisation du Sprint 5, en décrivant leur déroulement textuel ainsi que les diagrammes de séquence associés.

3.6.3.1 Analyse de cas "Exporter une maquette"

Ce cas d'utilisation décrit le processus permettant à un utilisateur d'exporter une maquette dans différents formats.

3.6.3.1.1 Description textuelle de Cas d'Utilisation : Exporter une maquette

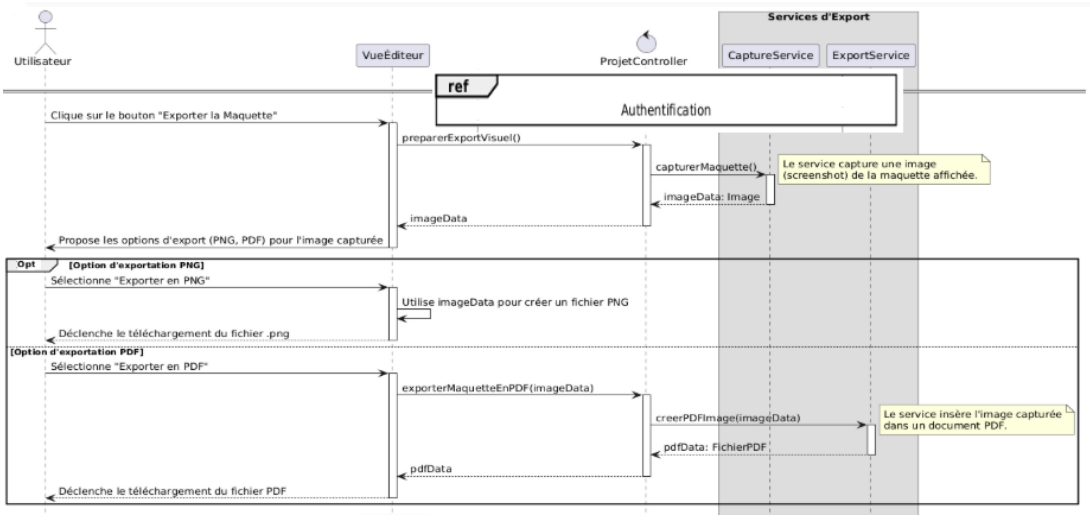
TABLE 3.15 – Description du scénario « Exporter une maquette »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Exporter une maquette
Acteur	Utilisateur, Designer, Admin
Pré-condition	L'utilisateur doit avoir un projet ouvert dans l'éditeur et une maquette valide à exporter.
Post-condition	Le fichier exporté (PNG ou PDF) est généré et téléchargeable par l'utilisateur.

Élément du cas d'utilisation	Détails
Scénario nominal	1. L'utilisateur clique sur le bouton "Exporter la Maquette" dans l'éditeur. 2. L'éditeur demande au contrôleur de préparer l'export visuel de la maquette. 3. Le contrôleur fait appel au service de capture pour générer une image de la maquette. 4. L'image capturée est renvoyée à l'éditeur. 5. L'éditeur propose à l'utilisateur les formats d'export disponibles (PNG ou PDF). 6. Selon le choix de l'utilisateur : — **PNG** : l'éditeur utilise l'image pour créer un fichier PNG et déclenche son téléchargement. — **PDF** : l'éditeur transmet l'image au contrôleur, qui utilise le service d'export pour générer un PDF, renvoyé à l'éditeur pour déclencher le téléchargement.
Scénarios alternatifs / erreurs	— La maquette est invalide ou vide : le système affiche un message d'erreur et l'export est annulé.

3.6.3.1.2 Diagramme de Séquence : Exportation d'une maquette

Le diagramme de séquence ci-dessous illustre les interactions entre l'utilisateur et le système lors de l'exportation d'une maquette, depuis la sélection du format.



sqExportation.pdf

FIGURE 3.44 – Diagramme de séquence – Exportation d'une maquette.

3.6.3.2 Analyse de cas "obtenir le code source"

Ce cas d'utilisation décrit le processus par lequel l'utilisateur génère automatiquement le code HTML et CSS correspondant à sa maquette. Il illustre comment le système analyse les éléments de la

maquette et produit un code exploitable pour le développement.

3.6.3.2.1 Description textuelle de Cas d'Utilisation : Obtenir le code source

Table 3.16 – Description du scénario « Obtenir le code source »

Élément du cas d'utilisation	**Détails**
Cas d'utilisation	Obtenir le code source de la maquette
Acteur	Utilisateur, Designer, Admin
Pré-condition	L'utilisateur est authentifié et une maquette valide est ouverte dans l'éditeur.
Post-condition	Le code HTML et CSS de la maquette est généré et affiché.
Scénario nominal	1. L'utilisateur clique sur le bouton « Générer le Code » dans l'éditeur. 2. L'éditeur envoie une requête de génération au `ProjetController`. 3. Le `ProjetController` récupère la maquette active via le `ProjectModel`. 4. Les éléments de la maquette (positions, propriétés, etc.) sont transmis au `CodeGenerator`. 5. Le `CodeGenerator` transforme la maquette en code HTML et CSS. 6. Le `ProjetController` renvoie le code généré à l'éditeur. 7. L'éditeur affiche le code HTML et CSS dans des zones dédiées pour consultation par l'utilisateur.
Scénarios alternatifs / erreurs	— Si aucune maquette active n'est trouvée, le système renvoie une erreur : « Aucune maquette active à générer » et l'éditeur affiche un message d'erreur.

3.6.3.2.2 Diagramme de Séquence : obtention du code source

Ce diagramme de séquence illustre les interactions entre l'utilisateur et le système lors de la génération automatique du code HTML et CSS à partir de la maquette active.

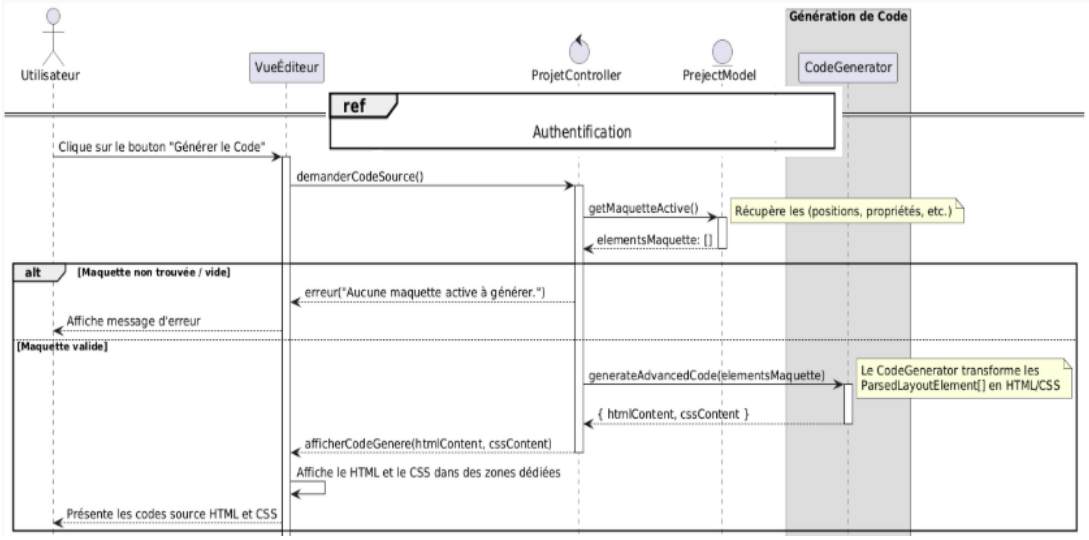


FIGURE 3.45 – Diagramme de séquence – Obtention de code source.

3.6.3.3 Réalisation du Sprint 5

Au cours de ce Sprint, l'interface utilisateur a été enrichie avec :

— Un menu d'exportation permettant de choisir entre PDF, PNG et JPG.

— Une option dédiée à la génération de code HTML/CSS.

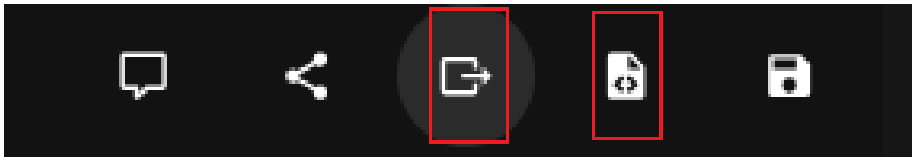


FIGURE 3.46 – Gestion des projets – intégration des options d'exportation et génération de code source.

Cette interface de gestion de projet a été enrichie avec des icônes dédiées à l'exportation et à la génération de code source, rendant ces fonctionnalités directement accessibles depuis la vue principale de gestion de projet.

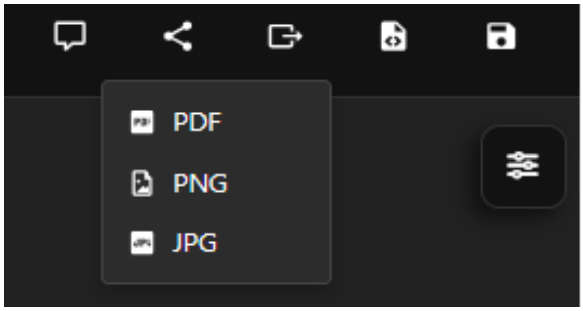


FIGURE 3.47 – Menu déroulant d'exportation – choix du format PDF, PNG ou JPG.

Ce menu contextuel illustre la flexibilité offerte pour l'exportation, répondant à divers besoins de partage et d'intégration.

3.6.3.4 Phase de test

La phase de test du Sprint 5 a porté sur les deux nouvelles fonctionnalités implémentées :

- **Exportation des maquettes** : des tests ont été réalisés directement depuis l'interface afin de vérifier la génération correcte des fichiers aux formats PDF, PNG et JPG. Chaque fichier exporté a été contrôlé pour s'assurer de sa lisibilité et de sa conformité avec la maquette originale.
- **Obtention du code source** : des tests unitaires et fonctionnels ont validé la génération automatique du code HTML et CSS. Les résultats ont montré que le code produit respectait la structure et les propriétés des éléments définis dans la maquette.

Ces tests ont confirmé la robustesse et la fiabilité des fonctionnalités, tout en garantissant une expérience utilisateur fluide et intuitive.

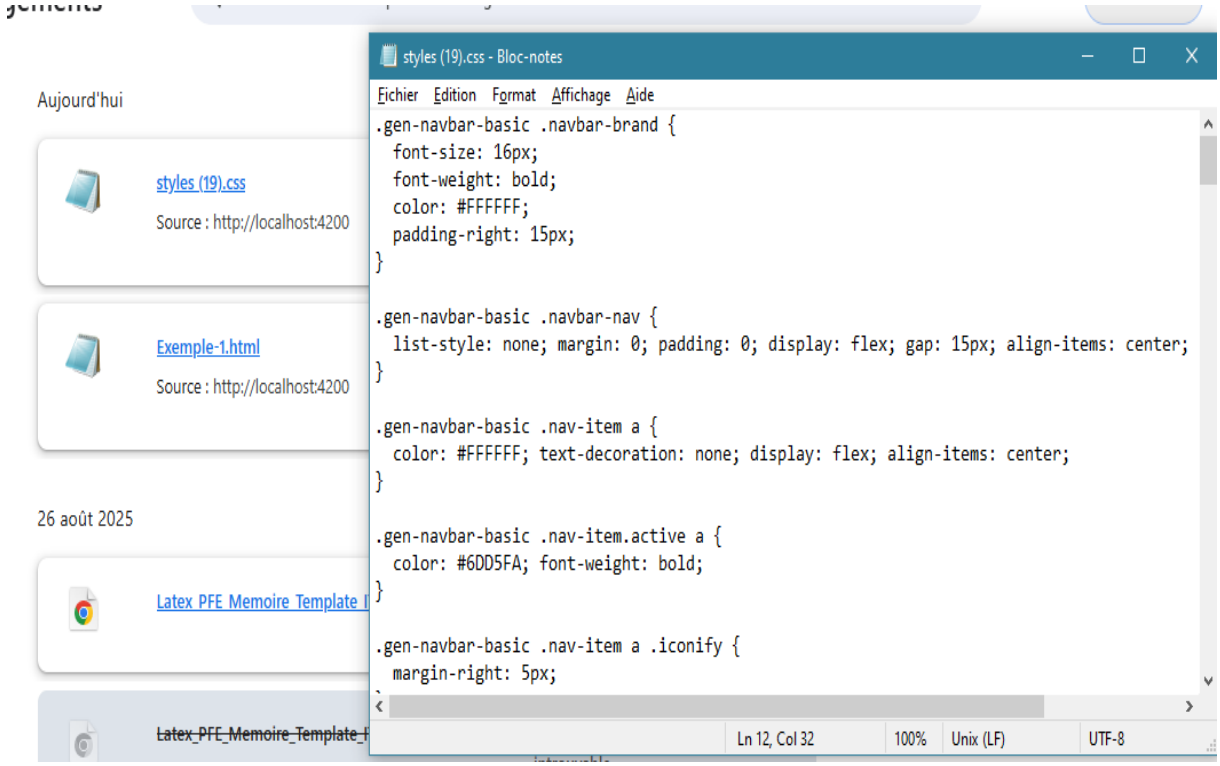


FIGURE 3.48 – Exemple de fichiers générés (HTML et CSS).

Cette figure présente un aperçu des fichiers CSS et HTML générés, illustrant la structure du code produit pour une maquette spécifique.

3.7 Sprint 6 : Conception et développement du tableau de bord statistique

3.7.1 Présentation du Sprint 6

Le sixième Sprint a permis de fournir à l'administrateur un tableau de bord centralisé pour visualiser les indicateurs clés de performance et d'activité de Protoweb. Ce tableau de bord inclut des statistiques sur le nombre d'utilisateurs, de projets et de modèles, des graphiques illustrant la répartition

des utilisateurs et des projets par département, ainsi que des listes affichant les dernières inscriptions et les utilisateurs les plus actifs. L'ensemble a été intégré dans une interface intuitive offrant une navigation facile et une consultation rapide des données, améliorant ainsi la supervision et la prise de décision sur la plateforme.

TABLE 3.17 – Backlog du Sprint 6

User Story	Description	Complexité (Points)	Estimation (jours)
$US_{6.1}$	En tant qu'Admin, je veux voir sur une seule page des cartes affichant les statistiques clés : nombre total d'utilisateurs, de projets et de modèles.	5	3
$US_{6.2}$	En tant qu'Admin, je veux un graphique montrant la répartition des utilisateurs et des projets par département.	8	4
$US_{6.3}$	En tant qu'Admin, je veux une liste des derniers utilisateurs inscrits et des utilisateurs les plus récemment actifs.	5	4

3.7.2 Diagramme de cas d'utilisation global du Sprint 6

Le diagramme ci-dessous illustre les interactions entre l'administrateur et le système pour la consultation des statistiques et l'accès aux différentes visualisations du tableau de bord.

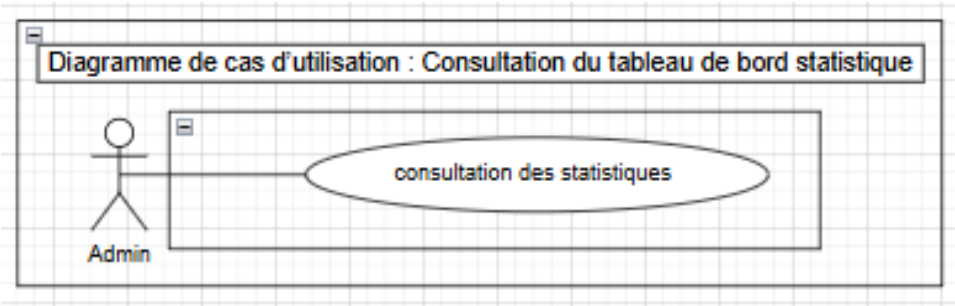


FIGURE 3.49 – Diagramme de cas d'utilisation : Consultation du tableau de bord statistique.

3.7.3 Raffinement des diagrammes de cas d'utilisation

Cette section détaille les scénarios d'utilisation principaux relatifs à la consultation du tableau de bord.

3.7.3.1 Analyse de cas "Consulter les statistiques"

3.7.3.1.1 Description textuelle de Cas d'Utilisation : Consulter les statistiques

Table 3.18 – Description du scénario « Consulter les statistiques »

Élément du cas d'utilisation	Détails
Cas d'utilisation	Consulter les statistiques
Acteur	Admin
Pré-condition	L'Admin est authentifié et accède à la page du tableau de bord.
Post-condition	Les statistiques sont affichées sous forme de graphiques et indicateurs dans l'interface du tableau de bord.
Scénario nominal	1. L'utilisateur clique sur l'onglet « Tableau de bord » depuis l'interface. 2. La `VueDashboard` envoie une requête au `AdminController`. 3. Le `AdminController` interroge le `Model` pour récupérer les données statistiques (utilisation, projets créés, performances, etc.). 4. Le `Model` renvoie les résultats calculés au `AdminController`. 5. Le `AdminController` transmet les résultats formatés à la `VueDashboard`. 6. La `VueDashboard` affiche les statistiques (graphiques, tableaux et indicateurs) à l'utilisateur.
Scénario alternatif	Si certaines données sont indisponibles ou corrompues, le `AdminController` renvoie un message d'erreur partiel, et la `VueDashboard` affiche uniquement les modules disponibles sans bloquer l'affichage général.

3.7.4 Diagramme de séquence de la consultation des statistiques

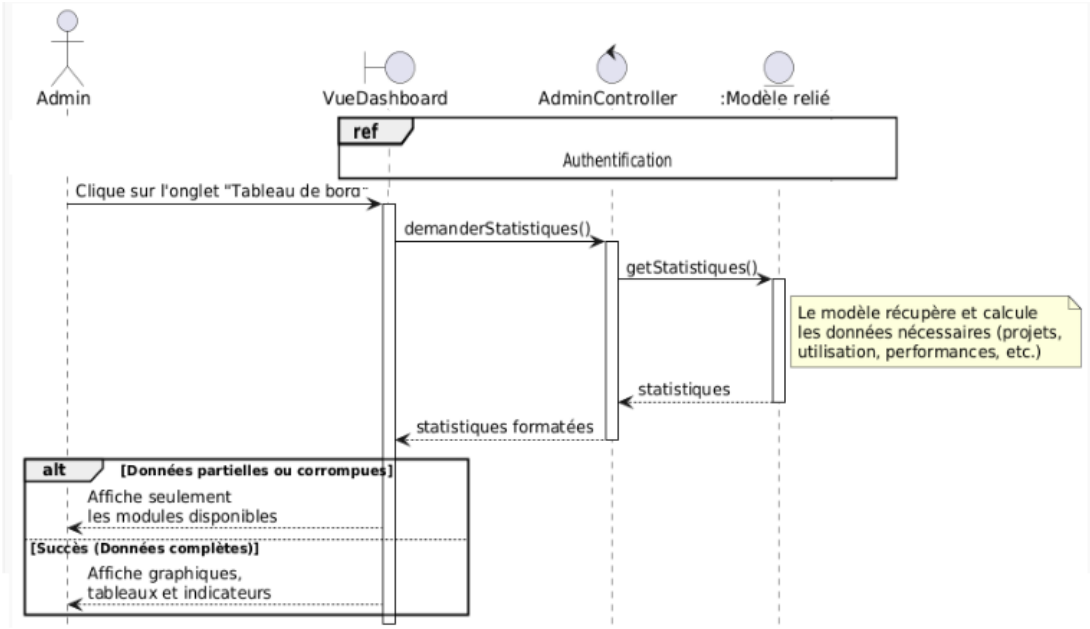


Figure 3.50 – Diagramme de séquence de la consultation des statistiques.

3.7.5 Réalisation des Interfaces Graphiques

Les interfaces graphiques ont été conçues pour offrir une expérience intuitive et interactive à l'administrateur.

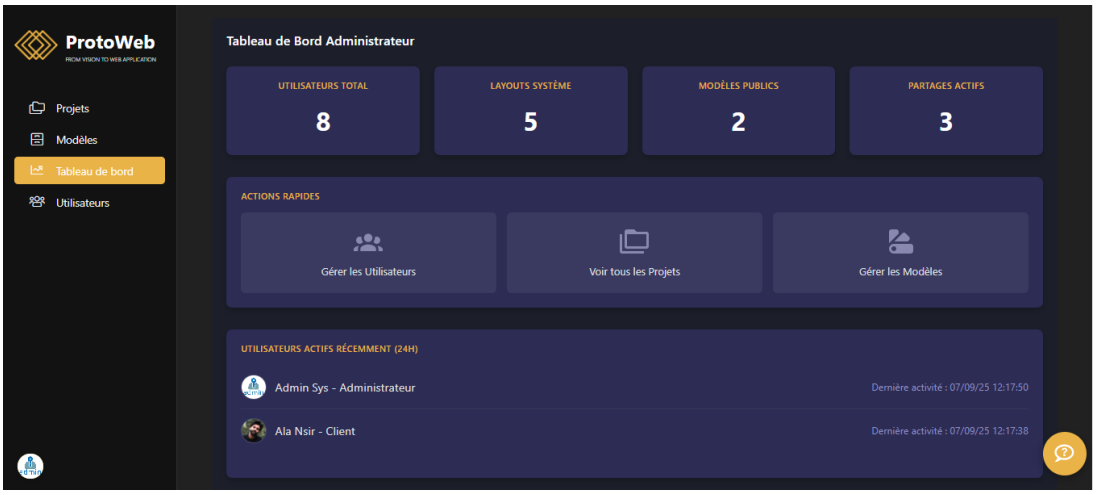


FIGURE 3.51 – Vue d'ensemble du tableau de bord administrateur avec statistiques clés et actions rapides.

Cette image présente l'aperçu général du tableau de bord administrateur, affichant des cartes synthétiques pour les statistiques clés comme le nombre total d'utilisateurs, de layouts, de modèles et de partages actifs. Elle met également en évidence une section pour les actions rapides et les utilisateurs récemment actifs.

FIGURE 3.52 – Détail de l'interface du tableau de bord administrateur - Liste des derniers utilisateurs inscrits.

Cette vue détaillée du tableau de bord met en lumière la liste des derniers utilisateurs inscrits, offrant une visibilité immédiate sur les nouvelles adhésions. Elle permet à l'administrateur de suivre l'évolution de la base d'utilisateurs

FIGURE 3.53 – Graphiques de répartition des utilisateurs et des projets par département.

Cette dernière image illustre des graphiques à barres comparant la répartition des utilisateurs et des projets par département, fournissant des insights visuels sur l'engagement et l'activité au sein des différentes équipes. Cette section aide l'administrateur à identifier les départements les plus actifs ou ceux qui nécessitent une attention particulière.

3.7.6 Phase de test - Sprint 6

La phase de test de ce Sprint s'est focalisée sur l'assurance qualité du tableau de bord statistique, garantissant l'exactitude des données affichées, la fiabilité des visualisations graphiques, et la robustesse de l'interface en cas d'anomalies. L'objectif était de s'assurer que l'administrateur dispose d'un outil de supervision fiable et précis. Des tests unitaires et d'intégration ont été appliqués à chaque composant du tableau de bord. Concernant les cartes de statistiques (nombre d'utilisateurs, projets, modèles), des scénarios ont été exécutés pour vérifier que les chiffres étaient correctement agrégés et mis à jour en temps réel. Pour les graphiques de répartition par département, l'exactitude des données et la bonne correspondance entre les chiffres et les barres ont été minutieusement contrôlées. Les listes des derniers inscrits et des utilisateurs actifs ont été testées pour leur pertinence et leur tri chronologique. Postman a été utilisé pour simuler les requêtes vers l'API backend afin de vérifier la bonne récupération et le traitement des données brutes nécessaires à l'élaboration du tableau de bord. Cela a permis de s'assurer que les contrôleurs et les modèles géraient correctement les requêtes de données, y compris les cas où certaines informations seraient absentes ou mal formatées, garantissant ainsi que l'affichage du tableau de bord n'était pas bloqué. Voici quelques exemples de tests réalisés :

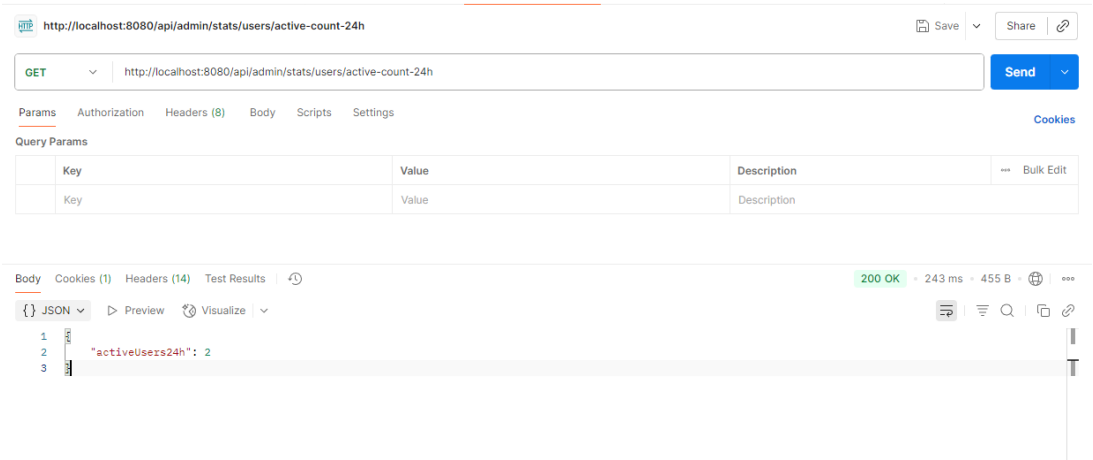


FIGURE 3.54 – Test Postman – Récupération de nombre d'Utilisateurs Actifs sur 24 Heures.

Cette capture d'écran de Postman illustre la récupération du nombre total d'utilisateurs actifs au cours des dernières 24 heures via une requête API, indiquant un total de 2 utilisateurs.

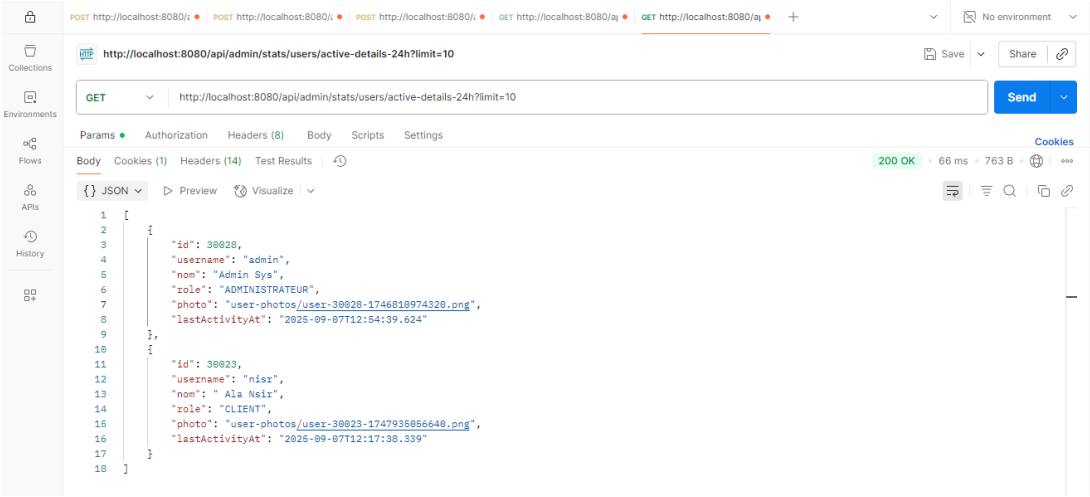


FIGURE 3.55 – Test Postman –Monitoring des Utilisateurs Actifs.

L'image présente une interface d'API affichant les détails des utilisateurs actifs, y compris leur identifiant, nom d'utilisateur, rôle, photo de profil et la date de leur dernière activité.

3.8 Conclusion

Cette première release a solidement établi les fondations fonctionnelles de la plateforme, en intégrant l'authentification, la gestion complète des utilisateurs et des profils, ainsi que les modules de création et de gestion des projets. L'éditeur visuel intuitif, enrichi par le glisser-déposer et la personnalisation en temps réel, offre une expérience de conception robuste. L'implémentation réussie des fonctionnalités de collaboration, d'exportation de maquettes et de génération de code source positionne Protoweb comme un outil polyvalent, complété par un tableau de bord administratif qui assure une supervision efficace de l'activité. Nous passons dans le prochain chapitre à la Release 2 pour étendre encore ces capacités.

Chapitre 4

Release 2 : Intégration du Chatbot IA et Finalisation de la Plateforme

4.1 Introduction

Cette deuxième et dernière release marque l'aboutissement du projet Protoweb. Après avoir établi les fondations fonctionnelles et les outils de collaboration lors de la Release 1, cette phase est dédiée à l'intégration d'une intelligence artificielle pour assister les utilisateurs, ainsi qu'à la finalisation de l'application par des tests exhaustifs et la préparation au déploiement. L'objectif est de livrer un produit complet, performant et autonome. Conformément à la planification définie, la Release 2 s'est déroulée sur une période d'un mois, structurée en deux Sprints. Le septième Sprint a été consacré à la sélection, à l'intégration d'un modèle de langage adapté pour le chatbot d'assistance. Le huitième et dernier Sprint a visé la phase cruciale de tests finaux et la préparation minutieuse de l'application pour son déploiement sur les serveurs de Safran.
Chaque Sprint de cette release a apporté une avancée majeure dans la maturité du produit, en garantissant à la fois une assistance intelligente pour les utilisateurs et une qualité irréprochable avant la mise en production.

4.2 Sprint 1 :assistance protypage via chatbot

4.2.1 Présentation du Sprint 1

Le Sprint 1 représente une étape clé pour Protoweb, marqué par l'intégration d'une dimension d'intelligence artificielle via un chatbot d'assistance. L'objectif principal était de doter la plateforme d'un assistant conversationnel capable de répondre en temps réel aux questions des utilisateurs concernant l'utilisation de l'outil.
Pour atteindre cet objectif, le travail a porté sur l'étude et l'analyse des différentes approches technologiques disponibles, le choix du modèle d'IA et de l'architecture la plus adaptée, ainsi que l'intégration effective du chatbot au sein de l'interface utilisateur.
Cette démarche a permis de concevoir un module d'assistance intelligent, capable de fournir des réponses contextuelles tout en garantissant la sécurité des échanges et la protection des données, conformément aux exigences de confidentialité strictes de Safran.

4.2.2 Choix technologiques du Chatbot

Pour répondre au besoin d'une assistance intelligente et intégrée, le choix de la technologie du chatbot a fait l'objet d'une analyse rigoureuse. L'objectif n'était pas d'intégrer un simple gadget, mais de construire un outil fiable, sécurisé et réellement expert de l'application Protoweb.

4.2.2.1 Comparaison des modèles d'IA dans les applications de prototypage et de support

Le tableau ci-dessous présente les principales applications de prototypage et de design, les modèles d'IA qu'elles utilisent, leur usage principal, et leur mode de déploiement (Cloud/API ou auto-hébergé) :

Table 4.1 – Applications de prototypage et modèles d'IA associés

Application	**Modèle d'IA utilisé**	**Usage principal**	**Mode de déploiement**
Figma [8]	GPT [29]/ Modèles propriétaires	Suggestions de design, génération de contenu textuel et layout interactif	Cloud / API
Canva[23]	Stable Diffusion / GPT[29]	Création de visuels, génération de textes, recommandations graphiques	Cloud / API
Adobe XD [2]	GPT [29]	Textes de placeholder, dialogues	Cloud / API
ChatGPT [30]	GPT-4 / GPT-4-turbo[29]	Assistance conversationnelle, réponse aux questions utilisateurs, support client	Cloud / API
Gemini (Google) [24]	Gemini 1.5[25]	Assistance conversationnelle, génération de contenu multimodal	Cloud / API

4.2.2.2 Les approches : Cloud vs. Auto-hébergement

Le marché des grands modèles de langage (LLM) se divise principalement en deux philosophies :

1. **Les solutions Cloud (via API) :** Portées par des acteurs comme OpenAI (GPT) ou Google (Gemini), ces solutions offrent une très grande puissance. Cependant, elles impliquent d'envoyer chaque question de l'utilisateur vers les serveurs du fournisseur.
2. **Les solutions auto-hébergées (Self-Hosted) :** Portées par des modèles *open source* comme Llama (Meta) ou Phi-3 (Microsoft), cette approche consiste à faire fonctionner le modèle d'IA directement sur les serveurs de l'entreprise.

Pour un environnement aussi sensible que celui de Safran, le critère de la **souveraineté des données** est non négociable. Nous avons donc comparé ces deux approches.

4.2.2.3 Tableau comparatif des approches

Table 4.2 – Comparaison des approches Cloud et Auto-hébergée pour le chatbot

Critère	**Solutions Cloud (ex : API OpenAI)**	**Solutions Auto-hébergées (Notre choix)**
Sécurité des données	Les données (questions, contexte) sont envoyées et traitées par un tiers. Risque de fuite et de non-conformité.	**Aucune donnée ne quitte le réseau de Safran.** La confidentialité est totale et maîtrisée de bout en bout.
Maîtrise de l'infrastructure	Dépendance totale au fournisseur (disponibilité, mises à jour, coûts).	**Contrôle total sur l'infrastructure.** Le service peut être déployé sur un serveur interne, garantissant sa disponibilité.
Personnalisation	Limitée. On utilise le modèle tel quel, avec peu de contrôle sur son comportement de fond.	**Personnalisation avancée.** Possibilité d'entraîner (*fine-tuner*) le modèle spécifiquement sur la documentation Protoweb pour en faire un véritable expert.

4.2.2.4 Modèles et déploiement des solutions auto-hébergées

Les solutions auto-hébergées permettent de déployer des modèles d'IA directement sur les serveurs de l'entreprise, assurant une maîtrise totale des données et une personnalisation avancée. Plusieurs modèles sont particulièrement adaptés à ce type de déploiement :

— **LLaMA (Meta) [33] :** Grand modèle de langage open-source, disponible en plusieurs tailles. Idéal pour la génération de texte contextuelle et l'assistance dans les applications métiers.

— **Phi-3 (Microsoft)[1] :** Modèle performant optimisé pour les environnements professionnels, capable de traiter des instructions complexes et d'être affiné sur des données internes.

— **MPT (MosaicML) [22] :** Modèle flexible pour la génération de texte, entraînable sur des corpus spécifiques pour répondre à des besoins métiers précis.

— **Falcon [4] :** Modèle open-source à haute performance, adapté pour des tâches conversationnelles et d'assistance technique.

Avantages principaux :

1. **Souveraineté des données :** Les conversations et informations confidentielles ne quittent jamais l'infrastructure interne.
2. **Personnalisation avancée :** Les modèles peuvent être *fine-tunés* sur les documents internes, guides utilisateurs et procédures spécifiques de l'entreprise.
3. **Maîtrise de l'infrastructure :** Déploiement flexible sur serveurs locaux, virtualisés ou cloud privé.
4. **Intégration directe :** Ces modèles peuvent être intégrés dans des applications, fournir un support contextuel et intelligent.

4.2.2.5 Notre architecture pour Chatbot

L'architecture ci-dessous illustre l'intégration du chatbot au sein de Protoweb, montrant le flux des requêtes depuis l'interface utilisateur jusqu'au modèle de langage auto-hébergé et le retour des réponses

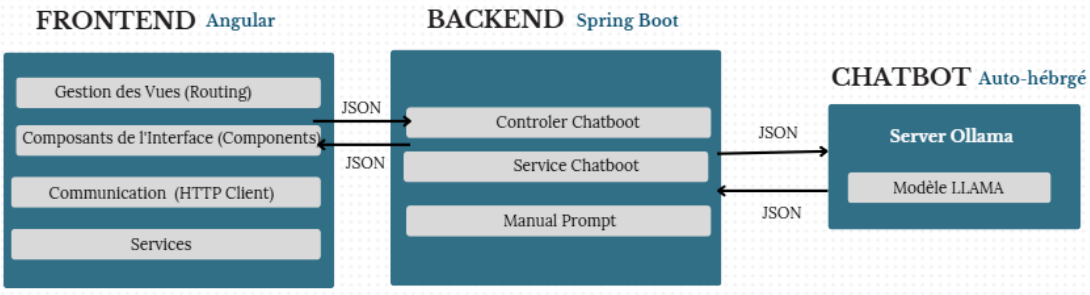


FIGURE 4.1 – Architecture d'intégration du Chatbot auto-hébergé dans Protoweb.

4.2.2.6 Décision et justification pour Protoweb

Au vu de cette analyse, le choix d'une **solution auto-hébergée** s'est imposé comme une évidence. La garantie que les conversations des utilisateurs ne transitent jamais par un service externe est un prérequis fondamental pour Safran.
Le modèle **Llama de Meta** [33]représente l'équilibre parfait entre performance et sécurité. Il nous offre la puissance d'un LLM moderne tout en nous permettant de le déployer sur une infrastructure interne.
Cette démarche assure une confidentialité totale des échanges, en accord avec les standards de sécurité les plus stricts.
Nous avons utilisé des outils comme Ollama [13] pour faciliter le déploiement local de ces modèles, offrant ainsi une maîtrise complète sur l'environnement d'exécution du chatbot.

4.2.3 Spécification fonctionnelle

Cette section décrit en détail les fonctionnalités à implémenter lors du Sprint 1, en précisant les besoins des utilisateurs, les interactions attendues avec le système, ainsi que les exigences spécifiques liées à l'intégration de l'IA.

4.2.3.1 Sprint Backlog

Ce Sprint a été planifié avec un backlog orienté vers la mise en place technique et fonctionnelle du chatbot, comprenant l'installation de l'environnement local, l'intégration du modèle et la création de l'interface utilisateur.

TABLE 4.3 – Backlog du Sprint 7

Tâche	**Description**	**Complexité (Points)**	**Estimation (jours)**
US 8.1	Création d'une interface utilisateur simple pour interagir avec le chatbot (fenêtre de discussion, zone de saisie).	5	3
US 8.2	Installation et configuration d'un serveur local pour héberger le modèle de langage.	8	5

Tâche	Description	Complexité (Points)	Estimation (jours)
US 8.3	Intégration et déploiement du modèle de langage (LLM) sur l'infrastructure locale.	13	7
US 8.4	Rédaction d'un manuel d'utilisation décrivant l'installation, la configuration et l'utilisation du chatbot.	3	5

4.2.3.2 Diagramme de cas d'utilisation global du Sprint 1

Le diagramme de cas d'utilisation ci-dessous présente une vue d'ensemble de l'intégration du chatbot.

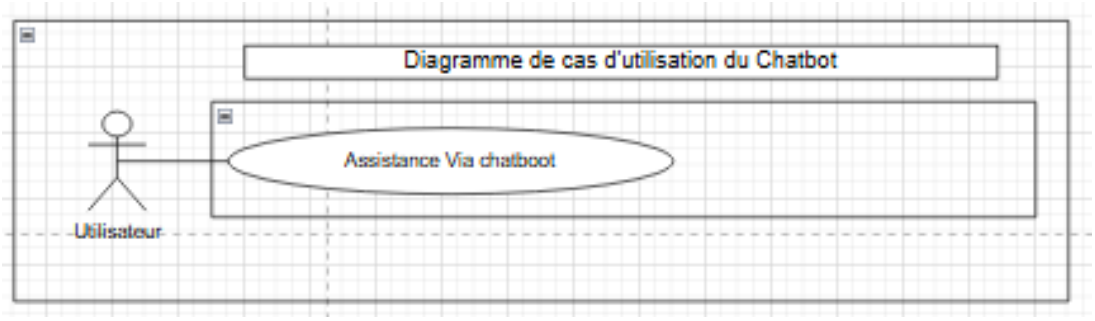


FIGURE 4.2 – Diagramme de cas d'utilisation du Chatbot

Le diagramme met en évidence le cas d'utilisation principal : "Interagir avec le Chatbot". Les utilisateurs peuvent initier une conversation, poser des questions, recevoir des réponses et consulter l'historique de la discussion. L'inclusion du chatbot est pensée pour être accessible à tous les rôles afin de maximiser l'assistance.

4.2.4 Raffinement des diagrammes de cas d'utilisation

Pour détailler l'intégration du chatbot, voici la description d'un cas d'utilisation représentatif et un diagramme de séquence.

4.2.4.1 Analyse de cas "Interagir avec le Chatbot"

Cette analyse présente le processus détaillé par lequel un utilisateur pose une question au chatbot et reçoit une réponse.

4.2.4.1.1 Description textuelle de Cas d'Utilisation : Interagir avec le Chatbot

Ce scénario décrit le processus standard par lequel un utilisateur obtient de l'aide via le chatbot.

TABLE 4.4 – Description du scénario « Interagir avec le Chatbot »

Élément du cas d'utilisation	Détails
Cas d'utilisation	Interagir avec le Chatbot

Élément du cas d'utilisation	Détails
Acteur	Utilisateur, Designer, Admin
Objectif	Obtenir une réponse pertinente aux questions posées sur l'application Protoweb.
Pré-condition	L'utilisateur est connecté et le panneau du chatbot est ouvert.
Post-condition	La question de l'utilisateur et la réponse du chatbot sont affichées dans la conversation.
Scénario nominal	1. L'utilisateur ouvre le panneau du chatbot depuis l'interface. 2. Il saisit une question dans le champ de texte et clique sur « Envoyer ». 3. La question est ajoutée à l'historique affiché sur l'interface. 4. Le système vérifie si la question correspond à une entrée du manuel prompt. - Si la question est trouvée dans le manuel, elle est transmise au modèle LLaMA (Ollama) pour générer une réponse dynamique. - Si la question n'est pas trouvée dans le manuel, une réponse statique ou hors sujet est renvoyée directement à l'utilisateur, sans passer par LLaMA. 5. Le modèle LLaMA analyse la question et génère une réponse appropriée (si applicable). 6. La réponse finale, dynamique ou statique, est renvoyée à l'interface utilisateur et affichée dans l'historique de la conversation.
Scénario alternatif	- **Question vide :** Si l'utilisateur tente d'envoyer une question vide, un message d'erreur s'affiche. - **Erreur système :** En cas d'indisponibilité du modèle ou d'erreur interne, un message générique informe l'utilisateur de l'impossibilité de traiter sa demande.

4.2.5 Réalisation du Sprint 1

La phase de développement du Sprint 1 a donné lieu à l'intégration de l'interface du chatbot et à la mise en place de son backend.

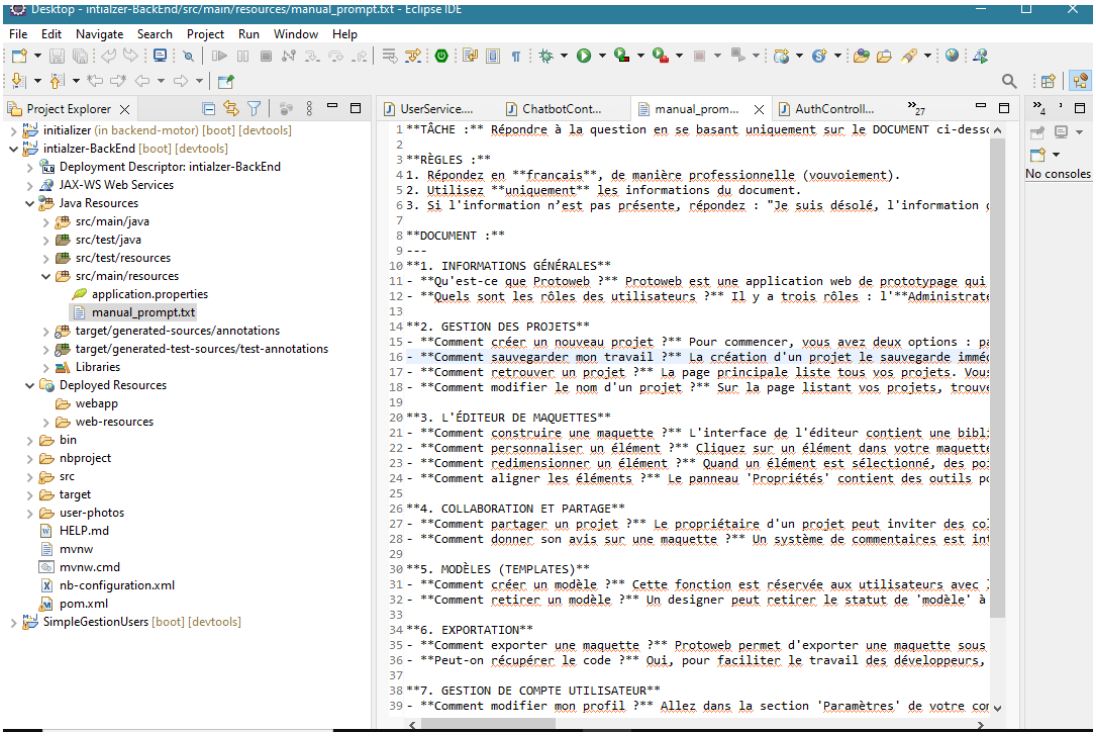


FIGURE 4.3 – Interface du manual prompt.

Cette image montre la configuration d'un chatbot, définissant ses règles de réponse via un fichier texte au sein d'un projet de développement backend.

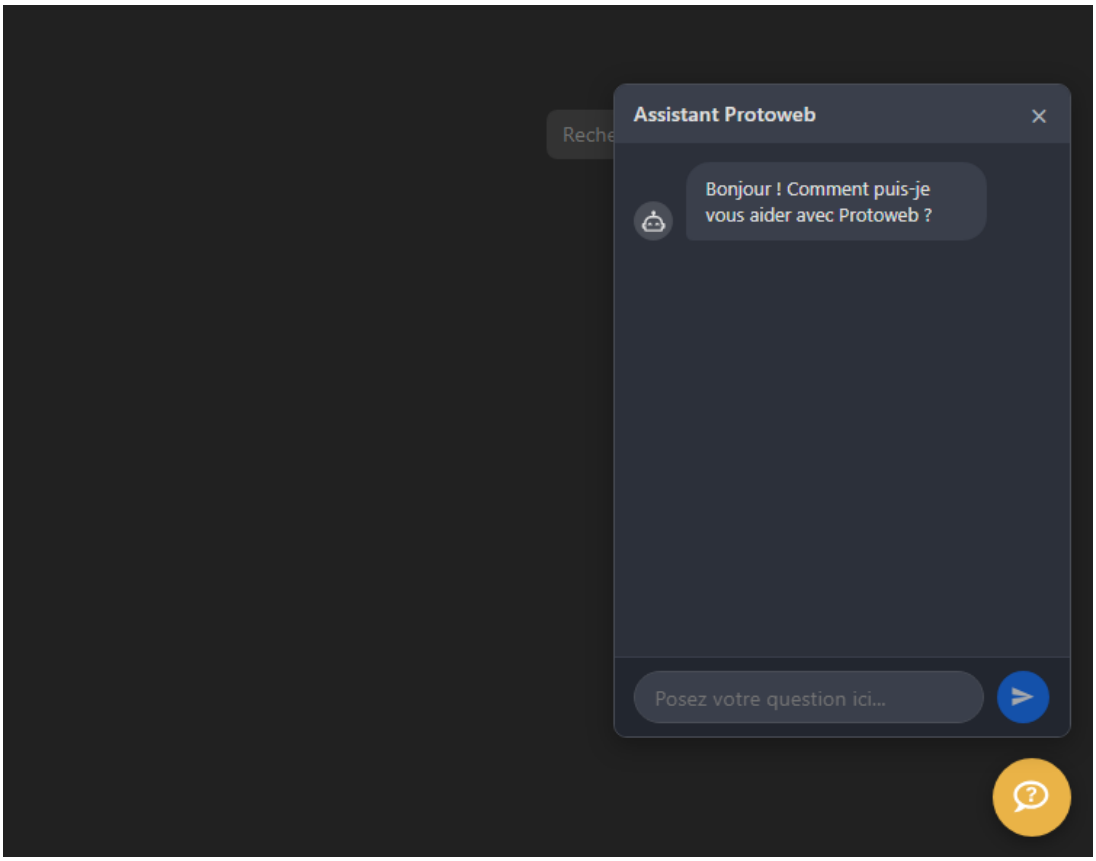


FIGURE 4.4 – Interface du Chatbot intégré.

Cette figure présente l'interface utilisateur du chatbot intégré à l'application Protoweb. On y voit le champ de saisie pour les questions, le bouton d'envoi et la zone d'affichage de l'historique des conversations, permettant une interaction fluide et intuitive.
L'icône du chatbot est également visible, permettant de l'ouvrir ou le fermer selon le besoin.

4.2.6 Phase de Test

La phase de test vise à évaluer les performances et la fiabilité du chatbot intégré à l'application Protoweb. Les principaux critères d'évaluation sont présentés dans le tableau ci-dessous.

TABLE 4.5 – Caractéristiques et critères de réussite du chatbot

Caractéristique	**Description**	**Critère de réussite**
Exactitude des réponses	Pourcentage de réponses correctes ou pertinentes par rapport aux questions posées	$\geq$ 85% des réponses correctes
Temps de réponse	Durée entre l'envoi d'une question et l'affichage de la réponse	$\leq$ 5 secondes pour les réponses LLaMA, immédiat pour le manuel prompt

Caractéristique	Description	Critère de réussite
Taux de réponses réussies	Pourcentage de questions traitées sans erreur, incluant les réponses prédéfinies et générées	100% des questions traitées
Gestion des erreurs	Capacité à gérer les questions vides, hors sujet ou en cas de panne du serveur	Tous les cas d'erreurs affichent un message approprié, sans blocage

4.2.6.1 Résultats des tests

Pour valider le bon fonctionnement du chatbot, plusieurs tests ont été réalisés sur les critères principaux : exactitude des réponses, temps de réponse, taux de réponses réussies et gestion des erreurs.

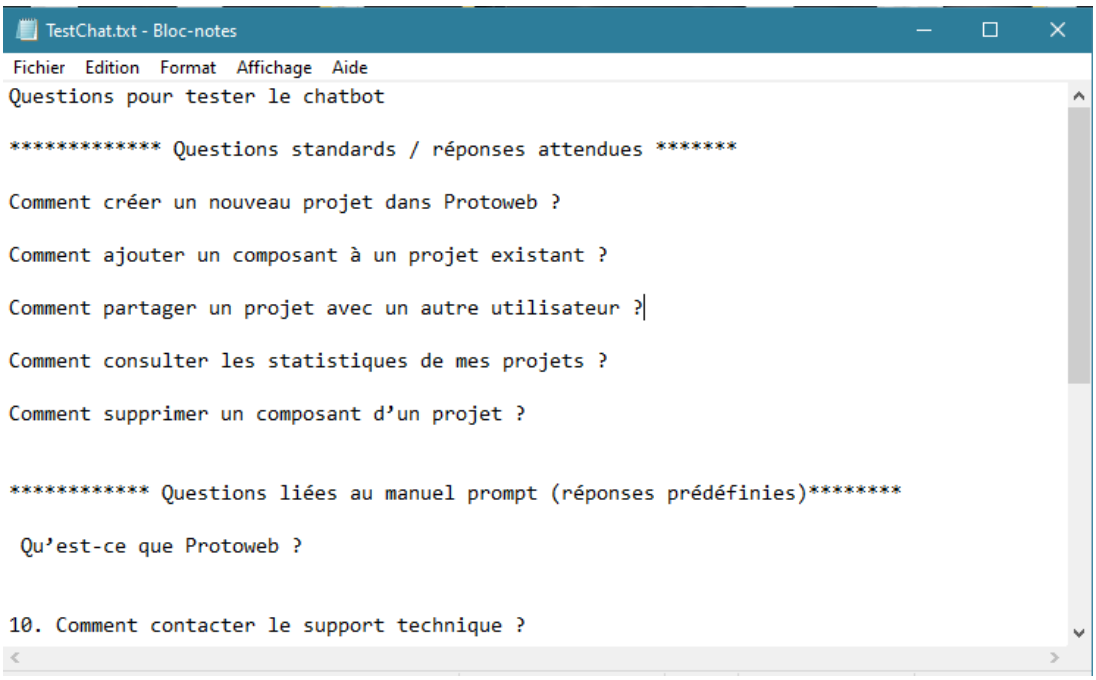


FIGURE 4.5 – Liste des questions de test.

Cette image montre la configuration d'un chatbot, définissant ses règles de réponse via un fichier texte au sein d'un projet de développement backend.

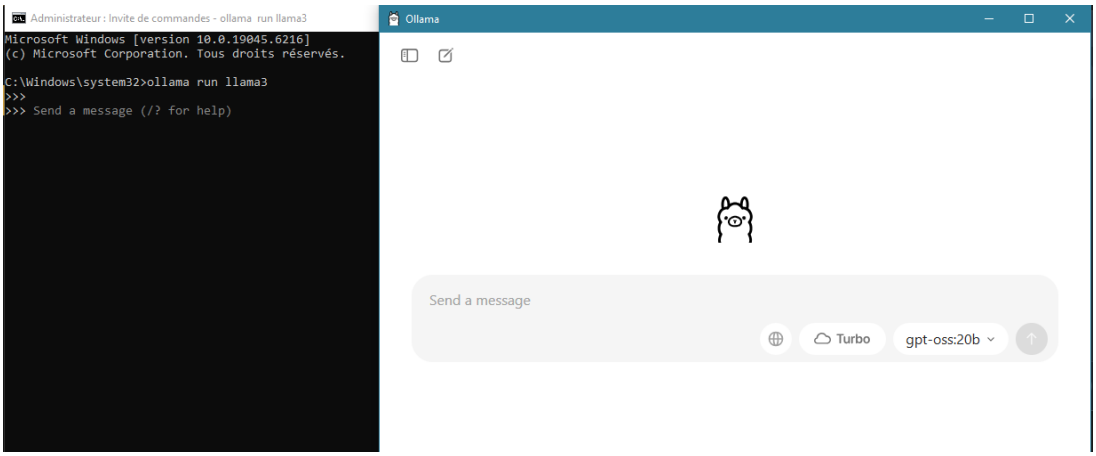


FIGURE 4.6 – CLI et Interface Graphique d'Ollama.

L'image illustre le lancement du modèle Llama3 via Ollama depuis l'invite de commande Windows, avec l'interface utilisateur graphique d'Ollama prête à faciliter l'interaction avec le modèle d'IA.

- **Exactitude des réponses :**
  Sur un ensemble de 20 questions, 18 réponses ont été jugées correctes et pertinentes.
  Résultat : 90% → critère de réussite atteint.
- **Temps de réponse :**
  Les réponses issues du manuel prompt sont quasi instantanées, tandis que les réponses générées par LLaMA sont en moyenne de 4 secondes.
  Résultat : $\leq$ 5 secondes → critère de réussite atteint.
- **Taux de réponses réussies :**
  Toutes les questions ont reçu une réponse, soit prédéfinie, soit générée par LLaMA.
  Résultat : 100% → critère de réussite atteint.
- **Gestion des erreurs :**
  Les questions vides affichent un message d'erreur, les questions hors sujet renvoient un message statique et une panne simulée du serveur LLaMA affiche un message d'indisponibilité approprié.
  Résultat : Tous les cas gérés → critère de réussite atteint.

Les tests montrent que le chatbot répond correctement, avec un temps de réponse satisfaisant et un taux de réussite complet. La gestion des erreurs est également appropriée, ce qui valide son bon fonctionnement

4.3 Sprint 2 : Exécution des tests finaux et préparation du déploiement

4.3.1 Présentation du Sprint 2

Le dernier Sprint a marqué la phase finale avant la livraison de Protoweb. Il a permis de consolider l'application en corrigeant les dernières anomalies, en optimisant les performances et en assurant le refactoring du code. Parallèlement, les artefacts de production ont été générés et validés avec les équipes IT de Safran en vue du déploiement. Des tests utilisateurs en conditions réelles ont confirmé

la conformité de la solution, tandis que la documentation technique et fonctionnelle a été finalisée afin de garantir une mise en production fluide et une maintenance durable.

4.3.2 Spécification fonctionnelle

4.3.2.1 Sprint Backlog

Le backlog de ce sprint a regroupé des tâches techniques essentielles pour la maturité du produit, ainsi que des activités de déploiement et de validation.

Table 4.6 – Backlog du Sprint 8

Tâche	**Description**	**Complexité (Points)**	**Estimation (jours)**
US 9.1	Correction des anomalies visuelles et fonctionnelles résiduelles, optimisation des performances (vitesse de chargement, réactivité de l'éditeur et du chatbot), refactoring du code pour une meilleure maintenabilité, et génération des artefacts de production (Angular, Spring Boot, scripts SQL).	5	1
US 9.2	Déploiement de l'application sur un serveur de test Safran avec configuration de l'environnement, suivi de l'exécution de la campagne de tests finaux et de validation fonctionnelle.	13	4

4.3.3 Processus de Réalisation

4.3.3.1 Phase de Stabilisation Pré-Déploiement

Cette étape a été consacrée à l'assainissement de la base de code et à l'amélioration de l'expérience utilisateur. Les efforts se sont concentrés sur la correction des anomalies résiduelles et sur l'optimisation des performances de l'application pour garantir sa fluidité.

— **Correction des bugs :**
Identification et résolution des anomalies visuelles, fonctionnelles et de comportement, incluant celles détectées sur l'interface et les réponses du chatbot.

— **Optimisation des performances :**
Réduction du temps de chargement initial, amélioration de la réactivité de l'éditeur et des interactions avec le chatbot.

— **Refactoring du code :**
Nettoyage, commentage et restructuration du code pour améliorer sa lisibilité, sa maintenabilité et faciliter les futures évolutions.

Résultats de la Stabilisation :
Ce travail de fond a permis d'obtenir une version de l'application significativement plus robuste et performante. Les améliorations clés incluent :

— **Stabilité accrue :**
Les erreurs connues ont été éliminées et le comportement de l'application est devenu prévisible et fiable.

— **Fluidité améliorée :**
Le temps de chargement initial a été réduit et les actions au sein de l'éditeur sont devenues plus rapides.

— **Code assaini :**
La base de code a été nettoyée, commentée et structurée pour de futures évolutions, réduisant la dette technique.

4.3.3.2 Déploiement sur le Serveur de Test

Pour rendre Protoweb accessible aux testeurs, nous avons suivi le processus de déploiement interne de Safran :

1. **Préparation des artefacts :**
Génération des paquets de production de l'application, incluant :
2. **Coordination avec les équipes IT :**
En collaboration avec l'équipe d'infrastructure de Safran, les artefacts ont été transférés sur le serveur de test.
3. **Mise en service :**
L'application a été lancée sur le serveur et rendue accessible via une URL interne dédiée aux tests.
Le modèle LLaMA (Ollama) a été démarré et configuré sur le serveur local désigné.

4.3.3.3 Campagne de Tests Finaux et Validation Finale du Projet

Une fois l'application en ligne sur le serveur de test, une session de tests a été organisée pour valider l'ensemble du projet.

— **Participants :**
Le Product Owner, ainsi que plusieurs futurs utilisateurs ont participé à cette session.

— **Objectif :**
Il s'agissait de valider que l'application répondait bien aux besoins métier, qu'elle était intuitive, performante.

— **Déroulement :**
Les participants ont été invités à réaliser plusieurs scénarios réels, incluant la création de prototypes, l'utilisation des outils collaboratifs, et l'interaction avec le chatbot pour obtenir de l'aide sur l'application.

4.3.3.3.1 Résultats et Validation Finale : La campagne de tests s'est conclue avec succès. Les retours des utilisateurs ont été très positifs, confirmant la fluidité de l'application et la pertinence de l'assistance fournie par le chatbot. Le projet a ainsi atteint tous ses objectifs et est considéré comme validé pour une mise en production.

4.4 Conclusion

Au terme de cette deuxième et dernière release, le projet Protoweb a atteint sa pleine maturité et son objectif final. Le Sprint 1 a permis d'intégrer avec succès un chatbot d'assistance basé sur un modèle de langage auto-hébergé, garantissant ainsi une aide intelligente et sécurisée pour les utilisateurs, tout en respectant les impératifs de souveraineté des données de Safran. Le Sprint 2 a consolidé l'ensemble de l'application à travers des tests finaux rigoureux et une préparation méticuleuse au déploiement. L'application Protoweb est désormais un outil complet, stable, performant, documenté et prêt à être livré. Le cycle de développement Agile est bouclé, et le produit final est prêt à être déployé chez Safran pour transformer efficacement la manière dont les maquettes sont conçues et gérées en interne.

Conclusion et Perspectives

Conclusion

Ce projet de fin d'études arrive à son terme. Au départ, notre objectif était de trouver une solution au défi de communication entre les équipes techniques et les clients internes de Safran, qui avaient du mal à décrire leurs besoins pour les interfaces, entraînant de nombreux allers-retours et ralentissant les projets. Pour y répondre, nous avons développé Protoweb, une plateforme web de prototypage par glisser-déposer, permettant à un utilisateur, même sans compétences techniques, de construire visuellement une maquette, de personnaliser les éléments et de partager son travail pour collaborer en équipe. Le développement a été mené selon la méthodologie Agile Scrum, ce qui nous a permis de construire l'application étape par étape, en livrant des fonctionnalités concrètes à chaque fin de cycle. Nous avons ainsi mis en place un système d'authentification sécurisé, un éditeur de maquettes intuitif, des outils de collaboration et un tableau de bord pour l'administrateur. L'aboutissement de ce projet n'est pas seulement une application fonctionnelle, mais une solution déployée sur un serveur de test de Safran et validée par les futurs utilisateurs, dont les retours positifs lors des tests d'acceptation ont confirmé que Protoweb répondait bien à la problématique initiale. D'un point de vue personnel, ce projet a été une expérience très enrichissante, me permettant de mettre en pratique des compétences techniques solides sur des technologies comme Angular et Spring Boot, mais aussi de gérer un projet de A à Z, en suivant une méthodologie Agile et en communiquant régulièrement avec le client.

Perspectives

Bien que Protoweb réponde aujourd'hui aux objectifs fixés, plusieurs pistes d'amélioration peuvent être envisagées pour l'avenir. Ces perspectives peuvent être classées en fonction de leur priorité.

À court terme :
La prochaine étape logique est le déploiement de l'application en production pour la rendre accessible à toutes les équipes de Safran. Il faudra accompagner ce lancement par la formation des utilisateurs et la collecte de leurs premiers retours en conditions réelles.

À moyen terme :
Une fois l'application adoptée, de nouvelles fonctionnalités pourraient être ajoutées pour enrichir l'expérience :

- **Élargir la bibliothèque de composants :** Ajouter des éléments plus complexes (graphiques, calendriers, etc.) et permettre aux designers de créer et partager leurs propres composants personnalisés.
- **Améliorer la simulation :** Ajouter des outils pour simuler l'affichage des maquettes sur différents appareils, comme les tablettes et les mobiles.
- **Intégrer un historique des versions :** Permettre aux utilisateurs de sauvegarder différentes versions d'un projet et de revenir à une version antérieure si besoin.

À long terme :
À plus long terme, Protoweb pourrait devenir un pilier de la conception d'applications chez Safran. Il pourrait s'intégrer avec d'autres outils internes de gestion de projet (comme Jira ou Trello) pour lier directement les maquettes aux tâches de développement, fluidifiant ainsi encore plus le cycle de vie des projets logiciels au sein de l'entreprise.

Annexe A : Guide Utilisateur de Protoweb

Description

Ce guide fournit un aperçu complet des fonctionnalités de la plateforme Protoweb. Il est destiné à tous (utilisateurs, Designers, Administrateurs) pour faciliter leur prise en main de l'outil.

1. Informations Générales

— **Qu'est-ce que Protoweb ?**
Protoweb est une application web de prototypage qui permet de créer des maquettes d'interfaces en mode glisser-déposer. Elle a été conçue pour fluidifier la communication entre les équipes de design et les clients internes de Safran, même s'ils n'ont pas de compétences techniques.

— **Quels sont les rôles ?**
Il y a trois rôles :
Administrateur gère les comptes et supervise la plateforme.
Designer peut créer des projets et des modèles réutilisables.
Utilisateurs peut créer des projets pour exprimer ses besoins.

2. Gestion des Projets

— **Créer un nouveau projet :** Vous avez deux options : partir d'une 'page blanche' pour une créativité totale, ou sélectionner un 'modèle' (template) déjà préparé par un designer pour aller plus vite.

— **Sauvegarder son travail :** La création d'un projet le sauvegarde immédiatement. Pour enregistrer des modifications ultérieures, vous devez cliquer sur le bouton de sauvegarde dédié.

— **Retrouver un projet :** La page principale liste tous vos projets. Vous pouvez utiliser la barre de recherche pour trouver un projet par son nom, et trier la liste par nom ou par date.

— **Modifier le nom d'un projet :** Cliquez sur le menu d'options (trois points) associé au projet, puis sélectionnez 'Renommer'.

3. L'Éditeur de Maquettes

— **Construire une maquette :** Glissez un composant (bouton, texte, etc.) de la bibliothèque latérale et déposez-le sur votre zone de travail.

— **Personnaliser un élément :** Cliquez sur un élément pour ouvrir le panneau 'Propriétés' à droite. Vous pourrez y modifier sa couleur, sa taille, son texte, sa bordure, etc.

— **Redimensionner un élément :** Quand un élément est sélectionné, utilisez les poignées sur ses coins et ses bords pour l'ajuster avec la souris.

4. Collaboration et Partage

— **Partager un projet :** En tant que propriétaire, vous pouvez inviter des collaborateurs et leur donner les droits 'peut voir' (lecture seule) ou 'peut modifier' (écriture).

— **Donner son avis :** Un système de commentaires est intégré pour permettre aux collaborateurs de laisser des remarques directement sur la maquette.

5. Gestion de Compte et Fonctionnalités Spécifiques

— **Modifier son profil :** Dans la section 'Paramètres', vous pouvez changer votre nom, votre photo de profil et votre mot de passe.

— **Créer un modèle (pour Designers) :** Un Designer peut promouvoir l'un de ses projets pour le transformer en modèle réutilisable par tous.

— **Exporter une maquette :** Protoweb permet d'exporter une maquette en image (PNG, JPG), en PDF, ou de récupérer le code source (HTML/CSS).

— **Gérer la plateforme (pour Admins) :** L'administrateur peut gérer les comptes utilisateurs et consulter un tableau de bord statistique.

Annexe B : Spécification des Routes Principales de l'API REST

Description
Cette section documente le contrat d'interface (API) entre le client (Angular) et le serveur (Spring Boot).
Une API bien définie est essentielle pour le développement découplé.
Le tableau ci-dessous présente les routes principales, les méthodes HTTP associées et leur fonction.

Table 7 – Tableau des routes principales de l'API Protoweb

Méthode HTTP	URL	Rôle Requis	Description
POST	`/api/auth/login`	Utilisateur	Authentifie un utilisateur.
GET	`/api/projects`	Utilisateur	Récupère la liste de tous les projets accessibles par l'utilisateur connecté.
POST	`/api/projects`	Utilisateur	Crée un nouveau projet vierge pour l'utilisateur connecté.
DELETE	`/api/projects/{id}`	Propriétaire	Supprime un projet. L'accès est restreint au propriétaire.
POST	`/api/projects/{id}/share`	Propriétaire	Partage un projet avec un autre utilisateur.
GET	`/api/admin/users`	Admin	Récupère la liste de tous les utilisateurs de la plateforme.
POST	`/api/admin/users`	Admin	Crée un nouveau compte utilisateur.
POST	`/api/chatbot/ask`	Utilisateur	Envoie une question au service de chatbot et retourne la réponse.
GET	`/api/layouts/my-layouts`	Utilisateur	Récupère tous les layouts accessibles de l'utilisateur connecté.
POST	`/api/layouts`	Utilisateur	Crée un nouveau layout.
PUT	`/api/layouts/{id}`	Propriétaire	Met à jour un layout existant.

Méthode HTTP	URL	Rôle Requis	Description
PUT	`/api/layouts/{id}/name`	Propriétaire	Met à jour le nom du layout.
DELETE	`/api/layouts/{id}`	Propriétaire	Supprime un layout.
GET	`/api/layouts/templates`	Utilisateur	Liste tous les layouts marqués comme templates.
POST	`/api/layouts/{layoutId}/mark-as-template`	Designer	Marque un layout comme template.
POST	`/api/layouts/{layoutId}/unmark-as-template`	Designer	Retire le statut template d'un layout.
POST	`/api/from-layout-template/{sourceLayoutId}`	Utilisateur	Crée un nouveau layout basé sur un template existant.
POST	`/api/layouts/{layoutId}/share`	Propriétaire	Partage un layout avec un autre utilisateur.
GET	`/api/layouts/{layoutId}/shares`	Propriétaire	Récupère les partages d'un layout.
PUT	`/api/layouts/shares/{permissionId}`	Propriétaire	Met à jour les permissions d'un partage.
DELETE	`/api/layouts/shares/{permissionId}`	Propriétaire	Supprime un partage.
GET	`/api/layouts/admin/all`	Admin	Récupère tous les layouts.
GET	`/api/layouts/{layoutId}/comments`	Utilisateur	Récupère tous les commentaires d'un layout.
POST	`/api/layouts/{layoutId}/comments`	Utilisateur	Ajoute un commentaire à un layout.
PUT	`/api/comments/{commentId}`	Propriétaire	Met à jour un commentaire existant.
DELETE	`/api/comments/{commentId}`	Propriétaire	Supprime un commentaire.
GET	`/api/admin/stats/users/active-count-24h`	Admin	Nombre d'utilisateurs actifs sur les dernières 24h.
GET	`/api/admin/stats/users/recent`	Admin	Derniers utilisateurs inscrits.
GET	`/api/admin/stats/users/total-count`	Admin	Nombre total d'utilisateurs.
GET	`/api/admin/stats/layouts/by-department`	Admin	Nombre de layouts par département.
GET	`/api/admin/stats/layouts/total-count`	Admin	Nombre total de layouts.

Méthode HTTP	URL	Rôle Requis	Description
GET	`/api/admin/stats/shares/total-count`	Admin	Nombre total de partages actifs.

Bibliographie

[1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Hany Awadallah, Nguyen Bach Awadalla, Amit Bahree, Arash Bakhtiari, Harkirat Behl, and et al. Phi-3 technical report : A highly capable language model locally on your phone. *arXiv preprint arXiv :2404.14219*, 2024.

[2] Adobe. Guide de l'utilisateur – adobe xd. `https://helpx.adobe.com/fr/xd/user-guide.html`. [Dernière mise à jour : 7 juin 2023].

[3] Bayan A. Alazzam, Manar Alkhatib, and Khaled Shaalan. Artificial intelligence chatbots : A survey of classical versus deep machine learning techniques. *Information Sciences Letters*, 12 :1217, 2023.

[4] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models. *arXiv preprint arXiv :2311.16867*, 2023.

[5] Anon. Angular documentation. `https://angular.io/docs`.

[6] Anon. Building an ai chatbot : Types, tech stacks, and steps — valuecoders.com. `https://www.valuecoders.com/blog/ai-ml/building-ai-chatbot-types-tech-stacks-steps/`.

[7] Anon. Chatbot et ia générative : Le guide complet pour 2024 — lead-ia.com. `https://www.lead-ia.com/blog/chatbot-et-ia-generative-le-guide-complet-pour-2024`.

[8] Anon. Figma : The collaborative interface design tool — figma.com. `https://www.figma.com/`.

[9] Anon. Figma vs adobe xd : Comparison guide | zero to mastery — zerotomastery.io. `https://zerotomastery.io/blog/figma-vs-adobe-xd-comparison-guide/`.

[10] Anon. Flowchart maker & online diagram software — app.diagrams.net. `https://app.diagrams.net/`.

[11] Anon. Microsoft sql server — microsoft.com. `https://www.microsoft.com/fr/sql-server`.

[12] Anon. Nlp vs llm — yonyx.com. `https://corp.yonyx.com/customer-service/nlp-vs-llm/`.

[13] Anon. Ollama — ollama.com. `https://ollama.com/`.

[14] Anon. Postman : The world's leading api platform | sign up for free — postman.com. `https://www.postman.com/`.

[15] Anon. Qu'est-ce que le traitement automatique du langage naturel ? – le nlp expliqué – aws — aws.amazon.com. `https://aws.amazon.com/fr/what-is/nlp/`.

[16] Anon. Spring boot — spring.io. `https://spring.io/projects/spring-boot`.

[17] Anon. Tunisie — safran-group.com. `https://www.safran-group.com/fr/pays/tunisie`.

[18] Anon. Visual studio code - code editing. redefined — code.visualstudio.com. `https://code.visualstudio.com/`.

[19] Anon. What is artificial intelligence (ai) ? | ibm — ibm.com. https://www.ibm.com/think/topics/artificial-intelligence.

[20] Anon. What is three-tier architecture ? | ibm — ibm.com. https://www.ibm.com/think/topics/three-tier-architecture.

[21] Anon. Scrum methodology. International Journal of Engineering and Computer Science (IJECS), 2016. [Online ; accessed 29-08-2025].

[22] Cody Blakeney, Mansheej Paul, Brett W. Larsen, Sean Owen, and Jonathan Frankle. Does your data spark joy ? performance gains from domain upsampling at the end of training. *arXiv preprint arXiv :2406.03476*, 2024.

[23] Canva Team. Canva docs – éditeur de documents en ligne gratuit et intuitif. https://www.canva.com/fr_fr/docs/, 2025.

[24] Google. Google gemini. https://gemini.google.com/app?hl=fr, 2025.

[25] Google DeepMind, Gemini Team. Gemini 1.5 : Unlocking multimodal understanding across millions of tokens of context. https://storage.googleapis.com/deepmind-media/gemini/gemini_v1_5_report.pdf, 2024.

[26] JetBrains. Intellij idea — ide professionnel pour java et kotlin. https://www.jetbrains.com/idea/, 2025.

[27] JGraph Ltd. diagrams.net. https://app.diagrams.net, 2025.

[28] Luckiwi. Méthodes de gestion de projet : Classique vs agile – comparatif — luckiwi.com. https://www.luckiwi.com/fr/blog/article/classical-vs-agile-methodology.

[29] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models : A survey. *arXiv preprint arXiv :2402.06196*, 2024.

[30] OpenAI. Chatgpt. https://chatgpt.com/, 2025.

[31] Saba M. Qadhi, Ahmed Alduais, Youmen Chaaban, and Majeda Khraisheh. Generative ai, research ethics, and higher education research : Insights from a scientometric analysis. *Information*, 15(6) :325, 2024.

[32] Daniel Schifano. Figma vs adobe xd : Comparison guide. https://zerotomastery.io/blog/figma-vs-adobe-xd-comparison-guide/, 2024.

[33] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama : Open and efficient foundation language models. *arXiv preprint arXiv :2302.13971*, 2023.